



{ LOGIC, FOR, PROGRAMMERS }

HILLEL WAYNE

Logic for Programmers

(version 0.11.1)

Hillel Wayne

Aug 06, 2025

Acknowledgements

Thanks to Tsvetan Tsvetanov, Predrag Gruevsky, Jeremy Kun, Saul Pwanson, Yeray Cabello, Igor Kuvychko, Puikei Cheng, Daniel Prager, Sviatoslav Abakumov, Harald M Müller, and Marianne Belloti for feedback on content. Thanks to David Mazarro, Jeroen Heijmans, Ophelia Stevens, Oyendrila Dobe, Nirmalya Sengupta, Mike Mull, and Marcus Millin for identifying errors in the text.

Thanks to Tikhon Jelvis for some advice on covering constraint solvers.

Thanks to Zac Hatfield Dodds for helping with Sphinx and Alexey Zubkov for advice on typesetting.

Tim Nelson, Marianne Belloti, and Saul Pwanson all helped me keep on schedule at various points in this book's production.

Contents

1	Intro	1
1.1	Beta Notes	1
1.2	New in v0.11:	1
1.3	Why this book	2
1.4	Design Philosophy	3
1.5	How to Read This Book	3
2	A Crash Course in Logic	5
2.1	Predicates	5
2.2	Sets	10
2.3	Quantifiers	13
2.4	Notation	16
2.5	In Practice: Rewrite Rules	18
2.6	Summary	21
3	Refactoring Code	22
3.1	Simplifying Conditionals	22
3.2	Refactoring with Quantifiers	24
3.3	Programs are not Math	29
3.4	Using sets	30
3.5	Summary	31
4	Writing Better Tests	32
4.1	Strong and Weak Tests	32
4.2	In Practice: Property-Based Testing	35
4.3	Notes on Property Testing	37
4.4	Summary	40
5	Functional Correctness	41
5.1	Assertions	41
5.2	Contracts	42
5.3	Contracts vs Types	46
5.4	Polymorphism and Refactoring	48
5.5	Summary	51
6	Proving Code Correct	53
6.1	What is a proof?	53
6.2	Proofs	54
6.3	Formal Verification	58
6.4	Summary	61
7	Case Analysis	63
7.1	Decision Tables	63

7.2	Another Requirements Example	66
7.3	Analyzing Code	67
7.4	Techniques	68
7.5	When is a Table the Wrong Choice?	71
7.6	Summary	72
8	Databases	73
8.1	A Relational Model Overview	73
8.2	Querying Data	74
8.3	Database Constraints	78
8.4	Constraints Are Queries	80
8.5	Summary	83
9	Data Modeling	85
9.1	Abstracting from Data	85
9.2	In Practice: Formal Specification	86
9.3	Finding Bugs with Specifications	91
9.4	Summary	93
10	System Modeling	95
10.1	Situation	95
10.2	The Logic	95
10.3	In Practice: TLA+	100
10.4	Specification in the wild	105
10.5	Summary	106
11	Solvers	108
11.1	Logic	108
11.2	In Practice: Solvers	110
11.3	Which to use?	115
11.4	Summary	116
12	Logic Programming	117
12.1	Prolog	117
12.2	Deductive Databases	119
12.3	Constraint Logic Programming	121
12.4	Planning	121
12.5	Summary	123
A	Math Notation	125
A.1	Basic Logic Symbols	125
A.2	Quantified Expressions	126
A.3	Tautologies	126
B	Useful Rewrite Rules	127

B.1	Table of Tautologies	127
C	Beyond Logic	129
C.1	The Limit: Russell's Paradox	129
C.2	Higher Order Logic	130
C.3	Constructive Logic	130
C.4	Modal Logic	131
D	Answers to Exercises	132
	Index	143

Chapter 1

Intro

1.1 Beta Notes

I'm doing early access with this book, so this is all beta. Most of the material is now in, but I still need to polish and revise it, add more exercises, improve formatting, and incorporate reader feedback.

I welcome any and all comments. I'm particularly interested in:

1. Do the examples seem useful to you? Were the exercises helpful?
2. Which topics need the most focus?
3. What resources would be good to recommend as “further reading”?
4. What examples and new topics would you like to see?
5. What needs more exercises?

You can email me at h@hillelwayne.com. Thank you very much!

Note

Anything in a note box is a message from me to you as early readers. Things I'm uncertain about, things I plan to polish more, things that I plan to write, etc. `[[double braces]]` are similar. Feel free to throw comments my way!

1.2 New in v0.11:

- Brand new chapter, “Proving Code Correct”, covering proofs, loop invariants, formal verification
- Total rewrite of “Database” chapter:
 - Now covers database representations, relational model, queries, joins, and constraints
 - Two new executable SQL examples on constraints
 - One new image
- Total rewrite of “Functional Correctness”:
 - Now covers assertions, MISU, polymorphism, advice

- Loop invariants and formal verification moved to proofs chapter
- Total rewrite of “Case Coverage”, now called “Case Analysis”:
 - New introduction and motivating example
 - More material on analysing code with decision tables, techniques, when not to use DTs
 - Redundant examples removed
- Logic chapter improved, now covers the way-more-common scoped quantifiers before unscoped
- Fixed “symmetric difference” exercise
- Six exercises removed, eleven added (+5 total)
- Better format for proof tables and rewrite rules
- Some initial table of contents tweaks
- Fixed PDF bug: admonition sidebars now render correctly in Acrobat

1.3 Why this book

If I start a build at 3:05 PM and it takes 12 minutes to complete, when will the build be finished?

To answer this question, we need to know how to manipulate numbers. The mathematics of numbers is called *arithmetic*. Arithmetic shows us how to multiply two numbers, use fractions, determine which of two numbers is larger, and more.

If I have the conditional `if(sensor_offline || inactive)`, and I know for sure that `sensor_offline` is true, does the value of `inactive` matter?

To answer this question, we need to know how to manipulate booleans. The mathematics of booleans is called *logic*. Logic shows us how to simplify a boolean expression, use sets, determine if one statement is stronger than another, and more.

But there is one key difference between arithmetic and logic. We were taught arithmetic in elementary school. Few of us were formally taught logic. Most programmers pick up a little logic by osmosis, but even that rarely exposes people to anything beyond the basics.

This makes logic the single most useful topic in math a programmer can learn. But how are we supposed to learn it? There are plenty of books available written for philosophers, mathematicians, and computer scientists, who all have far more need for the theory than the practice. There are no books on logic meant for the

self-studying programmer, who is looking for practical skills useful in day-to-day work. It is as if nobody will teach us how to ride a bicycle, only how to build one.

That is the goal of this book. I aim to teach you the basics of logic and how to apply it to various everyday software problems, like testing code, designing a database, or working out customer requirements. By the end of this book, you will be comfortable manipulating logical expressions and have a greater understanding of all of the ways software uses logic, implicitly or not.

1.4 Design Philosophy

This book is meant specifically for programmers with little to no familiarity with formal math. In all cases, I opted for accessibility and ease-of-use over precision or rigor. This is a technical how-to, not a textbook.

1.4.1 Notation

Mathematics shares many operations in common with programming but uses different representations, such as writing “and” as $\wedge$ instead of `&&`. I will use programming terminology wherever possible. I included an [appendix](#) (page 125) which maps conventional programming symbols to math symbols.

In cases where math symbols don’t have common programming analogs (such as $\forall$), I have opted to use an explicit English equivalent (such as `all`).

Lastly comes the question of array indexing. Does the array `arr` start at `arr[1]` or `arr[0]`? There is no universal programming convention, as different languages make different choices. I would use the mathematician’s convention except that does not exist either: different branches of mathematics make different choices too! So I will default to 0-based indexing unless demonstrating a tool or language which uses 1-based indexing, which I will explicitly note.

1.5 How to Read This Book

I recommend first reading *A Crash Course in Logic* (page 5), and then moving to whichever technique looks most interesting. Techniques chapters are independent except when otherwise noted, in which case backreferences are provided.

The first five techniques (starting with *Refactoring Code* (page 22)) focus on how logic applies to everyday software. The last four (starting with *Data Modeling* (page 85))

cover special logic-based tools that unlock powerful new solutions to difficult software problems.

Large code samples are available online at <https://github.com/logicforprogrammers/book-assets>.

1.5.1 Exercises

Exercises are provided to help you check your knowledge and develop your skills further. All exercises have solutions in the back of the book. Some of the exercises have multiple possible solutions. Your answer can be correct even if it differs from the “official” solution!

Some questions involve writing short snippets of code. In these cases, use whatever language you like. I will personally give examples in Python or pseudocode. When writing Python, I have tried to make it as accessible as possible, meaning it does not do things in an idiomatic way.

Chapter 2

A Crash Course in Logic

Formal logic is a very powerful tool, but it's also very simple. Over this chapter, we'll motivate and explain all of the basic concepts and syntax. Much of it may already be familiar to you from programming experience!

2.1 Predicates

To a first approximation, a *predicate* is a function that returns a boolean. You have probably written dozens of predicates as a programmer. These are all predicates:

- `Positive(x)` is true if `x` is greater than 0
- `IsSum(x, y, z)` is true if `x` plus `y` equals `z`
- `RAMAtLeast(c, r)` is true if the computer `c` has at least `r` bytes of physical RAM.

I say *to a first approximation* because predicates are a mathematical concept, not a programming construct. A program function needs to come with a way of computing the answer, while a predicate simply defines what the answer is. Take `RAMAtLeast`: the software implementation would depend on the programming language, operating system, and possibly even user permissions. But the predicate? True if the computer has the RAM, false if not. That's it.

This means predicates can be more abstract than programming functions, expressing things that we don't yet know how to compute or even *can* compute. These are all valid predicates, too:

- `CanRunProgram(c)` is true if the computer `c` is capable of running our program, whatever "capable" ends up meaning
- `RainyDayInCa(date)` is true if on `date`, it rained somewhere in Canada
- `NotAlone()` is true if aliens are real.

[[That said, predicates don't *have* to be abstract. If we know how to compute the result of a predicate, there's nothing wrong with just implementing it! The power of predicates is that they can span the full range of abstraction.]] So let's introduce some syntax. If a predicate is abstract, I will wrap the body in `` backticks``:

```
# concrete
Positive(x) = x > 0
IsSum(x, y, z) = x + y == z
```

(continues on next page)

(continued from previous page)

```
# abstract
CanRunProgram(c) = `c can run our program`
```

This is not a common mathematician convention, but it's clear enough to programmers. To distinguish predicates from “ordinary functions” like `add_two`, predicates will always be `TitleCased` and functions will always be `snake_cased`.

2.1.1 A Practical Example

Predicates act as a bridge between how we talk about systems in a human language and how we encode them in a programming language. Let's come back to `CanRunProgram`. I included that example because I once saw a program with these requirements:

The computer must have enough RAM and a fast CPU or a good graphics card (GPU).

I found this confusing. The sentence sounds natural enough in English, but we can find a problem by formalizing with logic. We will start by first writing predicates for each subrequirement, like so:

```
RAM(c) = `c has enough RAM`
CPU(c) = `c has a fast CPU`
GPU(c) = `c has a good GPU`
```

These predicates are abstract because we don't know the specifics of what these mean. Is 64gb “enough RAM”? Is 32gb? The specifics don't matter for us, because this is already enough to write `CanRunProgram` as a concrete mathematical expression.

```
CanRunProgram(c) = RAM(c) && CPU(c) || GPU(c)
```

(Here we're using `&&` for AND and `||` for OR. This is just the convention for this book: you may see other resources use “and” and “or” or something else. Mathematicians use $\wedge$ and $\vee$. I'm *not* going to use these because they're not found on the keyboard. We'll also use `!` for “not”; mathematicians use $\neg$.)

Now the problem is clearer: is `a && b || c` supposed to be read as `(a && b) || c` or as `a && (b || c)`? The predicate is malformed and we have two different ways of making it make sense:

```
# way 1
CanRunProgram(c) = RAM(c) && (CPU(c) || GPU(c))
```

```
# way 2
CanRunProgram(c) = (RAM(c) && CPU(c)) || GPU(c)
```

Both interpretations make sense in English! But they have different outputs for some inputs. We can see this by listing every single possible combination of values for RAM/CPU/GPU, and see what they give for CanRunProgram. This is called a *truth table*.

R (RAM)	C (CPU)	G (GPU)	R && (C G)	(R && C) G
T	T	T	T	T
T	T	F	T	T
T	F	T	T	T
T	F	F	F	F
F	T	T	F	T
F	F	T	F	T
F	T	F	F	F
F	F	F	F	F

There are two combinations of inputs where one interpretation is false and the other is true. It's possible that the vendor *meant* the first interpretation when writing the requirements, but I *read* it as the second interpretation. Then I am sure that the program will run on my computer, the vendor never expects it to, and I get mad that they “lied” to me. Much better to express the requirement mathematically!

Expressing properties with formal logic is less ambiguous than with informal English. For the purpose of teaching, we will assume the intended predicate is $(\text{RAM}(c) \ \&\& \ \text{CPU}(c)) \ || \ \text{GPU}(c)$.

Tip

If you ever have trouble generating a truth table, you can try to use a truth table generator, like [here](https://web.stanford.edu/class/cs103/tools/truth-table-tool/)¹. Try $p \ || \ !q$ and experiment from there.

¹ <https://web.stanford.edu/class/cs103/tools/truth-table-tool/>

2.1.2 Conditional Predicates

Let’s now make a variation on our predicate. Some programs have a native version and a web version. The native version uses the local computer’s resources, the web version does most of the processing on some cloud computer somewhere. So the native version requires a beefy computer, but *any* computer can run the web client.

If a computer is running the native version, it must have enough RAM and a fast CPU or a good graphics card (GPU) to use this program. But if it’s not running the native version, you’re fine.

To model this, we’ll need a new predicate, `Native(p)`. `Native` is a property of the *program*, not the *computer*. `CanRunProgram` then depends on both the program and the computer.

```
CanRunProgram(c, p) = `true unless Native(p),
  in which case (RAM(c) && CPU(c)) || GPU(c)`
```

I used backticks here because half the predicate is still in informal English. It turns out, though, that we already have the tools we need to express this. We want that if `Native(p)` is false, `CanRunProgram(c, p)` is automatically true: we don’t need to even look at the computer specs.

```
CanRunProgram(c, p) = !Native(p) || ((RAM(c) && CPU(c)) || GPU(c))
```

How does this work? It’s easier to see if we pull out the right hand side into a new predicate, like `Beefy(c)`, so we have `!Native(p) || Beefy(c)`. Here’s the truth table for that expression (using `N(p)` for `Native(p)` and `B(c)` for `Beefy(c)`):

N(p)	B(c)	!N(p) B(c)
T	T	T
T	F	F
F	T	T
F	F	T

When `Native(p)` is false, `!Native(p) || Beefy(c)` is true, regardless of the value of `Beefy(c)`. When `Native(p)` is true, then the expression is equal to the value of `Beefy(c)`. So we’re only checking the computer specs if we’re running the native version, and ignoring it otherwise.

This “trick” of writing `!P || Q` to mean “check `Q` only if `P` is true” is incredibly common in math. So common that mathematicians use a special operator for it: $\Rightarrow$, which is

named “implies” (or implication). $P \Rightarrow Q$ (“P implies Q”) is the same as writing $\neg P \vee Q$. Expressed this way, our predicate is

```
CanRunProgram(c, v) = Native(v) => (RAM(c) && CPU(c)) || GPU(c)
```

$\Rightarrow$ binds less tightly than $\&\&$ and $\vee$: $A \&\& B \Rightarrow C$ is $(A \&\& B) \Rightarrow C$, not $A \&\& (B \Rightarrow C)$.

Exercise 1 (Implication)

Say we had two more “conditions”, so that `CanRunProgram` was instead

```
CanRunProgram(c, p) =
  `true unless Native(p) and either Q(p) or R(p),
  in which case (RAM(c) && CPU(c)) || GPU(c)`
```

Write this without using $\Rightarrow$. Then write this *with* $\Rightarrow$. Which is easier to read?

[Solution](#) (page 132)

Exercise 2

Right now `RAM(c)` means that “computer c has sufficient RAM”. Modify it to mean “computer c has enough ram to run program p”. Make similar changes for our other predicates and write `CanRunProgram`.

[Solution](#) (page 132)

Exercise 3

1. Using $\Rightarrow$, write the expression “if `Native(p)` is true then `Web(p)` is false, and if `Web(p)` is true then `Native(p)` is false.”
2. Using $\&\&$, write the expression “`Native(p)` and `Web(p)` are not both true.”
3. Using $\vee$, write the expression “`Native(p)` is false or `Web(p)` is false.”

[Solution](#) (page 132)

Exercise 4 (Implication as conditional)

Take the predicate

```
IfElse(c, x, y) =
  (c => x) && (!c => y)
```

Assume c , x , and y are all booleans.

1. When is `IfElse` true? When it is false?
2. What common code construct does this look like?

[Solution](#) (page 132)

2.2 Sets

Predicates are untyped by default. In `CanRunProgram(c)`, c can be a computer, but c can also be a robot, or the number 26, or the string “the number 26”. In programming, we would want to give it a type to make it clear that we should only pass in computers. Something like

```
CanRunProgram(c) = `c is a computer`
  && ((RAM(c) && CPU(c)) || GPU(c))
```

Now, even if we glue a good GPU to a poodle, `CanRunProgram(poodle)` will still be false. To make the concept “ c is a computer” mathematically representable, mathematicians use *sets*. A set is an unordered collection of unique values, like “all computers”, “all strings longer than five characters”, or “all sorted arrays of integers”. Conventionally, we write the elements of a set like this:

```
Computer = {my_laptop, your_laptop, your_other_laptop, ... }
```

Then “ c is a computer” is equivalent to saying “ c is an element of the set `Computer`”, which we will write as `c in Computer`.

```
CanRunProgram(c) = c in Computer && ((RAM(c) && CPU(c)) || GPU(c))
```

As syntactic sugar, I could instead write `CanRunProgram(c: Computer)` to mean “ c must be an element of `Computer`”, like this:

```
CanRunProgram(c: Computer) = (RAM(c) && CPU(c)) || GPU(c)
```

This will make writing predicates with several constrained parameters easier.

Note

The set of all elements our predicates are acting on is called the *domain of discourse*. So as to prevent eldritch math horrors, predicates *cannot* be in the domain of discourse: there are no predicates that take other predicates. Otherwise you can do what you want. Usually the domain of discourse is contextually evident, and we don't need to write it. If you want to know more about eldritch math horrors, check out *Beyond Logic* (page 129).

Notice that if we define EnoughRAM as the set of all computers with enough RAM, then every element of that set is also in the set Computer. We say that EnoughRAM is a *subset* of Computer.

The set of all subsets of a set is called the *power set*. As an example, if a program can take two flags, -n and -v, there are four possible combinations of flags you can pass in:

```
power_set({-n, -v}) = {
    {},
    {-n},
    {-v},
    {-n, -v},
}
```

(Remember: functions are snake_cased.)

2.2.1 Set operations

Programming lists have a lot of structure, so there's a lot of ways you can manipulate them. Given [A, B] and [B, C], I can concat [B, C] to [A, B], concat [A, B] to [B, C], concat [B, A] to [B, C], interlace the two...

Sets don't have much structure, so there are only a few basic operations. Given sets {A, B}, and {B, C}, the basic things we can do are:

1. *Union* them together, or smush them into one big set: $\{A, B\} \cup \{B, C\} = \{A, B, C\}$
2. *Intersect* them, or find the common elements: $\{A, B\} \cap \{B, C\} = \{B\}$
3. Take the *set difference*, or subtract one set from the other: $\{A, B\} - \{B, C\} = \{A\}$
4. That's it!

Mathematicians like sets for their simplicity, and use them as the foundational bedrock to build out more complex concepts, like lists. As programmers, we are already used to working with complex concepts. Even so, sets are still useful in programming. We will see this in the next chapter.

Exercise 5 (Sets vs Predicates)

Say that instead of having a predicates $\text{RAM}(c)$, $\text{CPU}(c)$, $\text{GPU}(c)$, we had the *sets* RAM , CPU , and GPU . Use these to construct the set CanRunProgram , the set of all computers that would pass $\text{CanRunProgram}(c)$.

[Solution](#) (page 132)

Exercise 6 (Disjoint Sets)

Given the sets Child and Adult , express the statements “nobody is both a child and an adult” by saying the sets do not overlap.

HINT: you can use $\{\}$ to mean the “empty set”.

[Solution](#) (page 133)

Exercise 7 (Symmetric Difference)

The *symmetric difference* of two sets is the set of all elements in exactly one of the two sets. For example, the symmetric difference of $\{A, B\}$ and $\{B, C\}$ is $\{A, C\}$.

Using just the basic set operations, find the symmetric difference of arbitrary sets S and T .

[Solution](#) (page 133)

It’s also quite useful to map and filter sets. The standard math notation is to write $\{f(x) | P(x)\}$, but that’s confusing, so instead I will steal Python notation.

- Map: $\{x^2 \text{ for } x \text{ in set}\}$
- Filter: $\{x \text{ in set: } x > 2\}$
- Map *and* filter: $\{x^2 \text{ for } x \text{ in set: } x > 2\}$

This is sometimes called a “set comprehension” or “set builder notation”.

2.3 Quantifiers

Let's move away from software requirements and switch to a different problem. Software development teams often require changes to the main code to be first proposed as part of a pull request, which must be reviewed by another team member. More concisely:

A pull request must be reviewed by a team member before it can be merged.

Let us assume that we have two sets, `PullRequest` and `Developer`, that we can use in our predicates. I would start with this:

```
ReviewedBy(pr: PullRequest, d: Developer) =
  `d reviewed pull request pr`

CanMerge(pr: PullRequest) = `someone reviewed pr`
```

Both of these predicates are abstract, but it seems like we should be able to make `CanMerge` concrete by defining it in terms of `ReviewedBy`. For this we need a *quantifier*, or a predicate over a whole set. There are two common quantifiers in predicate logic. The first, the one we will use here, is called *some*: *some x in set: P(x)* means that *P(x)* is true for *at least one* *x* in the set set.

```
CanMerge(pr: PullRequest) = some d in Developer: ReviewedBy(pr, d)
```

Note

`ReviewedBy` is already typed to only “accept” developers (be false if *d* is a pooler). But the point of logic is to communicate clearly, so it is better to be clear and explicit here.

I would read this as “`CanMerge` is true for the Pull Request element *pr* if there is at least one element *d* in the set of Developers where `ReviewedBy(pr, d)` is true”. Or, as just “there is some developer that reviewed the *pr*.”

The token *some* is “quantifying over” the set `Developer`, or alternatively is *scoped* to that set. This makes our use of it a *scoped quantifier*. More rarely, an expression is true for *any* value we care to name. For example, the statement *some x in set: P(x) => some x in set: P(x) || Q(x)* is true *regardless* of set is. In this case, we can choose to leave out the sets and write

```
some x: P(x) => some x: P(x) || Q(x)
```

This use of some is not scoped to a set, so we call it an *unscoped quantifier*. Almost all quantifiers we used will be scoped.

2.3.1 all

As it stands, CanMerge is too permissive. What happens if the reviewer found a major security flaw? What if five developers review the pull request and two find flaws? Most companies use a stricter merge requirement:

A pull request must be reviewed by at least one team member, *and all reviewers must approve the request*, before it can be merged.

As is our habit, we start by writing the requirements as abstract predicates.

```
ApprovedBy(pr: PullRequest, d: Developer) = `d approved pr`
```

```
SomeoneReviewed(pr: PullRequest) =
  some d in Developer: ReviewedBy(pr, d)
EveryoneApproves(pr: PullRequest) =
  `everyone who reviewed pr also approved it`
```

```
CanMerge(pr: PullRequest) =
  SomeoneReviewed(pr) && EveryoneApproves(pr)
```

This gives us an opportunity to introduce the other quantifier: *all*. *all x in set: P(x)* says that *P(x)* is true for *every* *x* in our set. With this, it *seems* like our new predicate can be written like this:

```
EveryoneApproves(pr: PullRequest) =
  all d in Developer: Approved(pr, d)
```

But this is wrong. This requires *every single* developer to approve the pull request, including developers out sick or on maternity leave. We only want to require that every developer *who reviewed the pull request* to approve it. We can fix this, though, with implication. Recall that $P \Rightarrow Q$ means $\neg P \vee Q$. Then $\text{ReviewedBy}(pr, d) \Rightarrow \text{Approved}(pr, d)$ means that either *d* approved the pull request or did not review it at all.

```
EverybodyApproves(pr: PullRequest) =
  all d in Developer: ReviewedBy(pr, d) => Approved(pr, d)
```

We often use $\Rightarrow$ to only “evaluate” an all on certain elements.

Note

I have seen six or seven different notations that logicians use for expressions and quantifiers. About the only thing they do agree on is the symbol for some and all: $\exists$ and $\forall$. You might notice *these symbols is not on your keyboard*, which is why I instead use ASCII words. As always, you can check the [appendix](#) (page 125) to see some of the more conventional notations.

Most programmer languages have built-in quantifier functions, as we will discuss in [a later chapter](#) (page 25). If your language of choice does not, you can usually approximate quantifiers with a loop. For example, you could write `SomeoneReviewed` like this (pseudocode):

```
fun SomeoneReviewed(pr: PR) {  
    for (d in developers) {  
        if(ReviewedBy(pr, d)) return true;  
    }  
    return false;  
}
```

Exercise 8

Why do we need `SomeoneReviewed` at all? Isn't it true that if everybody who reviewed the PR approved it, then someone *must have* reviewed it? Find the edge case where `EveryoneApproved` is *true* and `SomeoneReviewed` is *false*.

[Solution](#) (page 133)

Exercise 9

Define `Nat` as the set of “natural numbers”: 0, 1, 2, etc.

1. Write the logical statement “every natural number is smaller than itself plus 1.”
2. Write the logical statement “0 is less than or equal to every natural number.”

[Solution](#) (page 133)

Exercise 10 (Nested Quantifiers)

1. Write the logical statement “for every PR, there is a developer that approved it.”
2. Write the logical statement “there is a developer that has reviewed every single pull request.”

In both cases you will need to put one quantifier inside a different quantifier.

[Solution](#) (page 133)

2.4 Notation

Mathematicians like to say that logic is a “language”. The point of language is to communicate complex ideas clearly, and sometimes the best way to do that is to come up with new words and grammar. In logic, too, we can come up with new constructs and ways of writing formulae, as long as 1) it’s consistent and 2) we explain clearly what we’re doing. In fact, this is encouraged. For example, the normal way of writing “the set of integers between 1 and 10” takes up a lot of space:

$$\{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$$

If I wanted something more concise, I can come up with a shorthand:

$$\{1, 2, 3, \dots 100\}$$

If I wanted to be even more compact, I can *define* new syntax:

$$\begin{aligned} 1..=100 &= \{1, 2, 3, \dots 100\} \\ 1..<100 &= \{1, 2, 3, \dots 99\} \end{aligned}$$

This isn’t completely unambiguous: what is $10..=9$? I will define it as the empty set: if $a > b$, then $a..=b$ is empty. Similarly, $a..<b$ is empty whenever $a \geq b$.

Exercise 11

Rewrite that rule (that if $a > b$, then $a..=b$ is empty) using the all quantifier. Assume both a and b are in the set of integers.

[Solution](#) (page 133)

Exercise 12

Write `1..=100` using set filter notation. Filter on the set `Int`.

[Solution](#) (page 133)

Exercise 13 (Divides)

Write `IsDivisibleBy(num, divisor)`, which is true if `num` is evenly divisible by `divisor`. Use `some` and `..=`.

[Solution](#) (page 133)

Another bit of syntactic sugar I find very useful is “conjunction lists”. Complicated systems often have complicated requirements:

```
Rules = A && B && (C || D) && (E || (F && G))
```

That’s hard to read! To make it easier, let’s instead write it like this:

```
Rules =
  1. A
  2. B

  3. || C
     || D

  4. || E
     || a. F
        b. G
```

Numbers like 4. and letters like a. will always mean “AND”. If I want a list of “OR”s, I will always use `||`.

One last bit of syntactic sugar: sometimes we want to quantify over multiple elements in the same set. Like this:

```
IsUnique(list) =
  all i, j in 0..

```

This will *almost always* be false. Do you see why?

It’s because I never said that `i` and `j` are different values! If `l` has at least one element, `IsUnique(l)` will be false. Normally I’d need a uniqueness condition, like this:

```
IsUnique(list) =
  all i, j in 0..

```

This works but is annoying when we want to quantify over three or more variables. So I will add a new modifier for quantifiers: *all disj* x, y in set: means “for all *disjoint* x and y in set”, aka all distinct pairs of values in the set. With that, I can write `IsUnique` in a more intuitive way.

```
IsUnique(list) =
  all disj i, j in 0..

```

Exercise 14

If I had some *disj* $x, y: P$ and wanted to rewrite it without *disj*, what would that look like?

[Solution \(page 134\)](#)

Exercise 15

If I had *all disj* $x, y, z: P(x, y, z)$ and wanted to rewrite it without *disj*, what would that look like?

[Solution \(page 134\)](#)

2.5 In Practice: Rewrite Rules

Note

This section is draft 0

In the beginning of the book, I said that logic is the mathematics of booleans, just as arithmetic is the mathematics of numbers. Knowing arithmetic lets us simplify expressions of numbers. For example, here is how we can simplify the function $f(x, y) = -10x + 2(y + 5x)$:

1. $2(y + 5x)$ is the same as $2y + 10x$.
2. $-10x + 2y + 10x$ is the same as $10x - 10x + 2y$.

- 3. The first two terms are opposites, so they cancel out.
- 4. So we have just $f(x, y) = 2y$.

In logic, these simplifications are called *rewrite rules*. You may have already used one rewrite rule as a kid:

Are you sorry? No? Well are you not not not not not sorry?

The rewrite rule here is $!!a == a$. This means $!!(!(!\text{Sorry}))$ is the same as Sorry.

Other rewrite rules include:

Name	Rule
De Morgan’s Law	$!(A \ \&\& \ B) == !A \ \ !B$ $!(A \ \ B) == !A \ \&\& \ !B$
Contrapositive	$P \Rightarrow Q == !Q \Rightarrow !P$
And/Or Distribution	$(P \ \&\& \ Q) \ \ R == (P \ \ R) \ \&\& \ (Q \ \ R)$ $(P \ \ Q) \ \&\& \ R == (P \ \&\& \ R) \ \ (Q \ \&\& \ R)$
Duality	$\text{all } x: !P(x) == !(some \ x: P(x))$ $some \ x: !P(x) == !(all \ x: P(x))$
Some/Or Distribution	$some \ x: (P(x) \ \ Q(x)) ==$ $(some \ x: P(x)) \ \ (some \ x: Q(x))$
All/And Distribution	$all \ x: (P(x) \ \&\& \ Q(x)) ==$ $(all \ x: P(x)) \ \&\& \ (all \ x: Q(x))$

Exercise 16

Use rewrite rules to simplify $!(some \ x: !P(x))$.

[Solution](#) (page 134)

Exercise 17

Give a real-world example of each distribution rule.

[Solution](#) (page 134)

Exercise 18

The following two are *not* rewrite rules:

1. $\text{all } x: P(x) \parallel Q(x) == (\text{all } x: P(x)) \parallel (\text{all } x: Q(x))$
2. $\text{some } x: P(x) \&\& Q(x) == (\text{some } x: P(x)) \&\& (\text{some } x: Q(x))$

Give an example where each is wrong.

HINT: for the some case, try starting with a valid example of the right-hand-side and show it doesn't match the left-hand.

[Solution](#) (page 134)

Over time you'll internalize a lot of rewrite rules. See [Useful Rewrite Rules](#) (page 127) for a list.

2.5.1 Theorems

Rewrite rules are *theorems*, meaning we can work them out from other rules.

Take contrapositive, for example: $P \Rightarrow Q == !Q \Rightarrow !P$. We can derive it this way:

1. Start with $!Q \Rightarrow !P$.
2. Apply the definition of implication to get $!!Q \parallel !P$.
3. Remove the double negative to get $Q \parallel !P$.
4. Apply the definition of implication again to get $P \Rightarrow Q$.

Tada, we just proved the contrapositive rewrite rule works! Try going the other way, starting from $P \Rightarrow Q$.

Exercise 19 (Contrapositives)

Start from $P \Rightarrow Q$ and rewrite it into $!Q \Rightarrow !P$.

[Solution](#) (page 134)

2.6 Summary

1. A *predicate* (page 5) is a boolean “function”, which can be defined over anything.
2. A *set* (page 10) is an unordered collection of unique elements. Sets can contain anything. Set of things we are working on is the “domain of discourse” (DoD).
3. Expressions can be quantified, checked if they’re true for all elements of a set or any element.
4. Math notation is flexible. We can come up with new notation, operators, grammar, etc as long as it’s clear and consistent.
5. Logical formulae can be rewritten and simplified.

Here’s all of the symbols we learned about:

1. Predicates are always TitleCase(x), functions always lowercase and snake_case(x).
2. And, or, and not: &&, ||, !
3. Implies: =>
4. Set union, intersection, difference: |, &, -
5. Set map and filter: {x^2 for x in set: x > 2}
6. all x and some x
7. Various syntactic sugar.

And that’s it! That’s all of the basics of formal logic. Really not that much, when you think of it.

The difficulty, of course, is in the application. It’s one thing to know division, quite another to realize that “scale a recipe with 5 eggs to use only 3 eggs” is a division problem. The rest of the book is about software situations where logic is useful, and how to *make* it useful. Let’s use logic to understand the world.

Chapter 3

Refactoring Code

We will start our overview of techniques by using logic to simplify complicated code. Later techniques will cover more impressive applications, but refactoring is a universal programming task and knowing more tricks is always handy. All code samples are either code I personally encountered or samples of production code I found on GitHub.

3.1 Simplifying Conditionals

In the last chapter, we learned about “rewrite rules”, which let us simplify some logical expressions. Using these rewrite rules, we can simplify code too. Starting with the conditional `!(x && y) || !x` {...}:

Step	Expression	Rule
0	<code>!((x && y) !x)</code>	Initial value
1	<code>!((x !x) && (!x y))</code>	distribution (page 19)
2	<code>!(T && (!x y))</code>	<code>x !x</code> is always true
3	<code>!(!x y)</code>	<code>true && y == y</code>
4	<code>!!x && !y</code>	De Morgan’s law
5	<code>x && !y</code>	double negation

Each transformation in the above chain uses a solid, rigorous logical rule. As long as we do not make a mistake in applying the rule, we do not change the value of the expression, and we can be confident our simpler code has the same behavior.

Most of the time, we don’t write out every single step along with the name of the applied rule, since the next steps are obvious in our heads. I “know” I can rewrite `(x && y) || !x` as `!x || y`, in the same way I “know” that four times three is twelve. But we can always fall back on the rewrite rules if we get confused or have deal with something messy.

Tip

Some equations can be simplified automatically with tools, like for example <https://www.dcode.fr/boolean-expressions-calculator>.

3.1.1 The power of $\Rightarrow$

As a rule of thumb, whenever `&&` and `||` correspond to something “obvious” in programming, $\Rightarrow$ will correspond to something “special”. This is true here, and in *testing* (page 32), and in many other places. The “special” thing here is that where `&&` and `||` represent the condition in an if statement, $\Rightarrow$ represents the if statement itself!

```
if P {Q} # is the same as
P => Q
```

Technically, this is what mathematicians would call an “abuse of notation”: the body of a condition can be any computation, while the right-hand-side of an implication must be a boolean expression. Even so, it turns out we can manipulate if statements and conditionals in basically the same way. Any rewrite rule for $\Rightarrow$ gives us a refactoring of conditional code. For example, $P \Rightarrow (Q \Rightarrow R)$ is the same as $P \&\& Q \Rightarrow R$. Therefore:

```
if P {
  if Q {
    R
  }
}

# is the same as
if (P && Q) {
  R
}
```

We can take this further. In a *previous exercise* (page 10) we learned that if P then Q else R is the same as $P \Rightarrow Q \&\& !P \Rightarrow R$. Presented with

```
if (P || !Q) {
  # body 1
} else {
  if (Q && R) {
    # body 2
  }
}
```

The else is equivalent to

Step	Rule
$!(P !Q) \Rightarrow (Q \&\& R \Rightarrow \text{body1})$	if-else
$!P \&\& Q \Rightarrow (Q \&\& R \Rightarrow \text{body1})$	De Morgan
$!P \&\& Q \&\& Q \Rightarrow (R \Rightarrow \text{body1})$	See above
$!(P !Q) \Rightarrow (R \Rightarrow \text{body1})$	De Morgan

This is the same expression that we started with except that we removed Q from the middle. In other words, checking Q is true in the second if is unnecessary; we already know it's true because we are in the first if's else branch! The code snippet simplifies to

```
if P || !Q:
    # stuff
else:
    if R:
        # other stuff
```

Exercise 20 (Rewriting ifs)

Recall that we earlier we showed if P then Q else R is mathematically equivalent to $P \Rightarrow Q \ \&\& \ !P \Rightarrow R$. Use that to show we can rewrite the same conditional as if !P then R else Q.

Hint: don't think too hard about it. You only need to apply a couple of common rules.

[Solution](#) (page 135)

3.2 Refactoring with Quantifiers

If you search GitHub, you can find a lot of code like this:

```
def is_toolchain(self, *args):
    actual_toolchain = self.ToolchainName()
    for v in args:
        if v.lower() == actual_toolchain:
            return True
    return False
```

Consider what this does. It searches through a list to see if any element of the list satisfies a property. Doesn't that look like one of our quantifiers?

```
IsToolChain = some v in args: ActualToolChain(v)
```

There's some subtle differences, in that lists are not sets, but it's close enough. Wouldn't `is_toolchain` be simpler if we could just use the some quantifier directly?

In fact, we can! Most languages have built-in quantifier functions. In Python, these are `all(bool_list)` and `any`. Here's what `is_toolchain` looks like using the quantifier:

```
def is_toolchain(self, *args):
    actual_toolchain = self.ToolchainName()
    return any(v.lower() == actual_toolchain for v in args)
```

Exercise 21 (Your language's quantifiers)

Find the quantifiers in your language of choice. One of them should be all and one should be some.

Bonus: does your language have any non-standard quantifiers, like “exactly one” or “none”?

[Solution](#) (page 135)

Going further, we can simplify expressions using quantifiers just like we would any other logical expression.

3.2.1 Simplifying Quantifiers

This anonymized block of Python code comes from a large public project.

```
if not all(P(x) for x in l) or any(not Q(x) for x in l):
    do_thing()
else:
    do_other_thing()
```

In formal logic, the condition is $\neg(\forall x: P(x)) \vee \exists x: \neg Q(x)$. I can use unscoped quantifiers because I don't care about the type of x ; this simplification should work regardless.

My first heuristic is to try to reduce the total number of quantifiers used. Based on the *quantifier distribution rules* (page 19), I know that some distributes over $\vee$, so I will make the *dual rewrite rule* (page 19) to turn the $\neg\forall x: P$ into $\exists x: \neg P$.

Step	Rule
$\neg(\forall x: P(x)) \vee \exists x: \neg Q(x)$	init
$\exists x: \neg P(x) \vee \exists x: \neg Q(x)$	duality
$\exists x: \neg P(x) \vee \neg Q(x)$	distribution

This is simpler mathematically and is more efficient programmatically, as we only iterate over the list once instead of twice. Stopping here would be absolutely fine. I

find it useful, though, to continue experimenting with rewrites. I see De Morgan’s Law can be used here, so I will try it anyway and see where it takes us.

Step	Rule
some x: !P(x) !Q(x)	init
some x: !(P(x) && Q(x))	De Morgan
!all x: P(x) && Q(x)	duality

This doesn’t seem to me like a significant improvement over where we were. Even so, there was no cost to trying and it still gives us good practice. This also opens up one more possible refactor: if P then Q else R is the same as if !P then R else Q. This lets us remove the top-level “not”:

```
# OLD
if not all(P(x) for x in l) or any(not Q(x) for x in l):
    do_thing()
else:
    do_other_thing()

# NEW
if all(P(x) and Q(x) for x in l):
    do_other_thing()
else:
    do_thing()
```

This last refactoring *could* be a step too far. Programmers tend to think of the if as the normal case and the else as the exceptional case, and by switching the two, we may have changed how they understand the code. We must always apply our best judgement as a software engineer.

Exercise 22

I once saw some code that used the same predicate in two quantifiers:

```
return any(P(x) for x in l) and all(P(x) for x in l)
```

Why is the any necessary? Rewrite this to use only one quantifier.

[Solution](#) (page 135)

3.2.2 Helper predicates

In the previous example, we treated the predicates $P(x)$ and $Q(x)$ as opaque. If we can modify the predicates then we can often simplify code even further. One more anonymized example:

```
if not all(not a.chunks
          or len(a.chunks[0]) == df.npartitions for df in dfs):
    raise_error()
```

The logical representation of this conditional is $\neg \forall df \in dfs: (\neg P(df) \vee Q(df))$. First, we can start by treating the predicates as opaque and rewrite the abstract expression:

Step	Rule
$\neg \forall df \in dfs: (\neg P(df) \vee Q(df))$	init
$\text{some } df \in dfs: \neg(\neg P(df) \vee Q(df))$	duality
$\text{some } df \in dfs: P(df) \wedge \neg Q(df)$	De Morgan

This corresponds to this code change:

```
if not all(not a.chunks
          or len(a.chunks[0]) == df.npartitions for df in dfs):
    # becomes
if any(a.chunks
        and not len(a.chunks[0]) == df.npartitions for df in dfs):
```

Looking at the code directly, though, reveals some useful details not present in our opaque predicates. The first is that $P(df)$ is actually just P : the body of the predicate, `a.chunks`, does not depend on `df` at all. There is no reason to keep it in the quantifier, and indeed we can extract it outside:

Step	Rule
$\text{some } df \in dfs: P(df) \wedge \neg Q(df)$	init
$\text{some } df \in dfs: P \wedge \neg Q(df)$	P doesn't depend on df
$P \wedge \text{some } df \in dfs: \neg Q(df)$	Extraction

```
if any(a.chunks
        and not len(a.chunks[0]) == df.npartitions for df in dfs):
    # becomes
if a.chunks
    and any(not len(a.chunks[0]) == df.npartitions for df in dfs):
```

The second detail is that is that $Q(df)$ takes the form $\text{len}(a.\text{chunks}[0]) == df.\text{npartitions}$. We can abstract this by replacing the left hand side (lhs) with the constant c and the rhs with the function $\text{np}(df)$, giving us $Q(df) = (c == \text{np}(df))$. Then $!Q(df)$ can be simplified to $c != \text{np}(df)$.

Step	Rule
$P \ \&\& \ \text{some } df \text{ in } dfs: !Q(df)$	init
$P \ \&\& \ \text{some } df \text{ in } dfs: !(c == \text{np}(df))$	Defining Q
$P \ \&\& \ \text{some } df \text{ in } dfs: c != \text{np}(df)$	Negation

The overall code change is

```
# OLD
if not all(not a.chunks
    or len(a.chunks[0]) == df.npartitions for df in dfs):
    raise_error()

# NEW
if a.chunks
    and any(len(a.chunks[0]) != df.npartitions for df in dfs):
    raise_error()
```

In general, we can define $R(x) = !Q(x)$ to “hide” a negation. This only makes sense if we can find a suitable, easily understandable $R(x)$ that doesn’t muddle things. This can be a new abstract function, like replacing `!correct_password(p)` with `wrong_password()`. Other times, this can involve replacing an infix operator.

Tip

In these examples, we converted the program to a logical formula and did all our rewriting before converting back. In practice it can be easier to switch between the representations: rewrite the logic, convert back to code, rewrite the code, convert back to logic, etc.

Exercise 23

Simplify the expression `!(x > 1 && x <= 10)`.

TODO: more

[Solution](#) (page 135)

3.3 Programs are not Math

Logic gives us lots of ways to rewrite conditionals. Unfortunately, we can't always use them: programming languages (PLs) follow their own rules, and these aren't always compatible with mathematics.

We've already seen one such difference: logic primarily uses *sets* while PLs use *lists*. In most cases this doesn't matter, but it can if you need to worry about ordering or duplicates. Some languages do have a native set type, but not all.

In many languages the quantifiers aren't *quite* quantifiers, because they don't take arbitrary predicates. They instead check that an array of booleans all evaluate true. This makes expressing nested and chained quantifiers (like $\forall x, y$) awkward. It can also lead to surprising behavior when run on *non-boolean* lists. Many languages have special "truthiness" rules for evaluating values as booleans. In Python, `any([0])` is *false*, while in Ruby `[0].any?` is *true*.

3.3.1 Emulating implication

Almost no language supports implication as an operator. Instead, implication in expressions usually maps to top-level conditional control flow. While this often is not a problem, it can cause us trouble when the implication is inside a quantifier:

```
# Not valid python
if all((len(l) != 0 => x in l) for l in lists):
```

Usually you can get away with writing $\neg P \vee Q$ instead. You can also sometimes use the trick of replacing an implication with a filter, as in this exercise:

Exercise 24 (Implication via filtering)

Most languages don't support $\Rightarrow$, but they *do* support some kind of collection filter. So if you need to encode

```
all x in set: P(x) => Q(x)
```

You can usually write it as

```
all x in {x in set: P(x)}: Q(x)
```

Explain why these are equivalent.

[Solution](#) (page 135)

3.4 Using sets

Our last refactoring tool is a little different. Not all languages have a built-in data type for sets. For those that do, set types can be a fantastic tool for simplifying code. Say that we are modeling a simple social network, where every user has a list of connections. We want to write a function that, for a given user, finds all other users “one hop away”: everybody that is connected to the input’s connections. For simplicity we will assume that we retrieve connections with `conn_list[user]`, where `connections` is some mapping that returns lists.

Listing 3.1: (Python)

```
def get_with_lists(user, conn_list):
    out = []
    for c in conn_list[user]:
        for u in conn_list[c]:
            if u != user and u not in out:
                out.append(u)
    return out
```

Notice that the output should be a collection of *unique* users. Since the list does not guarantee that by default, we have to enforce that with checks. On the other hand, if `g.members` instead returned a *set*, we could simplify the code considerably:

```
def get_with_sets(user, conn_set):
    out = set()
    for c in conn_set[user]:
        out |= conn_set[c] #union=
    out -= {user} #difference=
    return out
```

We no longer need checks because the set type “takes care” of the uniqueness constraint for us. In addition to being simpler, this can often be more efficient. Since sets do not need worry about order or duplicates, language implementers can make some set operations more efficient than the corresponding list operations. I provided a Python benchmark in the [book assets](https://github.com/logicforprogrammers/book-assets/tree/master/code/chapter-03)²: as the connection graph grows larger, the set-based approach becomes orders-of-magnitude faster. This holds even we include the time taken to construct a set representation from a list representation!

On top of these benefits, sets also provide a useful signal to other programmers reading our code. When I see a codebase that uses both sets and lists, I can be confident they are using the sets for unique unordered data and lists for data that *must* be or-

² <https://github.com/logicforprogrammers/book-assets/tree/master/code/chapter-03>

dered or duplicated.

3.5 Summary

1. Logic provides us rewrite rules we can use to refactor boolean expressions. Some control flow statements, like if, can be manipulated like implication.
2. Many languages support quantifiers, which let us further simplify code.
3. Set types can, in some circumstances, be clearer and more efficient than list types.
4. Be careful: not all logical refactorings are supported by all languages!

No refactor is complete until we have thoroughly tested that the behavior is the same. In the next chapter, we will learn a logic-based technique to test refactorings and code more broadly.

3.5.1 Learn More

[[None yet for this chapter!]]

Chapter 4

Writing Better Tests

The most common form of software test is the “example” test: pass an input into a function and check that it returns the correct output. Here are some example tests for `max`:

```
test max([1, 2, 3]) == 3
test max([1, 3, 2]) == 3
test max([2, 3, 3, 2]) == 3
```

Example tests are easy to write, but they are also limited. Many functions that are *not* `max` pass these tests:

1. A function that returns the largest absolute value in the list
2. A function that returns the most common element, breaking ties with max value
3. A function that returns the maximum of the first five elements
4. A function that just returns 3.

The more examples we write, the more invalid functions we rule out. But this is tedious and error prone. Logic provides us an alternative: express the essential meaning of the function, and then use this to generate hundreds of tests for us.

4.1 Strong and Weak Tests

Some tests are stronger than others.

“Stronger” has a precise logical meaning. This is because *tests are equivalent to predicates*. The first test in the last section is equivalent the same as the predicate $P = \text{max}(l) == 3$. The test passes when P is true and it fails when P is false. For convenience, I will use “ P ” to refer both to the predicate and the corresponding test.

This means we can use our same logical operators to express statements about tests. $P \ \&\& \ Q$ is true if (the tests corresponding to) P and Q both pass. $P \ || \ Q$ is true if at least one of the two test passes. $!P$ is true if the test P fails. Finally, $P \ ==> \ Q$ (aka $!P \ || \ Q$) is true if P passing implies that Q also passes.

What does that mean in practice? It means that there is no possible version of `max` that *passes* P and *fails* Q . If a failing test means a buggy implementation, then any bug that “slips past” P will slip past Q , too. This means that P is at least as *strong* as Q ,

which is totally captured in the logical expression $P \Rightarrow Q$. If P can catch a bug that Q will miss, then P is *stronger*. As an example:

$P = \max([1, 2, 3]) == 3$

$Q = \max([1, 2, 3]) >= 0$

$R =$

1. $\max([1, 2, 3]) >= 0$

2. $\max([0, 1, -1]) >= 0$

If P passes, Q also passes. If R passes, Q also passes. This means $P \Rightarrow Q$ & $R \Rightarrow Q$. We can further see that both are stronger than Q. Notice that P and R are stronger than Q in different ways: P gives a more specific answer for the same input, while R tests a wider range of inputs. Finally, neither R nor P are stronger than each other: each will pass some version of max the other would reject. Mathematicians would say that $\Rightarrow$ forms a “partial ordering”.

[[TODO graphical diagram of this]]

Exercise 25 (Partial Ordering)

1. Give a buggy implementation of max that R passes but P fails, and a buggy implementation that P passes and R fails.
2. Modify the two clauses of R to create a test T that’s stronger than both P and R. It should fail both implementations you wrote above.
3. How would you express “T is at least as strong as both P and R?” Does this mean T is stronger than Q, too?

[Solution](#) (page 135)

Exercise 26 (The Flaw with False)

For any predicate P, $\text{false} \Rightarrow P$. So any possible bug in max that’s caught by a test will also be caught by test false, making it the strongest possible test imaginable. And in fact Explain the flaw in this reasoning.

[Solution](#) (page 136)

While P and R are stronger than Q, neither is strong enough, on its own, to guarantee that max is correct. This version of max passes both tests but still is incorrect:

```

max(list) =
  if list == [1,2,3] then 3 else
  if list == [0,1,-1] then 1 else
  -infinity

```

We now have two separate ways of making a test stronger: widen the number of inputs it tries, or make more specific claims about the outputs. The most powerful possible test would try every possible valid input to max and make the most specific claim possible about the output. We would call such a test a *total specification* (or total “spec”) of max. It would pass if *and only if* max was correctly implemented, making any other kind of direct testing redundant. In other words, if T is *any* test of max, then TotalSpec => T.

What then, would be that test?

4.1.1 Specifying a function

First we need to define the *domain* of max- the set of all valid inputs. For the purposes of this chapter, I’ll say max should work for any nonempty, noninfinite list of integers. The total specification looks like this:

```

IsMax(x, l) = `x is the maximum value in l`.

TotalSpec =
  all l in NonEmptyIntegerLists:
    IsMax(max(l), l)

```

It sometimes convenient for our purposes to restrict the domain to something expressible in a language’s type system.

```

all l in IntegerLists:
  len(l) > 0 => IsMax(max(l), l)

```

Now we have to define what it means to be the “maximum value” of a list. First of all, it has to be an element of the list: if we take the max value and add ten, we no longer have the max value. Second, no element of the list is larger than it. It is easier to see how to formalize this property if we start by defining IsMax for *sets*:

```

IsMax(x, set) =
  1. x in set
  2. `no element of the set is larger than x`

```

Another way to say “no element is larger than x” is to say “for all elements y in the set, x is as least as big as y.” That looks like an *all* (page 14) to me!


```

IsMax(x, set) =
  1. x in set
  2. all y in set:
      x >= y

```

[[Programmers work in lists, not sets. We can't use quantifiers on lists, but we can instead use them on the set of their indices]]:

```

IsMax(x, list) =
  some i in 0..

```

Try writing a few valid tests for max, and then see if they are implied by TotalSpec.

Exercise 27 (Uniqueness)

Write the predicate IsUnique(l), which is true iff every element of l is unique. IE

```

IsUnique([1, 2, 3])
!IsUnique([1, 2, 1, 3])

```

[Solution](#) (page 136)

4.2 In Practice: Property-Based Testing

Reminder that our total specification for max was this:

```

IsMax(x, list) =
  some i in 0..

```

Implementing IsMax in our favorite programming language is straightforward, as is calling max on a list and checking that the output passes IsMax. Trying this for *all* infinity non-empty integer lists is impossible (at least without some tools covered in the next chapter). What we could do as a substitute is test one hundred randomly generated different lists. This would not be as strong as TotalSpec, but it would be much stronger than `max([1,2,3]) == 3`.

This is the idea behind *Property-Based Testing* (PBT). We first write a test that applies to any possible input, and then we randomly generate inputs to test it. There are some engineering details to figure out (“how *do* we generate non-empty integer lists?”), but most languages have a high level libraries that handle these details for us. Here’s an example, using the python library [Hypothesis](https://hypothesis.works/)³:

```
import hypothesis.strategies as s
from hypothesis import given
@given(s.lists(s.integers(), min_size=1))
def test_max(l):
    max_val = f(l) # our max function
    assert max_val in l # (a)
    assert all(max_val >= x for x in l) # (b)
```

The @given is a generator (“strategy” in hypothesis’ terms) that says the input can be any nonempty list of integers. We define all of the function’s inputs this way, pass them to the test, run the function normally, and get the output. Finally, we check if the output satisfies our specification.

Tip

Reminder, you can download this code sample directly from <https://github.com/logicforprogrammers/book-assets>.

Compare that to our total specification. The quantified set NonEmptyIntegerLists becomes the generator (only test nonempty lists) and the body of the quantifier becomes our assertions.

In addition to handling the random generation, Hypothesis also gives us some convinces. In addition to purely random lists, it will also try common pathological cases. If an input fails, it will “shrink” the failing input to a smaller, simpler failing input. For example, if my implementation of max looked only at the first five elements of the list, here’s what it could give me back:

```
Falsifying example: test_max(
    l=[0, 0, 0, 0, 0, 1],
)
```

³ <https://hypothesis.works/>

Finally, Hypothesis stores a database of known failures and retries them on future runs.

Exercise 28 (Property Testing Find)

Look into whatever your favorite language's PBT library is, and then write a property test for `find`. You may have to write your own version `myfind` for your language, if the builtin does something besides return `-1` for a missing value (like raise an exception).

[Solution](#) (page 136)

4.3 Notes on Property Testing

4.3.1 Partial Specifications

A *partial specification* is any spec that is covered by a total spec, ie any test where $\text{TotalSpec} \Rightarrow \text{PartialSpec}$. Every test we have seen so far *besides* `test_max` is a partial specification.

In theory, we should never need to test a partial specification. In practice, the majority of the tests we write are partial for two reasons. [[One, most of the functions we work with in software are too complex to be easily total specifiable.]] And even if we can totally spec a function, partial specs help us localize the source of bugs. A total spec failing tells us that the function is incorrect, but a partial spec failing tells us *why* it's incorrect.

For this reason, using property testing well means coming up with strong, testable partial specifications. Most functions will have at least *something* expressible, often to do with the domain of the problem:

- A dating app's match function shouldn't match people with cats to people with cat allergies.
- Making a chess move and undoing it should return us to the original game state.
- A customer who clicks "submit payment" ten times should only be charged once.
- If we cut frames 126-143 of a video, the output will be seventeen frames shorter and the 906th frame will now be the 889th.

Note

I could probably make those exercises.

There are also universal “tactics” that apply to many different problems in many different domains. One of the simplest and most famous tactics: the code does not crash on some input. This is called *fuzzing* and is very popular for low-level code (where memory leaks can lead to security vulnerabilities) and parsers. Similarly, we could test that no queries made to an API return a 500 error. If we have exception handling in code, we can test that only “expected” inputs raise errors, and that no other errors are raised.

Refactoring with Tests

Another popular tactic is “our function matches a reference function”.

$$\text{all } x: f(x) = g(x)$$

Why might I want to test that I have two identical functions? One common reason is that I might have a simple function that solves my problem, but is too slow for production. I can use that to test a faster-but-more-complex version of the same function. Or I might have a simplified function that works for the happy path, and I want to make sure an edge-case-handling version still gets the same results on “good” inputs.

My favorite use-case, though, is testing that a refactoring did not change the code’s behavior. We can take *an example from the last chapter* (page 25) and show exactly that.

```
import hypothesis.strategies as s
from hypothesis import given

def old_function(l, P, Q):
    if not all(P(x) for x in l) or any(not Q(x) for x in l):
        return 1
    else:
        return 2

def refactor(l, P, Q):
    if all(P(x) and Q(x) for x in l):
        return 2
    else:
        return 1
```

(continues on next page)

(continued from previous page)

```

@given(s.lists(s.integers()),
      s.functions(like=lambda x: ...,
                  returns=s.booleans(), pure=True),
      s.functions(like=lambda y: ...,
                  returns=s.booleans(), pure=True)
    )

def test_max(l, P, Q):
    assert old_function(l, P, Q) == refactor(l, P, Q)

```

Notice that Hypothesis is able to randomly generate *functions*. These behave somewhat like mocks or stubs in unit testing: they are set to take any number of parameters and return a boolean value. In one run P might return False for every integer, in another it might return True for integers -1, 15, and 7.

Running this test shows that for all values, the simplified version of our function returns the same result.

Other tactics

One of the most famous tactics is the “round-trip” property, that converting data into another format and then back doesn’t change the data.

```

Roundtrip(x_to_y(x), y_to_x(y)) =
  all x in X: y_to_x(x_to_y(x)) = x

```

The polars dataframe library found a bug this way. They generated dataframes, converted the columns into python lists, then converted the lists back into dataframe columns. The roundtrip is that column -> list -> column should give back the original column. Hypothesis found that this could drop timezones.

Roundtrip properties are generally effective when you have a custom datatype you want to convert into a portable format, like json or CSV.

A final useful class of tactics is “metamorphic properties”. These are properties that relate multiple function calls together. For example, if your computer vision system recognizes an object, it should recognize the same object if you tilt the picture by two degrees. Or if you have query API with filters, adding a new clause to a filter should give you a subset of the results you get without it (a real bug this found in Spotify).

4.4 Summary

1. A function *specification* (page 34) is a mathematical description of how it behaves and its properties. Specifications can be full or partial.
2. Specifications can determine what are valid inputs and how they relate to outputs.
3. Tests ultimately check that a function matches its specification. Unit tests do this by checking a single input. Property tests instead generate lots of random inputs and check they all satisfy the properties.
4. We may not be able to get a full specification for your function, but we can still usefully use partial specifications.
5. Not all properties are function-local. Some span multiple functions or inputs.

As useful as PBT is, the idea that functions have specifications goes further. With it, we can expand on the idea of using specifications to verify the correctness of larger sets of code.

4.4.1 Learn More

[[Talk about fuzzing, quickcheck here, model-based testing]]

- Property Testing with Complex Inputs: <https://www.hillelwayne.com/post/property-testing-complex-inputs/>
- In Praise of Property Testing: <https://increment.com/testing/in-praise-of-property-based-testing/>
- The Fuzzing Book: <https://www.fuzzingbook.org/html/Fuzzer.html>
- Choosing properties in practice: <https://fsharpforfunandprofit.com/posts/property-based-testing-3/>
- Metamorphic Testing: <https://www.hillelwayne.com/post/metamorphic-testing/>

Chapter 5

Functional Correctness

Code has the notorious habit of relying on other code, which means relying on other code's specifications. It doesn't matter how thoroughly we test `max` if some other code calls it with an empty list.

In other words, specifications leak. Using logic, we can analyze how properties “flow” through a larger program, and find bugs that occur when otherwise-correct functions are composed in the wrong way.

To build this machinery, we will start with a simple and ubiquitous language feature: the `assert` statement.

5.1 Assertions

An *assertion* is a statement that *should* be true of a correct program, and is only false if the program has a bug. In almost all modern languages, assertions are implemented with the `assert P` statement, which ends the program if `P` is false.

Note

Assertions differ from exceptions in that exceptions can be thrown even if the code is correct, but encounters some unexpected condition at runtime. If we try to read a file and the file doesn't exist, we throw an exception. Missing files are not what we want but it occasionally happens. If we try to read a file and somehow compute a negative file size, we raise an assertion error. *Negative files should not be possible.*

With `assert` statements we can take the program specification from a test and embed it directly in the function. Let's do that with last chapter's `max` function.

Listing 5.1: (Python)

```
def max(l):
    assert len(l) > 0 # (a)
    out = l[0]
    for i in l:
        if i > out:
            out = i
    assert out in l #(b)
```

(continues on next page)

(continued from previous page)

```
assert all(out >= x for x in l)
return out
```

When we add these assertions, the only property test we need to write is

```
@given(s.lists(s.integers(), min_size=1))
def test_max(l):
    max(l)
```

How does this work? Say we wrote max incorrectly, like this:

```
+for i in map(abs, l):
    -for i in l:
```

The property test engine will generate a random input, like [-1]. Our function will set out = 1, and then run assert 1 in [-1]. The assert fails, raising an error, which is caught by the test harness and reported as a test failure.

This means that we can express the specification of a function via assertions just as we do via predicates. Let MaxPre (for *precondition*) cover all of the assertions at the beginning of the function and let MaxPost (for *postcondition*) be all of the assertions at the end. Then max is correct if all l in IntegerLists: MaxPre => MaxPost.

Computer scientists often use “requires” and “ensures” to mean “precondition” and “postcondition”, as those words are easier to read and type. I will use those terms interchangeably.

Now that we know about assertions, preconditions, and postconditions, we can use them to build up “contracts”.

5.2 Contracts

Does this function contain a bug?

Listing 5.2: (Python)

```
# Get max price of available items
def max_avail_price(items):
    avail = []
    for item in items:
        if item.available:
            avail.append(item.price)
    return max(avail)
```


Yes: if no items are available, then we call `max([])`, which fails the assertion `assert l != []`.

```

    if item.available:
        avail.append(item.price)
+   assert avail != [] # surprise!
    return max(avail)

```

Max **requires** that every caller satisfies its preconditions. In return, max **ensures** the postconditions are true of whatever it returns. For reason, we say that max has a *contract*, as in “I require you fulfill your side of the contract, and I ensure I will fulfill my side.”

For `max(avail)` to satisfy max’s contract, `avail` must be nonempty, which means there must be some available item in `items`. And now `max_avail_price`, too, has a contract:

```

def max_avail_price(items):
+   assert any(i.available for i in items)
    ...

```

The postconditions also propagate. We know that every value in `avail` corresponds to an available item in `items`, and that the function returns the largest value in `avail`. So `max_avail_price` ensures that its output is the price of the most expensive *available* item in the input.

We are beginning to run into a common issue implementing contracts: programming languages are not as expressive as logic, and encoding these postconditions purely in `assert` statements gets cumbersome (and computationally expensive)! So it can be helpful to first express the contract mathematically. This is sometimes done as comments above the function:

```

# returns: out
# requires: some i in items: i.available
# ensures:
#   some i in items:
#       1. i.available
#       2. i.price == out
#       3. all lesser in items:
#           lesser.available => lesser.price <= out
def max_avail_price(items):
    ...

```

We could also keep them separate from the code and create our own “contract notation” for functions. This would make it easier to name subpredicates of our contract, annotate output values, use helper predicates and functions, etc. Something like this:

```

max_avail_price(items) returns o
helpers:
  available = 'list of available items in items'
requires:
  'has an available item': available != []
ensures:
  'output is priciest available item':
    some i in available:
      1. i.price = out
      2. all i2 in available: i2.price <= i.price

```

In practice, I have found that preconditions and inline assertions are both easier to directly encode and more impactful than postconditions.

Exercise 29 ([Defensive Programming])

Take the following change to `max_avail_price`:

```

    for item in items:
        if item.available:
            avail.append(item.price)
+   if avail == []:
+       return None
+   else:
        return max(avail)

```

What happens to the function's preconditions? What happens to its postconditions?

[Solution](#) (page 137)

Exercise 30 (Fun with square roots)

1. Write the functional specification for `sqrt(x: number)` in contract form. It should require `x` is not negative and ensure that the output squared gives back `x`.
2. The “quadratic formula” finds the values of `x` such that $ax^2 + bx + c = 0$. It’s written

$$\frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$

Write a function `quadratic(a, b, c)` that computes the quadratic formula, using the preconditions and postconditions of both `sqrt` and division (no dividing by 0!) Use any language you’d like.

3. Given

```
x = 5
# requires (a): x >= 0
y = sqrt(x)
# requires (b): y >= 0
z = sqrt(y)
```

Is requirement (b) satisfied? You may need to modify your functional specification of `sqrt` to show it’s valid, or you may have already added the extra postcondition.

[Solution \(page 137\)](#)

5.2.1 Correctness and Debugging

In a typical language, if any of the assertions fail, the program crashes. Sometimes, we prefer to recover gracefully. Other times, a crash is our best option.

To see why, consider the case where we didn’t have any contracts at all and called `max_avail_price` with no available items. The best case, `max` tries to index an empty list and throws an error, so we crash anyway. The worst case (JavaScript, Ruby), `max` indexes an empty list, returns `null` or `undefined`, which is then sent to causes trouble in some distant part of our code.

Not only do the contracts raise problems earlier, they help us debug the problem better. Think of the contracts as a series of checkpoints the code must pass through:

Note

(v0.11) I need to make a picture for this but in the meantime, here's a sketch in a codeblock:

```

                                max
                        -----
MAPPre => MaxPre => MaxPost => MAPPost
                        -----
                                max_avail_price

```

If the MapPre contracts pass but the MaxPre contracts fail, then `max_avail_price` must have done something wrong in between setup and calling `max`. If MaxPre passes and MaxPost fails, the bug is probably in the implementation of `max`. If MAPPre fails then the bug is in whatever is calling `max_avail_price`.

Tip

Sometimes we prefer to check our assertions in development and testing, but disable them in production. For this reason, most languages with assertions support disabling them with a flag. In Python this is the `-O` flag.

5.3 Contracts vs Types

At this point, contracts and assertions look very similar to a more popular software tool: the type system. `max(l)` has the type signature `list[int] -> int`, meaning it *requires* that `l` be a list and *ensures* that it returns an integer. Many languages can check types statically, and the typechecker determines that `max` does *not* return an integer, then there must be a problem with our function's implementation. How does this compare to contracts?

It's hard to summarize two enormous fields of research, but as a general rule, contracts are better at expressing properties than types, while types are easier to check are correct. Types can be “replaced” with contracts but not vice versa. For example, replacing the type signatures of `max`:

```

def max(l):
    # l is a list of integers
    assert type(l) == list
    assert all(type(x) == int for x in l)

```

(continues on next page)

(continued from previous page)

```
# the algorithm...
assert type(out) == list
return out
```

It's not as obvious how to convert the contracts “l is nonempty” or “out is the largest element in l” back into types! Contracts can encode arbitrary computations, while types cannot.

The flip side of “encoding arbitrary computations” is that types can be checked at compile time and contracts cannot. At least, not without special tools and a lot of work (see *Proving Code Correct* (page 53)). For this reason, it's generally a good idea to use types where possible and contracts only where necessary.

In many languages, it is possible to encode complex properties in clever type definitions. Say we want to give the item data type the two boolean fields `available` and `cancelled`, and we want to guarantee that they cannot both be true. Instead of two booleans, we could use the enumerated field `status`: {avail, unavail, cancelled}. Then there is no possible way to have an item that is available and cancelled. This technique is known as “Making Illegal States Unrepresentable”, or MISU.

5.3.1 Type Invariants

We have types, we have contracts, we must have contracts on types. A *type invariant* is a property that must be true for all values of a type. “An item cannot be both available and cancelled” is a type invariant, one that we can capture in the type definition. Another is “items must have a positive price”, which we have to leave as a contract:

```
# invariant: price >= 0
# invariant: available => !cancelled
# invariant: cancelled => !unavailable
struct Item {
    price: int;
    status: {avail, unavail, cancelled}
    # ...
}
```

We need to check the invariant every time we create *or* mutate a value of the type. If any function can directly modify `item.price` then we have our work cut out for us chasing down every single use. If functions have to go through an `item.setPrice()` method then we only need to put an assert in one place.

For this reason, type invariants can be quite useful in object-oriented programming. OOP invariants are also called *class invariants*.

Note

Languages that support them oftentimes (but not universally) only check the invariant after calling public method. If the public method calls a private method, *that* is allowed to break the invariant, as long as it is restored by the end of the public method.

5.3.2 Change assertions

If we are working with functions over mutable values we may as well come up with a notation for “this is what changed” in our contracts.

```
buy(acct, item) returns ok: bool
ensures:
  `ok => price deducted from acct.balance, but balance still >= 0`
  `!ok => balance unchanged`
```

In representing these contracts we have two additional difficulties. The first is that we need some way to refer to the “old value” and the “new value” of mutable data. The few languages that support change variants use `acct` for the new value and something like `old(acct)` for the original value.

```
ensures:
  if ok then
    1. acct.balance >= 0
    2. acct.balance + item.price == old(acct.balance)
  else
    acct.balance == old(acct.balance)
```

Unfortunately, few languages support tracking the old value of a mutation: they just change the state and are done with things. Some niche languages can track this but for the most part we have to reason about change assertions logically, not check them directly.

5.4 Polymorphism and Refactoring

All modern languages have some sort of *polymorphism* feature: the ability to pass many different types to the same function. It might be provided as interfaces, class-based inheritance, Haskell typeclasses or Rust traits, or something more exotic. Regardless of how it is done, the purpose is the same: we define a function to take an “abstraction” (for lack of a better word) and then it an “implementation” of that abstraction.

A common abstraction is the “Mappable”: anything where we can insert and retrieve values at specific keys. Here V is a “generic” for any type.

```
# string keys for simplicity
abstract Mappable[V] {
  keys(): set[str]

  # requires: k in keys()
  get(k: str): V

  # ensures: get(k) == v
  put(k: str, v: V)
}
```

If I write a function f that takes a Mappable, I could pass in *any* type that implements $get()$, $put()$, and $keys()$. But I have placed contracts on the abstraction’s methods: get has a precondition and put has a postcondition. If the body of f is compatible with these contracts, it doesn’t matter *what* implementation I put in, I should be safe.

But those implementations come with contracts of their own! Consider the implementation Counter, which we might use to track how many of each value are in a list:

```
impl Counter {
  d: Dict[Int]
  # other definitions...

  get(k: str): Int {
    d.get(k) if k in d.keys() else 0
  }
}
```

This get doesn’t have the same preconditions as the abstraction; it doesn’t have any abstractions at all! If we take last chapter’s notion of weaker and stronger, where $\text{Strong} \Rightarrow \text{Weak}$, it is always safe to *weaken* preconditions in an implementation. MappingPre $\Rightarrow$ CounterPre. On the other hand, if our new precondition is stronger or incomparable, code satisfying the abstract precondition might not satisfy the implementation. Such an implementation isn’t guaranteed to be “safe”.

Postconditions behave differently. Let HistoryMap be an implementation of Mappable that tracks the history of each key’s values.

```
impl HistoryMap[V] {
  hash: Dict[V]
  hist: Dict[List[V]]
}
```

(continues on next page)

(continued from previous page)

```
# ensures: get(k) == v
# ensures: last(hist[k]) == old(get(k))
put(k: str, v: V) {
  hash[k] = v
  hist.append(k)
}

}
```

If any code following a `put` depends on `Mapping`'s ensurances, they can also depend on `HistoryMap`'s ensurances. So it is always safe to *strengthen* postconditions: `HistoryMapPutPost => MappingPutPost`. If our implemented postcondition does not imply our old one, we are once again at risk: our implementation may not longer provide the guarantees we need.

Note

If you are familiar with object-oriented languages, you might notice how similar this is to the Liskov Substitution Principle. This is not a coincidence. Barbara Liskov's model was originally defined in terms of contracts. See the paper [A Behavioral Notion of Subtyping](https://www.cs.cmu.edu/~wing/publications/LiskovWing94.pdf)⁴.

⁴ <https://www.cs.cmu.edu/~wing/publications/LiskovWing94.pdf>

Exercise 31 (A Square is not a Rectangle)

In object-oriented inheritance, there is a common saying that “a square is not a rectangle”. In other words, this is in *invalid* inheritance

```
class Rectangle {
    int length, width;
    setWidth(x) { self.width = x; }
    # etc
}

class Square inherits Rectangle {
    int side;
    setWidth(x) { self.side = x }
    getWidth(x) { return self.side }
    # et
}
```

If we treat Rectangle as the abstraction and Square as the implementation, what’s wrong with this change?

[Solution \(page 138\)](#)

These rules apply to any kind of “replacement”, like “replacing code with a refactoring.” If we rewrite `max_avail_price` in a way that preserves or weakens the preconditions, we are guaranteed to not break any existing use of the function anywhere in our codebase, and the same with preserving or strengthening a postcondition. This does not mean that violating this rule is guaranteed to fail. If we know that our codebase always calls `max_avail_prices` with a hundred available items, we can strengthen safely the precondition to always require a hundred items. However, this carries a *risk* that some rarely-seen codepath can now blow everything up.

5.5 Summary

- Asserts are statements that are only false if the program has a bug. Typically, if an assertion fails, we crash the program (though this is often configurable).
- Assertions that must hold going into a block of code are called preconditions/requirements, and those that must hold exiting a block of code are called postconditions/ensurances. These are part of a function’s “contracts”.
- Contracts “spread” from everywhere a function is used. If X calls Y, X must guarantee Y’s preconditions and can safely assume Y’s postconditions. This makes them useful for catching and debugging errors.

- Types and contracts share similar roles but have different, synergistic properties. Types can also have contracts, which are called “type invariants”.
- We can use contracts to understand if certain refactorings or substitutions are “safe”.

Now that we are familiar with specifications and contracts, we can do something extraordinary: **we can mathematically prove our code correct**. This will be the focus of the next chapter.

5.5.1 Learn More

Assertions have been around since the era of vacuum tube computers. The first language with function contracts was the [Euclid Research Language](https://dl.acm.org/doi/10.1145/954666.971189)⁵. They were further popularized in OOP by Bertrand Meyer and his language Eiffel, who also named the term “Design By Contract”. [[More recently, contract-heavy programming as seen a revival as a component of the broader “Negative Space Programming” style, such as with Tigerbeetle]].

Other languages with built-in contract support include D, Ada, Clojure, and Racket (which is the predominant language used to “research” contracts). Most languages have at least an assert statement, and many have a third party contract library in the ecosystem (such as Java’s [JML](https://www.cs.ucf.edu/~leavens/JML/index.shtml)⁶).

In practice, contracts and assertions tend to be most often used in “low-level” or “algorithmic” programming, which needs to maintain more internal properties (and where more things can go wrong). John Regeher [has an excellent overview](https://blog.regehr.org/archives/1091)⁷ on the use of assertions in this context.

⁵ <https://dl.acm.org/doi/10.1145/954666.971189>

⁶ <https://www.cs.ucf.edu/~leavens/JML/index.shtml>

⁷ <https://blog.regehr.org/archives/1091>

Chapter 6

Proving Code Correct

In programming, we want our software to be correct. Common programming tools give us confidence in correctness but not certainty. Testing only shows code is correct for *some* inputs, while compilers and conventional typechecking show code *partially* correct for all inputs.

If we want to be sure that a function is totally correct for all inputs, we need a more powerful approach. We need to use logic to *prove* the correctness.

First we will cover what we mean by a proof and how we can “prove software correct”, and then show how it’s done. This chapter is a little more mathematically involved than the other technique chapters.

6.1 What is a proof?

A mathematical proof is a rigorous argument that something is true, possibly given some other assumptions. We have already encountered proofs in chapter 2 when we “rewrote” the contrapositive rule: starting from the assumption of $Q \Rightarrow P$, we concluded that $\neg P \Rightarrow \neg Q$.

In the context of programming, we most often want to prove “correctness”, that a function or program’s implementation matches its total specification. As we have seen, there are many ways to write the total specification of a function, but the contract model makes learning proofs much easier. So a function is *correct* if satisfying its preconditions guarantees its postconditions.

To do this, we take the information we know to be true at the start of the condition (the preconditions), then update our information on every step of the algorithm, and if what we know to be true at the end implies the postconditions, then our function is correct.

We should not overstate what correctness actually means. “Correct” does *not* mean “guaranteed to do what we want in all circumstances”. It means “conforms to the specification”. Proven code can fail in practice because the spec makes assumptions that are not true in practice, like “the hardware will not randomly flip bits” or “people’s names do not contain emoji”.

6.2 Proofs

Take the function `qr(x, y)`, which returns the quotient and remainder for two positive numbers. For `qr(19, 3)`, the result should be `(6, 1)`, since $6 \cdot 3 + 1 = 19$. The contract form of the total specification is

```
qr(x, y) returns (q, r)
requires:
  x >= 0, y > 0
ensures:
  a. q*y + r == x
  b. 0 <= r < y
  c. q >= 0
```

To prove that `qr` is fully “correct”, we need to prove that (a), (b), and (c) always hold for **all** inputs that satisfy our preconditions. Let’s start with a simple, linear version of the function:

```
# requires: x >= 0, y > 0
# ensures (a): q*y + r == x
# ensures (b): 0 <= r < y
# ensures (c): q >= 0
def qr(x, y):
    q = floor(x / y)
    r = x - q*y
    return (q, r)
```

When starting out, it is easier to reason through a function if we expand it to one instruction per line.

```
# requires: x >= 0, y > 0
# ensures (a): q*y + r == x
# ensures (b): 0 <= r < y
# ensures (c): q >= 0
def qr(x, y):
    # requires: y != 0
    tmp1 = x / y
    # assert tmp1 * y == x
    q = floor(tmp1)
    # assert q <= tmp1
    # assert q + 1 > tmp1
    tmp2 = q * y
    # assert tmp2 <= x
    r = x - tmp2
    return (q, r)
```

The first line has a division, which requires that $y \neq 0$. We know from qr 's requirements that $y > 0$, and $y > 0 \Rightarrow y \neq 0$. So, *assuming the preconditions hold*, this will not throw a divide-by-zero error at runtime.

The division ensures that $\text{tmp1} * y == x$, which is knowledge we can use in the next steps. Following through the rest of the algorithm tells us that $r == x - \text{tmp2} == x - q * y$, so we use some high-school algebra and rewrite that to $q * y + r == x$. That satisfies postcondition (a).

That alone is not enough: $-11 * 3 + 14 == 19$, but those certainly are not the quotient and remainders! That's why we have the second postcondition, which requires a little more reasoning. Just like we did our rewrite rules step by step, we can do the algebra here step-by-step:

Step	Rule
$r == \text{tmp1} * y - q * y$	init
$r == (\text{tmp1} - q) * y$	distribution
$r == (\text{tmp1} - \text{floor}(\text{tmp1})) * y$	definition of q
$0 \leq \text{tmp1} - \text{floor}(\text{tmp1})$	as $\text{tmp1} \geq \text{floor}(\text{tmp1})$
$\text{tmp1} - \text{floor}(\text{tmp1}) < 1$	as floor chops off the decimal
$0 \leq (\text{tmp1} - \text{floor}(\text{tmp1})) * y < y$	multiply all terms by y
$0 \leq r < y$	definition of r

I find it helpful to add assert statements to the bodies of functions as “checkpoints”, confirming what knowledge I know for sure is true at that point.

This proves postcondition (b). Postcondition (c) will be left as an exercise for the reader.

Exercise 32 (A Missing Ensurance)

Prove that qr ensures $q \geq 0$.

[Solution](#) (page 138)

6.2.1 Loop invariants

We could also implement `qr` by repeatedly subtracting `y` from `x` until we are left with a number under `y`. Then the number of subtractions is `q` and the remainder is what's left.

```
# requires: x >= 0, y > 0
# ensures: q*y + r == x
# ensures: 0 <= r < y
# ensures (c): q >= 0
def qr_loop(x, y):
    q = 0
    r = x
    while r >= y:
        r -= y
        q += 1
    # assert 0 <= r < y
    return (q, r)
```

This is more complicated to prove because we don't know how many times the while loop will run. `qr(100, 60)` will only run the loop once, while `qr(100, 6)` will run it sixteen times!

What we need to do is find a *loop invariant*, something that holds true on every iteration of the loop. That means, at the very least:

1. It must be true when we *enter* the loop
2. It must be true after every loop iteration
3. It must be true when we *exit* the loop.

This is the invariant I will pick for our loop:

```
# loopinv: q*y + r == x
```

Now is it true on loop entry? Yes: `q == 0` and `r == x`, and `0*y + x == x`. Is it true on each loop iteration? Yes, because every loop increases `q` by 1 and `r` by `y`, and $(q+1)*y + (r-y) == q*y + y + r - y == q*y + r$, which we already know on loop entry.

Loop invariants on for loops look a little different. Say we want to prove `max` is correct. Since we are iterating through a list, our loop invariant is that our `out` is the maximum number *seen so far*. Here is one way to prove it:

```
# requires: len(l) > 0
# ensures out in l
# ensures all(out >= x for x in l)
def max(l):
```

(continues on next page)

(continued from previous page)

```

assert l != []
out = l[0]
for elem, i in enumerate(l):
    # loopinv pt 1
    # true on entering loop and every iteration
    assert all(out >= x for x in l[:i])
    if i > out:
        out = i
    # loopinv pt 2, true on exiting loop
    assert all(out >= x for x in l)
return out

```

The loop invariant looks a lot like our top-level ensurance, just on prefixes of the list instead of the whole list. This is common and intentional: the loop invariant progressively “builds up” the top level postcondition, by showing it holds for every step of building the output. A similar approach can be used to prove the correctness of recursive functions.

6.2.2 We cannot prove incorrect code

“We can prove this code is correct” is logically $\text{HaveProof} \Rightarrow \text{Correct}$. Contrapositively, $\text{!Correct} \Rightarrow \text{!HaveProof}$, as in it is impossible to prove *incorrect* code.

This is where proof most differs from our usual forms of verification. Incorrect code can still *test* correctly, if it is correct for most of the inputs. And it can still typecheck properly, as long the bug does not change the types of the values.

To see this, let’s look at an *incorrect* version of qr:

```

# requires: x >= 0, y > 0
# ensures (a): q*y + r == x
# ensures (b): 0 <= r < y
# ensures (c): q >= 0
def qr_loop_bad(x, y):
    q = 0
    r = x
    # loopinv: q*y + r == x
    while r > y: # here
        r -= y
        q += 1
    # assert 0 <= r < y
    return (q, r)

```

The change is that I replaced $r \geq y$ with $r > y$. This adds a bug that only appears

when x is a multiple of y . If we try to prove this code correct from scratch, we will quickly determine that while (a) and (c) still hold, (b) does not: $r == y$ is a possibility!

It's fairly likely that test suite would cover that case, but for more complex behavior, we are more likely to miss some unusual edge case in our test suite.

Then again... for complex code, we are also more likely to make a mistake in the proof, or not notice that the existing proof is invalidated by a code change. At least a test suite can be rerun on every change. For proofs to be practical, we would need some way to programmatically check proofs for correctness.

Enter *formal verification*.

6.3 Formal Verification

Formal verification is to writing proofs what automated testing is to manually trying function inputs. Instead of relying on human diligence to check a proof, we use a special program to read a proof and check if it is correct. Then we never have to worry about making a mistake or a proof getting stale.

Mainstream languages can be verified with special tools. For example, [Frama C](https://www.frama-c.com/)⁸ extracts contracts and proof steps from C comments and uses them to prove C programs. Other languages, like [Dafny](https://dafny.org/)⁹, are built for formal verification from the start. Dafny has dedicated syntax for contracts, proof steps, and assertions, as well as a set of more complicated use cases like memory and concurrency. [[It has a tight integration between the language design, the compiler, and the prover.]] To check proofs, Dafny uses an *SMT Solver*. We will learn how to use SMT solvers ourselves in the [Solvers](#) (page 108) chapter.

6.3.1 qr in Dafny

This is what qr looks like in Dafny:

```
method qr(x: int, y: int) returns (q: int, r: int)
  requires x >= 0
  requires y > 0
  ensures q*y + r == x // a
  ensures 0 <= r < y // b
  ensures q >= 0 // c
{
```

(continues on next page)

⁸ <https://www.frama-c.com/>

⁹ <https://dafny.org/>

(continued from previous page)

```

    q := x / y;
    r := x - q*y;
  }

```

(Dafny treats division between integers as floor division). The compiler will then try to prove all of the ensurances are satisfied, given the requirements. In this case, it is smart enough to prove correctness without any help from us. What happens if we introduce a bug?

```

- q := x / y;
+ q := x / y - 1; // bug

```

The compiler gives us an error (Fig. 6.1) saying it cannot prove ensures $r \leq y$. Interestingly, it can still prove ensurances (a) and (c). Similarly, if we remove requires $x \geq y$, it can no longer prove that ensurance (c), but can still prove (a) and (b).

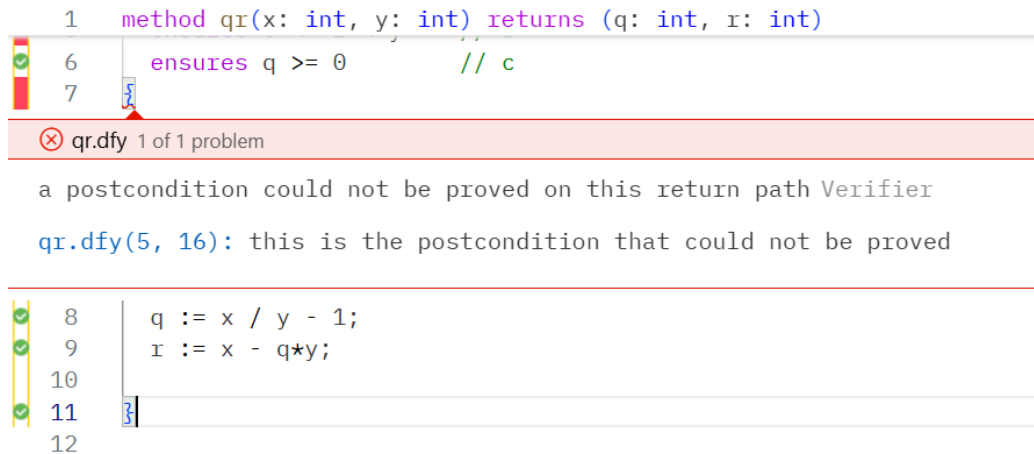


Fig. 6.1: Dafny shows a postcondition cannot be proven (VSCode)

Now let's look at qr_loop:

```

method qr_loop(x: int, y: int) returns (q: int, r: int)
  requires x >= 0
  requires y > 0
  ensures q*y + r == x
  ensures 0 <= r < y
  ensures q >= 0
{
  q := 0;
  r := x;

```

(continues on next page)

(continued from previous page)

```

while r >= y
  invariant q*y + r == x
  {
    q := q + 1;
    r := r - y;
  }
}

```

Dafny fails compilation, unable to prove ensures $r \leq 0$. Formal verification tools often need help proving things that seem obvious to us. Then again, sometimes what's obvious to us is actually incorrect, and Dafny will never make that kind of mistake. The only help we need to give it is to add another loop invariant saying that $r \geq 0$ on every loop iteration:

```

while r >= y
  invariant q*y + r == x
  + invariant r >= 0
  {

```

Now the code successfully compiles.

6.3.2 The Limits of Formal Verification

If formal verification can prove code correct, why bother writing tests?

Because formal verification is *hard*. Very hard. It demands we prove every single postcondition we declare, often to the satisfaction of limited tools. Dafny's first major success story was the [IronFleet paper](https://www.andrew.cmu.edu/user/bparno/papers/ironfleet.pdf)¹⁰, where researchers verified two distributed systems in Dafny. In their retrospective, they noted that it took 3.7 person-years to develop and prove 5114 lines of code, a rate of about four lines of verified code per workday. This is considered fast by proof standards.

And just because code is proven correct does not mean it is *actually* correct! It is only “correct” if all of our assumptions hold *and* we only depend on proven properties. A customer might depend on `qr` being reasonably fast; replacing it with `qr_loop` would ruin their day. And `max` assumes nothing else is modifying `l` as we iterate through it. In a multi-threaded program this may not be a safe assumption.

For most use cases, it is more economical to rely on informal proofs, inline contracts, and property testing. Formal verification only becomes economical for high-risk, high-severity software, like code involved in cryptography or low-level

¹⁰ <https://www.andrew.cmu.edu/user/bparno/papers/ironfleet.pdf>

systems. And even in those systems, only one or two core libraries need formal verification, while the rest does not need that level of scrutiny. For this reason, FV languages often support compiling into mainstream programming languages. Dafny supports Python, C#, and Go, among others.

Listing 6.1: Dafny qr compiled into Python

```
class default__:
    def __init__(self):
        pass

    @staticmethod
    def qr(x, y):
        q: int = int(0)
        r: int = int(0)
        q = _dafny.euclidian_division(x, y)
        r = (x) - ((q) * (y))
        return q, r
```

It never be as elegant or as idiomatic as code written in the native language, but it will always be reliable.

Even so, the techniques of *informal proof* are widely applicable. Reasoning through the correctness of a function shows us what we need to test, where to pay the most attention to, etc. And we don't always need to do full verification of the total specification. Many languages can prove some properties through the type system, and Rust's compiler can prove the absence of memory errors.

6.4 Summary

- A proof is a mathematically rigorous argument that something is true.
- If we can define function “correct” to mean preconditions guarantee postconditions, we can mathematically prove them correct.
- We can make mistakes in writing proofs. With formal verification languages, the compiler can check our proofs for errors. Some FV languages can be compiled into other languages.
- Formal verification is very difficult, and often only reserved for mission-critical software. Informal proof remains useful for reasoning about code in general.

Formal verification is part of the broader topic of *formal methods*, which we will return to *in a later chapter* (page 85). For now, though, we have spent a long time on

using logic to improve our coding, but writing code is only part of our professional work. In the next chapter, we will use logic to help us better understand our project requirements.

6.4.1 Learn More

Proving code correct via contracts starts with Tony Hoare’s [Hoare Logic](#)¹¹, first introduced in 1969:

```
{x in Int; x < 10}
x = x+1
{x <= 10}
```

Dafny uses an extension of Hoare Logic called “separation logic”, which better covers language features like aliasing, memory manipulation, and concurrency. The [Dafny website](#)¹² has more material on its advanced features and a collection of [tutorials and references](#)¹³.

Not all formal verification languages are based on contracts. Languages like Liquid Haskell and Idris have type systems that are far more powerful than mainstream languages, powerful enough to encode the complete specifications of functions. Contracts *tend* to be more popular with procedural languages, and types *tend* to be more popular with functional languages.

Formal verification is also often done with *proof assistants*, tools meant for proving mathematical theorems, adapted to instead prove programs correct. Isabelle, Rocq, and Agda have been around for a long time. Recently, [lean](#)¹⁴ is relatively new, but rapidly rising in popularity. You can also learn Lean through [interactive games](#)¹⁵, like “The Natural Numbers Game”.

To help people compare different formal verification languages, I maintain the [Let’s Prove Leftpad](#)¹⁶ project, where the properties of JavaScripts which compares over two dozen such languages with explanations.

¹¹ https://en.wikipedia.org/wiki/Hoare_logic

¹² <https://dafny.org/>

¹³ <https://dafny.org/latest/toc>

¹⁴ <https://lean-lang.org/>

¹⁵ <https://adam.math.hhu.de/#/>

¹⁶ <https://github.com/hwayne/lets-prove-leftpad>

Chapter 7

Case Analysis

A surprising number of the problems we solve with software are about making “decisions” based on combinations of inputs:

- An application might decide what to show a user based on what feature flags are set and what part of the world they are in.
- A load balancer might decide whether to spin up or wind down servers based on server load, minimum/maximum constraints, and time of day.
- An airline might decide whether to offer a perk based on the user’s ticket type, traveller class, and credit card used.
- A popular text editor decides every setting’s value based on the global default, language default, the custom user global setting, custom user language setting, custom user project setting, custom user project language setting, and whether the option takes a number or an object.

When we are asked to implement this kind of software, we don’t get the requirements as an exhaustive set of possible combinations, we are given a set of rules. And this leads to bugs in the *requirements themselves*, where the rules have a gap in their coverage... or a contradiction.

The simplest logical tool for analyzing cases is the *decision table*. The concept can be learned in minutes, used even by nontechnical team members, and is broadly useful in finding problems in both code and human requirements. They only work if the decision depends on a finite set of combinations, but that is a large enough category to make them worth knowing.

7.1 Decision Tables

To make a decision table, write down every combination of possible inputs, write the output for each input, and then put them in a sorted table.

That’s it, that’s decision tables.

In fact, we have *already used* decision tables earlier in the book. A *truth table* (page 7) is just a decision table where all of the inputs and outputs are booleans.

Let’s see an example. Imagine we are managing an event’s ticket page, and are asked to provide these discounts:

- First 100 registrations get a 10% discount

- Next 100 registrations get an 8% discount
- Seniors get a 5% discount
- Otherwise, attendees pay full price.

This table has one output, the discount. It has two inputs, registration number and senior status. While the registration number isn't a finite input, we can collapse it into three cases: 0-100, 101-200, and 201-. Senior status is just a boolean. We would expect our table to have six rows, three values for the first input times two for the second.

Table 7.1: Discounts (Ambiguous)

reg#	senior?	discount
-100	T	???
-100	F	10%
101-200	T	???
101-200	F	3%
201-	T	5%
201-	F	0%

The requirements are incomplete because they do not specify what should happen when someone is eligible for two discounts. Incomplete requirements have multiple valid “extensions”: there are different, perfectly sensible ways to complete them. I have seen at least four different solutions in real-world systems:

1. Only allow the one highest discount (here 10%)
2. Apply discounts in sequence (here 14.5%)
3. Add all discounts together and apply at once (here 15%)

Let us assume that in *this* example, the client's choice is (1), only the maximum discount applies. In this case, if the attendee is an early registrant, it does not matter whether they are a senior or not. If a value “doesn't matter” in the final decision, we can make the table shorter by collapsing all of the that value's possibilities into one row. We conventionally call this an *any* value and mark it with a dash.

Table 7.2: Discounts (complete)

reg#	senior?	discount
-100	-	10%
101-200	-	8%
201-	T	5%
201-	F	0%

This table has only four real rows, but each of the any values covers two possible values for senior?, so counts as two effective rows. This means this table has six “effective” rows, as we expected.

If the table were to have *less* than six effective rows, we would immediately know that some input was missing. If the table had *more* than six effective rows, we would immediately know that it repeats one input on two different rows, mapping them to different outputs. A table that does not miss any inputs is called *sound*, while a table that does not contradict itself is called *complete*.

This is enough to define “validity” for decision tables: a valid table is one that is both sound and complete, while an invalid table is unsound or incomplete. This means that a table without exactly the right number of rows is automatically invalid, revealing a problem with our requirements.

Exercise 33 (Exactness is not Validity)

The converse is *not* true: a table can have the correct number of rows and still be invalid. Give an example of this.

HINT: The table would be unsound *and* incomplete.

[Solution \(page 138\)](#)

I’m being careful to use *valid*, not *correct*. A decision table can be valid but incorrect—say, if it does not capture what the client asked of us. But if it is invalid then it is *definitely* incorrect. Logically, Correct => Valid. Validity is structural, correctness is businessal.

Exercise 34 (Fizzbuzz)

fizzbuzz(x: Int) is a function that returns “fizz” if x is divisible by 3, “buzz” if x is divisible by 5, “fizzbuzz” if divisible by *both* 3 and 5, and otherwise returns the number unchanged. Write the decision table for fizzbuzz.

[Solution \(page 139\)](#)

7.2 Another Requirements Example

Note

This might be removed in the next version (v0.12), unless enough people complain about that

We have some videocall software with a “share screen” feature. The host can set two options:

1. Whether more than one person can share at a time
2. Who can share (Host, Participants)

We’ll use a decision table to model whether I can share my screen or not. Based on these options, there are four possible inputs in our decision table:

1. Can only the host share?
2. Am I the host?
3. Is someone else sharing?
4. Is multishare enabled?

All of these are booleans. Our table then has $2^4 = 16$ virtual rows total. Here we go:

	O	H	S	M	out
1	T	T	T	-	ERROR
2	T	T	F	-	T
3	T	F	-	-	F
4	F	-	T	T	T
5	F	-	T	F	F
6	F	-	F	-	T

I marked row (1) as “error” because it should be an impossible state: “only the host can share”, “we are the host” and “someone else is sharing” cannot all be true at the same time. If we see this case in production, there’s a bug somewhere in our system.

Counting the rows with one any as two virtual rows, and the rows with two anys as four, we have a total of 16 unique rows. That means there’s no missing rows indicating a missing requirement, and no duplicate rows indicating a requirement contradiction.

But just because the table’s complete doesn’t mean it’s *correct*. This version of the table has a bug. Can you see it?

It’s row (5). If multishare is disabled and someone else is sharing, then I can’t share, even if I’m the host.

One nice thing about decision tables is that even nontechnical people can understand them, so you can get their involvement in checking requirements. Fixing the table:

	O	H	S	M	out
5a	F	T	T	F	T (kicks other)
5b	F	F	T	F	F

Exercise 35

Speaking of screensharing, I recently embarrassed myself on a video call. My microphone has a hardware mute switch, which I had toggled on, and assumed I didn’t need to press the software mute button. What I didn’t realize was my webcam *also* had a microphone, zoom was using it, and my hardware mute switch was doing nothing. Everybody could hear me loud and clear.

Model “Am I (m)uted” as a decision table, with the columns “(z)oom mute”, “(h)ardware mute”, and “(w)hich mic” (desk or webcam).

Bonus: how can I prevent this from happening again?

[Solution](#) (page 139)

7.3 Analyzing Code

If a code path uses branches and no loops or recursion, we can represent its high-level behavior as a decision table. This can be useful if the code depends on external sources for some of the inputs. [[While the implementation may have to spread its logic over the whole function, we can still use a decision table to organize the high-level behavior.]]

For example, Python’s [file open function](#)¹⁷ has different behavior depending on what mode string was passed in, whether the string includes a “+”, and whether the file exists. In the [implementation](#)¹⁸, the function parses the mode string and then,

¹⁷ <https://docs.python.org/3/library/functions.html#open>

¹⁸ https://github.com/python/cpython/blob/main/Modules/_io/fileio.c#L248

much later, checks if the file exists. Representing the function as a decision table makes it easier to see the high-level behavior.

mode	file_exists?	effect
r	T	open
r	F	error
w	T	truncates file
w	F	creates file
a	T	appends to file
a	F	creates file
x	T	error
x	F	creates file
' '	-	same as r

7.4 Techniques

7.4.1 Separate Independent Outputs

In Python’s open function’s mode string, we can write "r+" instead of "r". Adding +? as an input to the table would bring it from ten effective rows to twenty. It would also not benefit us in any way, because the + does not change any of the file effects.

Writing "r+" instead of "r" instead changes the file handle’s read/write permissions. But those permissions do not change if the file already exists or not, unless that would be the source of an error (like "x+" on an existing file).

These independences may not be obvious to someone looking at a giant twenty-row, six-column table. We can instead present them with two smaller tables that only contain the inputs relevant to each decision.

+?	key	can_read	can_write
T	r	T	T
T	w	T	T
T	a	T	T
T	x	T	T
T	' '	error	/
F	r	T	F
F	w	F	T
F	a	F	T
F	x	F	T
F	' '	T	F

There are trade offs here in how concise vs comprehensive we want the tables to be. If it is important for the table to keep track of all error cases, then we need all three inputs.

7.4.2 Representing Mutations

Decision tables aren't the best tool for representing a whole lot of state changes, or something getting changed multiple times, but sometimes it's useful to show how a single value changes.

```
# example code
if x % 2 == 0
    x = x/2;
else
    x = 3*x+1;
```

In a previous chapter we represented changes with *old(x)* (page 48). In this case, we are going to borrow a mathematical notation that is a little more compact. In some branches of math and science, they write x' to mean the *new* value of x after some change. Here, that would give us the table

$x \% 2$	x'
0	$x/2$
1	$3x+1$

Ideally, the table should model a *single* step. This means that we can update multiple values in the same table, as in a swap function:

$$\frac{x' \quad y'}{y \quad x}$$

We could represent the values in the step *after* with x'' , but if we are doing that then decision tables are probably the wrong tool for the job.

7.4.3 Impossible Rows

If we want to demonstrate not just that an output is impossible, but that a particular combination of *inputs* is impossible, we can use a / to say not just “it doesn’t matter”, but “it’s not possible”.

password-correct?	2auth-enabled?	2auth-correct?	Login
T	T	T	T
T	T	F	F
T	F	/	T
F	-	/	F

Logically, this is the same as writing a dash/any, but it signals to the reader that it shouldn’t happen, not that it doesn’t matter.

7.4.4 Validity Footguns

The easiest way to accidentally make a table unsound is through misuse of anys. Fortunately, this is also relatively easy to detect.

Table 7.4: Unsound Table

A	B	o
T	-	T
-	T	F

If we expand the anys, we have both $TT \rightarrow T$ and $TT \rightarrow F$, which is unsound. Here we can detect the issue because the table isn’t complete, as we don’t have a row for FF . If we add that as a special case, then we’ll have 5 effective rows total, which should alert us that the table is unsound.

One easy way to avoid this is to never place an any to the left of a fixed value. This can lead to some table bloat but is better than having an invalid table!

Another common mistake is to not exhaustively enumerate all values in a column.

count	o
-10	T
11-20	F
21-30	T

If we know for sure count maxes out at 30 then this table is complete. If it can go higher, we have not covered all possibilities, so the table is incomplete.

7.5 When is a Table the Wrong Choice?

Four questions I ask myself when considering a decision table:

- 1. Am I modeling something with a clear map between independent inputs and outputs?
- 2. Can I cleanly and concisely enumerate the inputs in a sensible way?
- 3. Would a table be the most useful way of presenting this information?
- 4. Would the table be legible?

Question (1) is “no” if the inputs strongly depend on each other, if the decision involves lots of side effects, or if the decision cannot be made “instantly”. If the decision requires a loop or recursion, tables are likely insufficient.

Question (2) is a “no” if some input has an infinitely-many values and there is no way of grouping them, or if one of the inputs is a list or other complex type.

Question (3) is a “no” if the decision can be represented by a simpler equation. This can happen with overrides, like if a CLI is configured based on code defaults, user options, and parameter flags. We could write a table where each column has three possibilities, and show the final configuration setting for all 27 rows. But it could be clearer to instead say “flags always override user options override user defaults”.

Question (4) is a “no” if the table is too big. How big is “too big” depends on a lot of factors but a very rough rule of thumb is that the table should all fit on one sheet of paper or monitor screen. If one column has eight possible values, or if the decision depends on sixteen input columns, then the table will be illegible.

7.6 Summary

- Truth tables enumerate every possible input and output of a logical expression.
- Decision tables extend truth tables to systems, by allowing multiple inputs and outputs.
- Decision tables are *complete* if they have no missing inputs, and *sound* if they don't have the same set of inputs twice.

In the next chapter, we will look at how logic can be used to better understand databases.

7.6.1 Learn More

Decision tables are an example of a *formal specification*. We have already written plenty “formal specifications” in previous chapters. What makes the term matter is just “a specification we can check for validity and manipulate.” We will cover formal specification in more detail in later chapters.

[[Other tools in the same very general category of decision tables include flowcharts, fault trees, and state machine diagrams.]]

[[TODO Something about combinatorics and Parnas tables]]

Chapter 8

Databases

So far, we have only applied logic to understand “software in motion”: algorithms and how they execute. But it is also an extraordinarily powerful tool for understanding databases, too. For simplicity we will restrict our attention to only SQL-driven relational databases. However, many of these concepts are adaptable with CSVs, dataframes, document databases, etc.

8.1 A Relational Model Overview

Modern relational databases are based on Edgar F. Codd’s *relational model*. We will not go into comprehensive detail on the model but provide an overview we need. See the *Further Reading* (page 84) for a deeper dive.

A database is a set of tables, and a table is a set of records. What is a record? We can start by saying that a record is an ordered list of values, or *tuples*. One example database could be

```
db = {users, groups, user_groups}

users = {
    (1, "h@hillelwayne.com"),
    (2, "kate@example.com"),
    (3, "asher@example.com"),
    # etc
}

# etc
```

For a given record *u*, we can get the first element with *u*[0], the second with *u*[1], etc. This satisfies our needs for both inconvenience and incomprehensibility. It would be better to give the elements names and types, so we could write *u*.id instead of *u*[0]. And, as we can create new notation *whenever we want* (page 16), I will add a simple record syntax:

```
record users {
    id: Int
    email: String | {NULL}
}
```

Recall that $|$ is *set union* (page 11). We will define this to mean that each element of the set `users` has two fields, one called `id` that must be an integer and one called `email` that may be a string or null. You may note this is very similar to how we would define a table in SQL:

```
CREATE TABLE users (
    id int NOT NULL,
    email varchar(9999)
)
```

This is intentional! We can choose our logical notation to closely match the software systems we build. At the same time, I chose to make nullable fields explicit, where SQL makes them the default. I find that leads to fewer mistakes.

That said, I find writing `set | {NULL}` a little unwieldy, so I will add a bit of syntactic sugar and define `set + x` to mean `set | {x}`, so that we can write `email: String + NULL`.

In any case, because tables are just sets, we can quantify over them like any other sets. And this is where the logical model really shines. Our two quantifiers represent the two essential purposes of a database: querying data and ensuring data integrity.

8.2 Querying Data

Given the query

```
SELECT u.email FROM users as u WHERE u.id = 5
```

The results of this query can be represented as the set filter $\{(u.\text{email}) \text{ for } u \text{ in users: } u.\text{id} == 5\}$. It will help us down the road if we can instead explore the properties of queries as if they were some expressions. So we can instead ask “does the query return any results at all”? That question corresponds to the expression `some u in users: u.id == 5`. If we wanted multiple `WHERE` clauses, we could just add more clauses to the logical expression:

```
SELECT * FROM users as u WHERE
    u.id = 5 AND (u.email = "" OR u.email IS NULL)

-- some u in users:
--   1. u.id == 5
--   2. (u.email == "" or u.email == null)
```

Nested subqueries and common table expressions are just nested quantifiers. If I want to query if user five belongs to *any* groups, I could write


```

SELECT * FROM users AS u WHERE
    u.id in (SELECT ug.user_ids FROM user_groups AS ug)

-- some u in users:
--   some ug in user_groups:
--       u.id == g.user_id

```

What does logical representation of queries actually get us? For one, it means we can abstract complicated SQL expressions with predicates. When do two users belong to the same group? When each user has a group membership for the same group. As a predicate, this is easy:

```

Member(user: users, group: groups) =
    some ug in user_groups:
        1. ug.user_id == user.id
        2. ug.group_id == group.id

Connected(u1, u2: users) =
    some g in groups:
        1. Member(u1, g)
        2. Member(u2, g)

```

And from there we can compose our predicates to make more complex queries. *But SQL databases cannot do this.* Standard SQL does not support directly using user-defined predicates in queries. As a consolation prize, we can instead generate the set of all values that pass a predicate and then use that set in other queries. The database term for this is a *view*.

```

CREATE VIEW memberships AS
    SELECT u.email, g.name FROM users AS u, groups AS g
    WHERE (
        SELECT COUNT(*) FROM user_groups AS ug
        WHERE ug.user_id = u.id AND ug.group_id = g.id
    );

-- memberships =
--   {(u.id, g.id) for u in user, g in groups: Member(u, g)}

```

Of course that's not how any actual SQL user would write memberships. They would use a join!

8.2.1 SQL Joins

A SQL join connects the information in two tables, for example:

```
-- users without an email in groups

SELECT * FROM users as u
      INNER JOIN user_groups as gu
      ON gu.user_id = u.id
      WHERE u.email IS NULL
```

Most SQL tutorials “explain” joins in terms of set unions and intersection, often with diagrams like this:

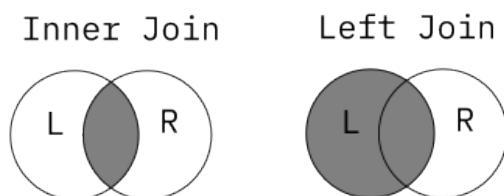


Fig. 8.1: A BAD explanation of joins

But this makes no sense. `users` and `user_groups` are disjoint sets, so the intersection should be empty!

To *properly* represent an inner join, we need to introduce one small new set operation. The *Cartesian product* of two sets S and T is the set of all tuples where the first element of the tuple is in S and the second element is in T . We can formally define this via the [set map](#) (page 12):

$$S \times T = \{(s, t) \text{ for } S \text{ in } S, t \text{ in } T\}$$

$$S \times T \times U = \{(s, t, u) \text{ for } S \text{ in } S, t \text{ in } T, u \text{ in } U\}$$

etc

For example, the Cartesian product of $\text{Nat} \times \text{Alphabet}$ is the set containing $(0, a)$, $(0, b)$, $(1, a)$, etc. The operator is named after René Descartes, who pioneered its use, and is called a “product” because $\#(S \times T) == \#S * \#T$ (where $\#S$ is the number of elements in S).

Exercise 36 (Cartesian Cardinalities)

Show that, if S and T are finite sets, then $\#(S \times T) == \#S * \#T$.

HINT: Think geometrically.

[Solution](#) (page 139)

With the Cartesian product, we can represent the inner join like this:

```
some (u, gu) in users x user_groups:
  1. u.id == gu.user_id
  2. u.email == NULL
```

There is no difference between the WHERE and ON clauses in the logical representation, just as there is no difference in SQL: most dialects will happily let you put a join condition in the WHERE or a filter in the ON. If we want to inner join across three tables, the syntax is exactly the same:

```
some (u, gu, g) in users x user_groups x groups:
  1. u.id == gu.user_id
  2. gu.group_id = g.id
```

Outer joins are more difficult for beginners to learn, which may be related to the fact that outer joins are also more complex to represent logically. A left outer join on S and T returns all the same rows as an inner join, but also the rows of S that don't join with *any* rows on T . This is the same as this query:

```
|| some (u, gu) in users x user_groups:
    u.id == gu.user_id

|| some (u, null) in users x {NULL}:
    all gu in user_groups:
        u.id != gu.user_id
```

A right outer join is defined analogously.

Note

What about aggregate functions, like GROUP BY? This is where our logic breaks down a little. Aggregates were never part of the relational model and act more like “postprocessing” steps on the query. I have not found any good formal models but personally think of them as “partition functions”.

given

```
SELECT g, h, aggrfunc(t) FROM Table AS t WHERE P
GROUP BY t.g, t.h
```

```
# we could write
partition_set = {(t.g, t.h) for t in Table}
rows_for(g, h) =
    {t for t in Table: t.g == g && t.h == h && P(t)}
{(g, h, aggrfunc(rows_for(g, h))) for (g, h) in partition_set}
```

It's kind of a mess but it can be worked with given patience. Please do not ask me about window functions.

8.3 Database Constraints

Getting data out of a database is only half of the challenge. The other half is getting data *into* the database, and more importantly keeping it correct. We don't want records to be missing values or foreign keys, or have duplicate ids, or miss any of the application specific requirements like “user balances must be above zero” or “no more than ten records can be active at once.”

We can start by looking at some ways databases represent constraints:

```
CREATE TABLE users (
    id INTEGER
    balance integer,
    email TEXT,
    -- ...
    CHECK (balance > 0),
    UNIQUE (email)
);

CREATE TABLE user_groups (
    user_id INTEGER,
    -- ...
    FOREIGN KEY(user_id) REFERENCES users(id)
);
```

These two tables define three constraints, each with its own special syntax. One applies to each record in a table, one to every *pair* of records in the table, and one to records between tables. All three constraints can be directly represented with logical expressions. Starting with “user balances must be above zero”:

```
constraint UserPositiveBalances = all u in users:
    u.balance > 0
```

I put constraint in front because it helps me distinguish predicates that represent actual system constraints from helper predicates. All logical constraints that of the form $\text{all } x \text{ in set: } P(x)$ — that is, constraints over a single table row— can be implemented via CHECK. UNIQUE is a standard uniqueness predicate (using *disj* (page 18)):

```
constraint userEmailUnique =
  all disj u1, u2 in Users:
    u1.email != u2.email
```

The last constraint, a foreign key constraint, says that every `user_group` record has a corresponding user record. Another way of thinking about this that if `user_group.user_id == 17`, there *must exist* a user with id 17, which means the *query* `some u in users: u.id == 17` is true. In other words, foreign keys constrain each record to guarantee a query! For this reason we can represent key constraints with a nested quantifier, placing a *some* inside an *all*.

```
constraint UserGroupUserFK =
  all ug in user_groups:
    some u in users:
      ug.user_id = u.id
```

Or, to make things simpler:

```
FK(from_tbl, to_tbl, col, to_col)
  all record in from_tbl:
    some ref in to_tbl:
      record[col] = ref[to_col]
```

```
constraint UserGroupUserFK =
  FK(user_groups, users, user_id, id)
```

Note

This raises a question: if all-some nested quantifiers have a deep meaning in databases, does some-all mean anything? I have no idea. I welcome suggestions from readers.

Exercise 37 (Compound keys)

SQL UNIQUE constraints can refer to multiple columns. If I add UNIQUE (user_id, group_id) to user_groups, this means that different user_group records can share the same user id or the same group id, but not both at the same time. Write this as a constraint.

[Solution](#) (page 139)

Exercise 38

Write the constraint “If a user belongs to a group, the user must have a non-null email”.

HINT: use =>.

[Solution](#) (page 140)

Exercise 39

Let #S be the number of elements in S. Write the constraint “all groups can only have five members at most.”

HINT: Use *set filter* (page 12).

[Solution](#) (page 140)

8.4 Constraints Are Queries

Predicate logic can express an enormous number of interesting constraints. Each of the mechanisms earlier can only implement a narrow subclass:

- REFERENCES only implements all s in S: (some t in T: s.col1 = t.col2).
- UNIQUE only implements all x, y in S: (x.col1 != y.col1) || (x.col2 != y.col2) ...
- CHECK only implements all x in S: P(x), **where P does not use any quantifiers**. In other words, CHECK cannot constrain a row based on other rows.

How would we implement something like “all books have an author born before the publication date”? It is easy to express logically:

```

constraint NoTimeTravel =
  all b in books:
    some a in authors:
      1. b.author_id = a.id
      2. b.published_on > a.birthday

```

Implementing NoTimeTravel is another matter. It cannot be done with REFERENCES (as it has two conditions), nor with CHECK (as it uses a second quantifier), nor with UNIQUE (obvious). So can our databases enforce this, or are we limited to application-side validation?

It turns out most SQL databases, can, in fact, enforce this constraint! We just have to put in a little work and apply some logical rules to get there. First, we can split NoTimeTravel into an “easy” constraint and a “hard” constraint:

```

# Just a foreign key
constraint BooksAuthorsFK =
  all b in books:
    some a in authors:
      b.author_id = a.id

constraint NoTimeTravel =
  all b in books:
    all a in authors:
      b.author_id != a.id ||
      b.published_on > a.birthday

```

We turned the some in NoTimeTravel into an all. We also changed the body to be “either the author is different or the book was published after the author was born”. While we could have used $a \Rightarrow$ instead, SQL does not have an implication operator, so writing $!P \parallel Q$ keeps us closer to the eventual implementation. This, combined with BooksAuthorsFK forcing each book to have exactly one author, is equivalent to our original constraint. Our next step is not strictly necessary, but will clarify our final outcome:

```

constraint NoTimeTravel =
  all (b, a) in books x authors:
    b.author_id != a.id ||
    b.published_on > a.birthday

```

Now for the insight. If this constraint does not hold, there must be a specific (book, author) counterexample that violates it. And we can write a query to find the counterexample! If the query turns up nothing we know there are no counterexamples, meaning the constraint holds. This is just another example of logical duality!

```

constraint NoTimeTravel =
  !some (b, a) in books x authors:
    !(b.author_id != a.id ||
      b.published_on > a.birthday)

# Apply De Morgan's Law
constraint NoTimeTravel =
  !some (b, a) in books x authors:
    1. b.author_id = a.id
    2. b.published_on <= a.birthday

```

This directly maps to a SQL query:

```

-- NoTimeTravel holds if this finds 0 rows

SELECT COUNT(*) FROM books as b
  INNER JOIN authors as a
    ON b.author_id = a.id
  AND b.published_on <= a.birthday;

```

To actually enforce the constraint, we can use a SQL “trigger”, or a stored procedure set to run on row or table changes. All we need to do is declare a trigger that makes this query and raises an exception if the query is nonempty. For brevity, an example is separately provided with the [code samples](#)¹⁹.

8.4.1 State Change Constraints

SQL triggers have one other useful feature: when triggered by a record update, they can check constraints on how the record changed. We can for example enforce that an `updated_at` timestamp can only go forwards in time or that when a nullable field has a non-NULL value, it cannot be set back to NULL.

We have already represented changes in previous chapters: in [change assertions](#) (page 48) we used `old(x)` and `x`, while in [decision tables](#) (page 69) we used `x` and `x'` (“x prime”). SQL syntax uses `NEW` and `OLD`, but to make a [later chapter](#) (page 95) easier I will use the prime syntax right now.

```

constraint NoNullAfterAdmin =
  all g in Groups:
    g.admin_id != NULL => g.admin_id' != NULL

```

Once the `admin_id` is not null, the next value cannot be null either, meaning that it can never go back to null.

¹⁹ <https://github.com/logicforprogrammers/book-assets>

This is also useful for state machine columns: a record can go WAITING -> READY or READY -> DONE, but not WAITING -> DONE. In that case it's considered good form to "allow x to change to itself":

```
constraint StateMachineTransitions ==
  all t in tasks:
    1. t.status = "WAITING" => t.status' in {"WAITING", "READY"}
    2. t.status = "READY" => t.status' in {"READY", "DONE"}
```

Exercise 40 (Transition Helper)

Write a helper predicate `ValidTransitions(task, from, to)`, so that we can write the body of `StateMachineTransitions` this way:

```
all t in tasks:
  1. ValidTransitions(task, "WAITING", {"READY"})
  2. ValidTransitions(task, "READY", {"DONE"})
```

Note that `to` is going to be a set of transitions.

[Solution](#) (page 140)

8.5 Summary

1. Databases are sets of tables, which are sets of records.
2. The some quantifiers corresponds to database queries. Joins are queries over the *Cartesian product* of two or more tables. In the case of SQL, most some expressions are directly translatable to queries, though you may need to inline abstract predicates.
3. all expressions correspond to database constraints, and all-some nested expressions correspond to foreign keys. Databases have different features for enforcing constraints. Constraints may be on data, or how data changes.
4. By using duality, we can check a constraint by querying its negation. SQL databases can use this to enforce complex constraints, via triggers.

So far we've been keeping the logic very close to the database: we're talking about properties of database tables and records. [[But the database is just an imperfect implementation of the data model, the conceptual slice of the world we're trying to make legible.]] Next chapter we will use logic to study our data model, one level of abstraction higher.

8.5.1 Further Reading

The database representation in this chapter comes from Edgar Codd's *Relational Model*. The relational model was first introduced in [A relational model of data for large shared data banks](#)²⁰, along with a set of operators that made the *Relational Algebra*. A gentler introduction to relational algebra can be found [here](#)²¹. SQL is based on relational algebra but does not follow it in its entirety.

The best way to learn about the capabilities of database invariants is to read the official database documentation. While this chapter is compatible with SQLite, the best documented is arguably Postgres:

- [CHECK constraints](#)²²
- [Trigger constraints](#)²³

²⁰ <https://dl.acm.org/doi/10.1145/362384.362685>

²¹ <https://cs186berkeley.net/notes/note6/>

²² <https://www.postgresql.org/docs/current/ddl-constraints.html>

²³ <https://www.postgresql.org/docs/current/sql-createtrigger.html>

Chapter 9

Data Modeling

Note

This is scheduled for a rewrite and needs to be updated after the databases chapter was rewritten in v0.11

In the last chapter, we used logic to figure out database constraints. To do so, we stuck close to database semantics: foreign keys are number columns, relationships between entities go through a many-to-many table, etc.

Any database schema is only one possible representation of the abstract data model. In this chapter, we will use logic to analyze the model directly.

9.1 Abstracting from Data

Let's pull our records from the last chapter:

```
record Users {  
  id: Int  
}  
  
record Groups {  
  id: Int  
  admin_id: Int  
}  
  
record GroupMembership {  
  id: int  
  user_id: int  
  group_id: int  
}
```

I see three “implementation details” that don't matter to the abstract model:

1. I don't care whether the group id is an integer or a UUID or something else, what I *really* care about is that the groups are distinct.
2. Why is admin_id an integer? Why can't we just say the admin is a user? The database needs an integer column, but in our heads, groups have admins, not integers.

3. For that matter, why do we *need* a GroupMembership record? What we actually *intend* is that groups have members that are users. Or maybe that users belong to groups. The many-to-many table is, once again, just an implementation detail to work within the database.

This all gets in the way of thinking about the actual data model. It'd be easier to throw these all away and just focus on the users, the groups, and their relationships.

Something like this:

```
sig User {}

sig Group {
  admin: User
  members: set User
}
```

I'm using "sig" for *signature*, because these are *not* records. They're just a data model, where groups have admins and sets of users. No implementation details have leaked into my model!

(Though even this is biasing things a little: what if we instead wanted to have `member_of` be an element of the User and not the group?)

One of the constraints from last chapter, that a group's admin must also be a group member, is easily expressed like this:

```
all g in Group:
  g.admin in g.members
```

9.2 In Practice: Formal Specification

Let me start by asking two questions:

1. Is it possible for one group to have every admin in the system as members?
2. Is it possible for one group to have no members?

These aren't too complicated, and you can probably reason through them in a couple of minutes. But as the complexity of a data model grows, and we add increasingly elaborate constraints, it becomes progressively more difficult to solve these in your head. This is a place where we want the computer to check our model for us.

And this is the domain of *Formal Specification*: creating models of data (or systems) and using tools to check them for correctness. It's the other side of the formal meth-

ods coin that we first introduces in an *earlier chapter* (page 58), just checking designs instead of code.

There are many different formal specification languages, but the one I want to use now is called Alloy²⁴. I’m not going to go into too many of the specifics of Alloy; that’s beyond the scope of this book. But I’ll show you how it solves these problems.

First, we define the components of our data model and our constraints:

```
sig User {}

sig Group {
  admin: User,
  members: set User
}

pred admins_members_of_groups {
  all g: Group |
    g.admin in g.members
}

pred is_admin[u: User] {
  some g: Group |
    g.admin = u
}
```

Note that Alloy uses a different syntax for quantifiers: `all g: Group | prop` instead of `all g in Group: prop`.

Once we have the basics, we can write a “command”, telling Alloy to find examples of systems where certain properties are true. In this case, ask it for examples of groups containing all admins:

```
run group_with_all_admins {
  admins_members_of_groups &&
  some g: Group |
    all u: User |
      is_admin[u] => u in g.members
}
```

Running Alloy’s built-in analyzer (I use VSCode) gives us a visualization of the example:

Alloy can also generate new examples to visualize, change the theme, and even run a REPL on specific examples. It’s a great tool for finding unexpected situations!

²⁴ <https://alloytools.org/>

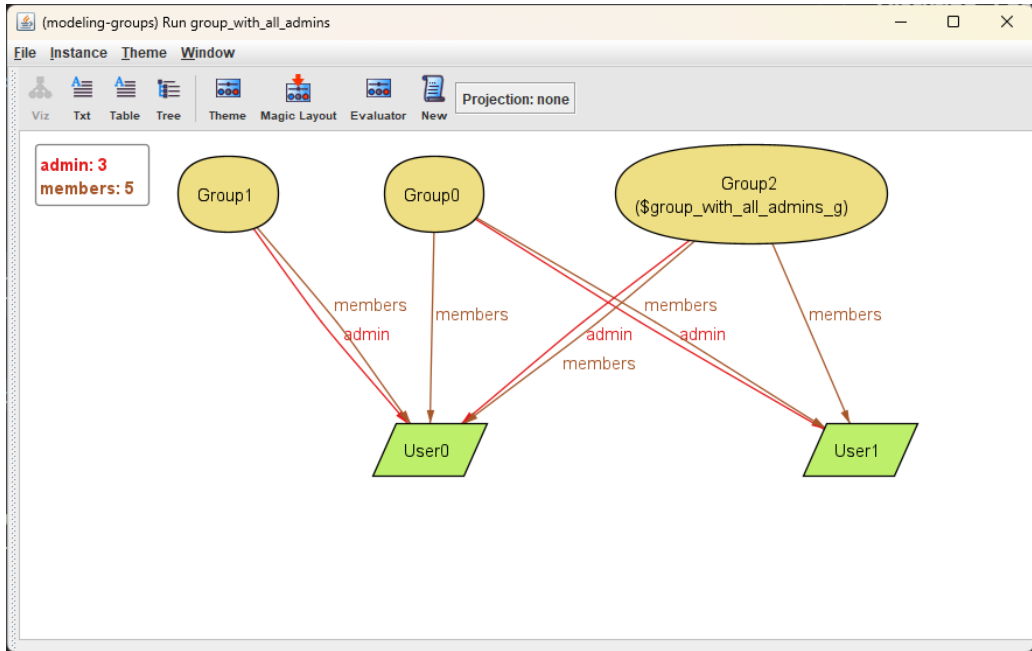


Fig. 9.1: An alloy visualization.

We can also ask Alloy to check that a property always holds. This is usually used to check that we guarantee a data invariant. For example, we might want a data invariant to be “groups are never empty”.

```
check no_empty_groups {
  admins_members_of_groups =>
    all g: Group | some g.members
}
```

Running the analyzer on this would give us a visualization of a counterexample, if it can find one. In this case, though, it doesn’t find anything, so we can be more confident the property holds.

```
Executing "Check no_empty_groups"
No counterexample found. Assertion may be valid. 2ms.
```

9.2.1 Abstractions

Specification languages live at a higher level of abstraction than programming languages, meaning they can express and check properties that would be too computationally infeasible to program. Let's add into our data model that some users can have another user who referred them. That's easy to express as a database record.

```
record Users {
  id: Int
  + referrer: Int + NULL
}
```

This implies a new data invariant: users cannot be their own referrer. As a SQL constraint, it would look something like `u.referrer != u.id`. In Alloy, it would look like this:

```
sig User {
  referrer: lone User // 0 or 1
}

pred no_self_loops {
  all u: User |
    u != u.referrer
}
```

Now, one more twist to the constraint: no user can *transitively* be their own referrer. If Alice refers Bob and Bob refers Eve, Eve cannot have referred Alice.

This is extraordinarily difficult in SQL. At the very least we'd need use recursive common table expressions, and the resulting query will be convoluted and computationally expensive.

On the other hand, transitive lookups are trivial in Alloy. In Alloy, `Alice.^referrer` is the "transitive closure" of referrals: the set containing Alice's ref, the ref's ref, the ref's ref's ref, etc.

The same constraint in Alloy:

```
pred no_cycles {
  all u: User |
    !(u in u.^referrer)
}
```

9.2.2 and Implementations

It's good that we can express the constraint in Alloy, but that doesn't help us with our actual SQL database. SQL still doesn't cleanly support transitive lookups.

But we can use Alloy to figure out an implementable SQL constraint that *also* guarantees no_cycles. Then we'd test

```
check { implementable_property => no_cycles }
```

In this case we'd say that `implementable_property` is *stronger* than `no_cycles`. One idea I have would be to place some ordering on users, like `id` or `signup date`. Then I'd predict that if we could only refer someone with an earlier `signup date`, we wouldn't have any cycles. This would be relatively easy to check in SQL.

Exercise 41

Write the constraint (in our notation, not Alloy's) "If a user has a referrer, the user's `created_at` is later than the referrers `created_at`."

[Solution](#) (page 140)

In Alloy:

```
sig User {
  referrer: lone User,
  created_at: disj Int
}

pred referral_must_come_later {
  all u, ref: User |
    u.referrer = ref => gt[u.created_at, ref.created_at]
}
```

Now we can check that our implementable constraint guarantees our data model property:

```
check implementation_works {
  referral_must_come_later => no_cycles
}
```

Alloy passes this with no counterexample, so we can be confident this constraint does what we want. Alloy helped us find a cheap way of enforcing an expensive data model constraint.

The technical term for “showing an implementation matches a more abstract model” is *refinement*.

9.3 Finding Bugs with Specifications

The main use-case of formal specifications is to find errors in designs. Design errors are more expensive than code errors, and so are more important to detect early. [[Since formal specifications live at a higher level of abstraction, they can more easily find design errors.]]

When I teach Alloy, I demonstrate this with a simplified model of access permissions. We have a set of Users and Resources. Resources can only be read by Users in their `readable_by` set.

```
sig User {}

sig Resource {
  readable_by: set User
}

pred can_access[u: User, r: Resource] {
  u in r.readable_by
}

run {some u: User, r: Resource | can_access[u, r]}
```

On top of this, we add that some resources have *parents*. If our resources are files, the parent could be the containing folder. As with our prior example of referrals, no resource can transitively be its own parent.

```
sig Resource {
  readable_by: set User
+ ,parent: lone Resource
}
+ fact no_cycles {
+ no r: Resource |
+   r in r.^parent
+}
```

Finally, we amend the access rule, so that a user can access a resource if they have permission to read its parent.

```
pred can_access[u: User, r: Resource] {
  u in r.readable_by
```

(continues on next page)

(continued from previous page)

```
+ || u in r.parent.readable_by
}
```

After adding this, I ask my class “if we can access a resource, are we guaranteed to access all of its children?”

```
assert parent_implies_child {
  all u: User, r: Resource |
    can_access[u, r] =>
      all child: r.~parent | //r.~parent is `children of r`
        can_access[u, child]
}
```

```
check parent_implies_child
```

Most people are surprised to find out *no*, this property does *not* hold! As before, we can see the counterexample as a graph visualizatin, but we can also output it as an ASCII table:

this/Resource	readable_by	parent
Resource\$0	User\$0	
Resource\$1		Resource\$0
Resource\$2		Resource\$1

To explain the error instance, this is the problem:

1. We start with three resources: Parent (Resource\$0), Child (Resource\$1), and Grandchild (Resource\$2). *Only* Parent has the User in readable_by.
2. Because the User can access Parent, the property asserts they can access Child.
3. In checking Child, we see that User in Parent.readable_by, so we can access it.
4. Because we have access to Child, the property asserts we can access Grandchild.
5. The User is not in Child or Grandchild’s readable_by. So we cannot access Grandchild.
6. We can access Child but not all of its children, leading to a property violation.

This is the real power of formal specification: the full spec is less than 30 lines and still finds a subtle error many experienced developers miss. To fix this, we can modify `readable_by` to transitively check a resource's entire ancestry.

```
pred can_access[u: User, r: Resource] {
  u in r.readable_by
  - || u in r.parent.readable_by
  + || u in r.^parent.readable_by
}
```

(As before, we would still need to find a way to implement a transitive constraint in our database. But it is always better to be working on implementing a correct design than to implement a possibly-broken one.)

9.4 Summary

1. We can represent data (or other systems) at a higher level of abstraction than what the database implements.
2. By doing this, we can test the abstractions directly, in a formal specification language.
3. Alloy is one such formal specification language, and can produce visualizations of satisfying properties. It can also test that properties hold.
4. Because we're at a higher level of abstraction, we can express invariants that would be impossible to directly enforce at the database level.
5. Alloy can test if an implementable constraint also guarantees an abstract invariant.

While data modeling is a good use case for formal specification, it really shines for modeling concurrent systems. In the next chapter, we will show how a formal specification can find race conditions in a software design.

9.4.1 Further Reading

- [Alloy Docs](https://alloy.readthedocs.io/en/latest/)²⁵
- [Formal Software Design with Alloy 6](https://haslab.github.io/formal-software-design/)²⁶
- [Software Abstractions](https://mitpress.mit.edu/9780262528900/software-abstractions/)²⁷ (book)

²⁵ <https://alloy.readthedocs.io/en/latest/>

²⁶ <https://haslab.github.io/formal-software-design/>

²⁷ <https://mitpress.mit.edu/9780262528900/software-abstractions/>

Examples of Alloy models:

- [Modeling Database tables in Alloy](#)²⁸
- [Modeling Git Internals in Alloy](#)²⁹ (3-parter)
- [Storm Surges](#)³⁰

²⁸ <https://bytes.zone/posts/modeling-database-tables-in-alloy/>

²⁹ <https://bytes.zone/posts/modeling-git-internals-in-alloy-part-3-operations-on-blobs-and-trees/>

³⁰ <https://jwbaugh.github.io/papers/baugh-abz-2016.pdf>

Chapter 10

System Modeling

In the last chapter, we showed how formal specification can be used to analyze a data model and look for problems. But that's only the tip of the specification iceberg. We can also use it to model *systems*.

10.1 Situation

We have some bank users. Bank users can wire money to each other. We have over-draft protection, so wires cannot reduce an account value below zero. That's easy to guarantee, just throw an *if* check on each wire and you're done!

...But what if users can send multiple wires at the same time? What if a computer crashes in the middle of processing a wire? What if someone tries to send *themselves* money? What if someone tries to send themselves money in multiple wires at the same time, and then one of the servers crash?

This is why we need to model systems. We want to see that our properties hold under *every possible* behavior, not just on the happy path.

And we'll use logic to model it.

10.2 The Logic

We're going to handle this system in three stages. First, we'll see how our regular predicate logic is enough to accurately model our problem. Then, we'll make a simple extension to our logic to more elegantly express the spec. Finally, we'll translate it to a real tool that can directly check our logic for errors.

For now we'll assume an extremely simple system: two hardcoded variables *alice* and *bob*, both start with 10 dollars, and transfers are only from Alice to Bob. Also, the transfer is totally atomic: we check for adequate funds, withdraw, and deposit all in a single moment of time. Our modeled system will be more complex; this is just to relate the ideas.

First, let's look at a valid *behavior* of the system, or possible way it can evolve.

```
alice: 10 -> 5  -> 3  -> 3  -> ...  
bob:   10 -> 15 -> 17 -> 17 -> ...
```

In programming, we'd think of *alice* and *bob* as variables that change. How can we express those variables *purely* in terms of predicate logic? One way would be to replace them with *arrays* of values. *alice*[0] is the initial state of *alice*, *alice*[1] is after the first time step, etc. Time, then, is “just” the set of natural numbers.

```
Time  = {0, 1, 2, 3, ...}
alice = [10, 5, 3, 3, ...]
bob   = [10, 15, 17, 17, ...]
```

That is a valid behavior. Here are some *invalid* behaviors:

```
alice = [10, 3, ...]
bob   = [10 15, ...]

alice = [10, -1, ...]
bob   = [10 21, ...]
```

The first is invalid because Bob received more money than Alice lost. The second is invalid because it violates our proposed invariant, that accounts cannot go negative. Can we write a predicate that is *true* for valid transitions and *false* for some transition in our two invalid behaviors?

Here's one way:

```
Time = Nat

Transfer(t: Time) =
  some value in 0..=alice[t]:
    1. alice[t+1] == alice[t] - value
    2. bob[t+1] == bob[t] + value
```

Go through and check that this is true for every *t* in the valid behavior and false for at least one *t* in the invalid behavior. Note that the steps where Alice *doesn't* send a transfer also pass *Transfer*; we just pick *value* = 0.

I can now write a predicate that perfectly describes what a “valid behavior” is:

```
Spec =
  1. alice[0] == 10
  2. bob[0]   == 10
  3. all t in Time:
      Transfer(t)
```

Now allowing “nothing happens” as “Alice sends an empty transfer” is a little bit weird. In the real system, we probably don't want people to constantly be sending each other zero dollars:

```

Transfer(t: Time) =
- some value in 0..=alice[t]:
+ some value in 1..=alice[t]:
    1. alice[t+1] == alice[t] - value
    2. bob[t+1] == bob[t] + value

```

But now there can't be a timestep where nothing happens. And that means *no* behavior is valid!

Exercise 42 (No valid behaviors)

Explain why the current version of the spec cannot have *any* valid behaviors, ie for at least some t , $\text{Transfer}(t)$ is false.

Hint: Remember, Time is an alias for the natural numbers, meaning every behavior has an *infinite* number of steps.

[Solution](#) (page 141)

So typically when modeling we add a *stutter step*, like this:

```

Spec =
  1. alice[0] == 10
  2. bob[0] == 10
  3. all t in Time:
    || Transfer(t)
    || 1. alice[t+1] == alice[t]
       2. bob[t+1] == bob[t]

```

(This is also why we can use infinite behaviors to model a finite algorithm. If the algorithm completes at $t=21$, $t=22,23,24\dots$ are all stutter steps.)

There's enough moving parts here that I'd want to break it into subpredicates.

```

Init =
  1. alice[0] == 10
  2. bob[0] == 10

Stutter(t) =
  1. alice[t+1] == alice[t]
  2. bob[t+1] == bob[t]

Next(t) = Transfer(t) // forshadowing

Spec =

```

(continues on next page)

(continued from previous page)

```

1. Init
2. all t in Time:
    Next(t) || Stutter(t)

```

Now finally, how do we represent the property NoOverdrafts? It's an *invariant* that has to be true at all times. So we do the same thing we did in Spec, write a predicate over all times.

```

property NoOverdrafts =
  all t in Time:
    alice[t] >= 0

```

We can even say that $\text{Spec} \Rightarrow \text{NoOverdrafts}$, ie if a behavior is valid under Spec, it satisfies NoOverdrafts.

Exercise 43 (Extending to Bob)

Modify the Next so that Bob can send Alice transfers, too. Don't try to be too clever, just do this in the most direct way possible.

Bonus: can Alice and Bob transfer to each other in the same step?

[Solution](#) (page 141)

10.2.1 Temporal Logic

This is good and all, but in practice, there's two downsides to treating time as a set we can quantify over:

1. It's cumbersome. We have to write $\text{var}[t]$ and $\text{var}[t+1]$ all over the place.
2. It's too powerful. We can write expressions like $\text{alice}[t^2-5] == \text{alice}[t] + t$.

Problem (2) might seem like a good thing; isn't the whole *point* of logic to be expressive? But we have a long-term goal in mind: getting a computer to check our formal specification. We need to limit the expressivity of our model to make it tractable to our tooling.

In practice, this will mean making time implicit to our model, instead of explicitly quantifying over it.

[[The first thing we need to do is limit how we can use time.]] At a given point in time, all we can look at is the *current* value of a variable ($\text{var}[t]$) and the *next* value ($\text{var}[t+1]$). No $\text{var}[t+16]$ or $\text{var}[t-1]$ or anything else complicated.

And it turns out we've already seen a mathematical convention for expressing this: *priming* (page 69)! For a given time t , we can define var to mean $\text{var}[t]$ and var' to mean $\text{var}[t+1]$. Then $\text{Transfer}(t)$ becomes

```
Transfer =
  some value in 1..=alice:
    1. alice' == alice - value
    2. bob' == bob + value
```

We don't even need to parameterize Transfer by time anymore! A predicate with primes in the body is sometimes called an *action*.

Exercise 44 (Stuttering with Primes)

Rewrite $\text{Stutter}(t)$ to use primes instead of t .

[Solution](#) (page 141)

Next we have the construct $\text{all } t \text{ in Time: } P(t)$ in both Spec and NoOverdrafts . In other words, “ P is always true”. So we can add *always* as a new term. Logicians conventionally use $\Box$ or $[]$ to mean the same thing.

```
property NoOverdrafts =
  always (alice >= 0 && bob >= 0)
  // or [](alice >= 0 && bob >= 0)

Spec =
  Init && always (Next || Stutter)
```

Exercise 45 (Always rules)

Here we will use $[]P$ to mean *always* P .

1. Show that that $[](\text{all } x: P(x))$ is equivalent to $\text{all } x: []P(x)$, where P is some sort of temporal predicate (which implicitly takes a time).
2. Show that $[](P \ \&\& \ Q)$ is the same as $[]P \ \&\& \ []Q$

[Solution](#) (page 142)

Now time is *almost* completely implicit in our spec, with just one exception: Init has $\text{alice}[0]$ and $\text{bob}[0]$. We just need one more convention: if a variable is referenced *outside* of the scope of a temporal operator, it means $\text{var}[0]$. Since Init is outside of the $[]$, it becomes

```
Init =
  1. alice == 10
  2. bob == 10
```

And with that, we've removed Time as an explicit value in our model.

The addition of primes and always makes this a *temporal logic*: one that can model how things change over time. And that makes it ideal for modeling software systems.

Note

You don't *have* to make a temporal logic to analyze systems. Before 2022, Alloy users modeled systems by making an explicit Time signature. But this proved to be cumbersome, so in 2022 Alloy incorporated a temporal logic model.

Regardless, we'll be using a specification language that was designed with temporal logic from the ground up.

10.3 In Practice: TLA+

One of the most popular specification languages for modeling these kinds of concurrent systems is *TLA+*. TLA+ was invented by the Turing award-winner Leslie Lamport, who also invented a wide variety of concurrency algorithms and LaTeX. Here's our current spec in TLA+:

Listing 10.1: (TLA+)

```
---- MODULE transfers ----
EXTENDS TLC, Integers

VARIABLES alice, bob
vars == <<alice, bob>>

Init ==
  alice = 10
  /\ bob = 10

AliceToBob ==
  \E amnt \in 1..alice:
    alice' = alice - amnt
    /\ bob' = bob + amnt
```

(continues on next page)

(continued from previous page)

```

BobToAlice ==
  \E amnt \in 1..bob:
    alice' = alice + amnt
  /\ bob' = bob - amnt

Next ==
  AliceToBob
  \/ BobToAlice

Spec == Init /\ [][Next]_vars \* [] (Next \/ Stutter)

NoOverdrafts ==
  [] (alice >= 0 /\ bob >= 0)

====

```

TLA+ uses ASCII versions of mathematicians notation: $\wedge$ and $\vee$ for $\&\&$ and $\vee$, $\forall$ and $\exists$ for all/some, etc. $==$ is used for definition, and $[][Next]_vars$ is TLA+ notation for $[(Next \parallel Stutter)]$.

Now that we have a specification and a property, we can use a *model checker* to generate all possible states of this system and see if any of them break our invariant. Like Alloy, TLA+ is most often checked from VSCode via [an extension](https://github.com/tlaplus/vscode-tlaplus/)³¹. But setting up a model run takes a bit of configuration, so I created a tool called [tlacli](https://github.com/hwayne/tlacli)³² to do more from the command line. It doesn't support all of TLA+'s features but is suitable for quick demos like this.

```
tlacli check transfers.tla --prop NoOverdrafts
```

And it gets no errors found:

```

Model checking completed. No error has been found.
421 states generated, 21 distinct states found.

```

So this is all well and good for our simple model, but what if more than one transaction could be in flight at the same time? Does our invariant still work if with concurrent, nonatomic transactions?

³¹ <https://github.com/tlaplus/vscode-tlaplus/>

³² <https://github.com/hwayne/tlacli>

10.3.1 Adding Concurrency

We could add concurrency to our “pure” TLA+, [[but that requires a few “TLA+-isms” I don’t feel like explaining right now.]] So instead we’re going to use *PlusCal*, a language that compiles to TLA+. It’s built-in with the TLA+ tooling and looks more like programming language than a math formula, so it’s very popular with beginners.

Listing 10.2: (TLA+/PlusCal)

```

---- MODULE transfers2 ----
EXTENDS TLC, Integers

People == {"alice", "bob"}
Money == 1..10
NumTransfers == 2

(* --algorithm wire
variables
  acct \in [People -> Money];

define
  NoOverdrafts ==
    [](\A p \in People:
      acct[p] >= 0)
end define;

process wire \in 1..NumTransfers
variable
  amnt \in 1..5;
  from \in People;
  to \in People
begin
  Check:
    if acct[from] >= amnt then
      Withdraw:
        acct[from] := acct[from] - amnt;
      Deposit:
        acct[to] := acct[to] + amnt;
    end if;
end process;
end algorithm; *)

=====

```

Most of this *looks* like a programming language with some unusual syntactic choices, but there’s some things to pay attention to. `acct` is set to *any value of* `[People ->`

Money], roughly the set of all mappings of Alice and Bob to numbers between 1 and 10. So `acct` can start as {alice: 1, bob: 10}, {alice: 3, bob: 6}, or any of the other 98 possible combinations.

Our model also starts with two distinct wires simultaneously (process wire \in 1..NumTransfers where NumTransfer == 2). Each wire has *its own* `amnt`, `from`, and `to`, which are *also* individually elements of sets. Different wires can pick different local values for this. Between this and the `amnt`, there are 40,000 possible initial states.

Inside wire we have `Check`:, `Withdraw`:, and `Deposit`:. These are **labels**, or groups of atomic actions. Each wire takes three steps to fully process: checking the balance is one step, withdrawing is one step, and depositing is one step. To a first order approximation, there are twenty possible ways the two wires can interleave. More precisely, slightly fewer, because some wires will end early at the `Check`.

Finally, `NoOverdrafts` is a straight translation of our old version, just generalized to any number of people.

To compile the PlusCal to TLA+, I ran `tlaccli translate transfers2.tla`. The translation is done in-file and appears below the code. Now let's model check it and see if `NoOverdrafts` still holds:

```
tlaccli check transfers2.tla --prop NoOverdrafts
```

If we do this, we suddenly get an error:

```
Error: Invariant NoOverdrafts is violated.
Error: The behavior up to this point is:
State 1: <Initial predicate>
/\ acct = [alice |-> 1, bob |-> 1]
/\ amnt = <<1, 1>>
/\ to = <<"bob", "alice">>
/\ from = <<"alice", "alice">>
/\ pc = <<"Check", "Check">>

\* four more states after this
```

This is an *exact* sequence of events required to trigger a violated invariant. In summary:

1. Alice has 1 dollar and creates two wires, one dollar each, to Bob.
2. Wire 1 runs `check`, sees Alice has at least a dollar, and proceeds to `Withdraw`.
3. *Before wire 1 withdraws*, Wire 2 runs the same `check`, sees the same dollar, and also proceeds to `Withdraw`.
4. Both wires withdraw one dollar, putting Alice at a negative balance.

This bug happens because checking and withdrawing are nonatomic: they happen in different time steps. If we make them happen in the same time step, the error should go away:

```
begin
- Check:
+ CheckAndWithdraw:
    if acct[from] >= amnt then
-   Withdraw:
        acct[from] := acct[from] - amnt;
+ \* remember to retranslate the file!
```

We can rerun the model checker and see that the error no longer occurs. If we want, we can set NumTransfers to 6 or add another three people, and TLA+ will seamlessly check our larger problem. This is what makes formal specification so useful for complex systems!

10.3.2 Liveness

If you look at the translation, you’d see this extra property PlusCal generated:

```
Termination == <>(\A self \in ProcSet: pc[self] = "Done")
```

ProcSet is the set of all wires (so 1..2). pc tracks the current step of each process: pc[1] = "Deposit" means which process 1 is ready to deposit. The whole quantifier is then “every process is at the ‘Done’ step.”

What about the <>?

Remember how [] was *always*, and meant all t in Time? <>, or *eventually*, instead means some t in Time.

```
Termination =
  some t in Time:
    all self \in ProcSet:
      pc[self][t] = "Done"
```

<>P means that P doesn’t need to be true at the start, but it needs to eventually *become* true in all possible timelines. This gets to one of the most powerful features of TLA+. Our invariant was a kind of *safety property*: a promise that something “bad” doesn’t happen. The other half of the coin is the *liveness property*: something “good” is guaranteed to happen. Like, for example, our processes eventually finish processing.

We can check Termination with `tlacli check wire.tla --prop Termination`. Surprisingly, it *fails*:

```

State 4:
/\ acct = [alice |-> 0, bob |-> 1]
/\ amnt = <<1, 1>>
/\ to = <<"alice", "alice">>
/\ from = <<"alice", "alice">>
/\ pc = <<"Deposit", "Done">>

```

State 5: Stuttering

“Stuttering” is TLA+-speak for “crashes”. The first wire is *almost* finished, it just has to complete “Deposit”, but crashes just before. Bob never gets his money.

By default, TLA+ assumes any process can crash at any step. [[It’s better to assume maximum perversity and force users to make their assumptions explicit.]] If we want to say the process doesn’t crash, we have to make it “fair”:

```

+ fair process wire \in 1..NumTransfers
- process wire \in 1..NumTransfers

```

Not all liveness bugs are solved so easily. Often, fixing a liveness bug requires rethinking the fundamental design. Better to do that rethinking while we’re still in the design phase, as opposed to after we released the product.

Exercise 46 (Eventually rules)

1. Show that that $\langle\!\rangle \text{some } x: P(x)$ is equivalent to $\text{some } x: \langle\!\rangle P(x)$, where P is some sort of temporal predicate (which implicitly takes a time).
2. Show that $\langle\!\rangle (P \parallel Q)$ is the same as $\langle\!\rangle P \parallel \langle\!\rangle Q$.
3. Show that $\langle\!\rangle P = ![!]P$.

[Solution](#) (page 142)

10.4 Specification in the wild

The past two chapters covered two different formal specification languages: TLA+ and Alloy. When people learn about these kinds of tools, they generally have two questions:

1. Is this actually used in the real world?
2. How do I make sure my code matches the specification?

Question one is easy to answer: there are a lot of high-profile case studies of formal specification saving everyday companies a lot of time and money. I’ve put some examples in the “Further Reading” section.

Question two is harder. As we’ve seen in the *functional correctness* (page 41) chapter, *formal verification of code* (page 58) is *hard*. Code needs to worry about a lot more things than specifications do. Our transfer model abstracted away everything from the specific packages we use to the “insufficient funds” dialog we show to users. That level of abstraction is what makes specification so powerful in the first place; verifying code loses that power.

(There *are* some specification languages that can “refine” spec into code, such as *Event-B*³³. These tend to be significantly more difficult and expensive to use, though.)

But the field of formal specification is young and we’re starting to see some interesting developments. The most exciting innovation, in my opinion, is using a formal spec to generate *tests*. You can see this in the paper *eXtreme Modeling in Practice*³⁴, where they used a TLA+ specification to generate a test suite for a C++ library.

10.5 Summary

1. TLA+ is a form of logic used to model software systems and express their properties.
2. You can *model check* a TLA+ specification to find timelines which break properties.
3. We can check that a property is true for all states in all timelines, or at least one state in each timeline.

The last two chapters covered two uses of and two tools for formal specification. But this is just the tip of the iceberg: specification is a rich field with all sorts of interesting languages and applications. I’ve worked with specification languages for modeling probability, robotics, system dynamics models, and even corporate bureaucracies!

Formal specification and formal verification together for *formal methods*, the discipline of directly applying math to write code. I’ve found formal specification languages (on abstract models) more useful to industry than formal verification languages (on actual code), mostly because it’s easier to learn and a lot cheaper to incorporate into a regular development workflow.

³³ <https://www.event-b.org/>

³⁴ <https://arxiv.org/abs/2006.00915>

In the next chapter, we will leave formal methods behind and focus on a different class of practical problems elegantly solvable with logic.

10.5.1 Further Reading

TLA+:

- [Learn TLA+](https://www.learntla.com)³⁵
- [Specifying Systems](https://lamport.azurewebsites.net/tla/book.html)³⁶ is the canonical textbook.

Case Studies:

- [Finding bugs without running or even looking at code](https://www.youtube.com/watch?v=FvNRlE4E9QQ)³⁷ [video]
- [Use of Formal Methods at Amazon Web Services](https://lamport.azurewebsites.net/tla/formal-methods-amazon.pdf)³⁸

³⁵ <https://www.learntla.com>

³⁶ <https://lamport.azurewebsites.net/tla/book.html>

³⁷ <https://www.youtube.com/watch?v=FvNRlE4E9QQ>

³⁸ <https://lamport.azurewebsites.net/tla/formal-methods-amazon.pdf>

Chapter 11

Solvers

Let's say we have a set of T tests, and each takes a different amount of time to run. T_1 might take 1 second, T_2 7 seconds, etc. We can divide the tests among N identical servers. We want to distribute them to minimize the *overall* testing time. If two servers take 1 second to run all of its tests and a third takes 27 seconds, the overall testing time is 27.

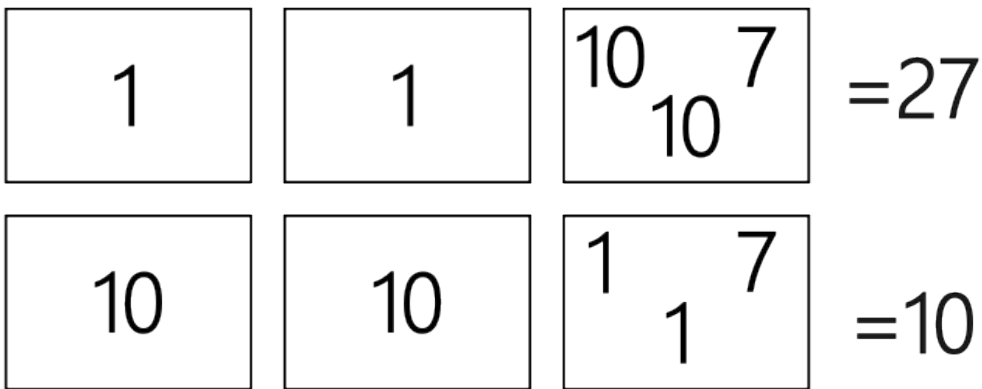


Fig. 11.1: Two different assignments with different overall test times.

To make the problem more interesting, some tests may belong to a *group*, and we want all tests in the same group to run on the same server.

This kind of problem is difficult to solve in a normal programming language, but it's very easy to express logically, and lets us use a new class of tools to solve them.

11.1 Logic

Normally, we use logic because it is more expressive than the average programming language. But this time we'll do things a little differently and write our problem in a *less* expressive way. Trust me, I have a reason for doing things this way.

First, how can we represent the test times? With a `test_times` array: `test_times[2]` is the time the *second* test takes. *Unlike everything else in this book*, this is 1-indexed.

Next, how can we represent the server assignments? If we say our servers are represented by integers (just like our tests) we can *also* represent the server with an

`array. assignment[5] == 2` means that test T5 is assigned to server 2.

Finally, we can represent the groups as- you guessed it- an array. `group[1] == 2` means that test T1 is in group 2. We'll also say there's no "group 0"; `group[2] == 0` instead means that T2 doesn't belong to any group.

Notice that two of those arrays are "constants": the `test_times` and the `groups` are fixed by outside forces. Only `assignment` is a "variable": we are looking to find its value that satisfies our constraints.

Note

This is a different meaning of "constraint" than the "constraints" in the database chapter. Different disciplines, different etymologies. If I was logically analyzing *both* databases and this class of problems at the same time, I'd probably call the database constraints "rules" or something.

The group constraint is simple:

```
constraint GroupsHaveSameAssignment =
  all t1, t2: 1..T:
    (group[t1] == group[t2] &&
     group[t1] != 0)
    => assignment[t1] == assignment[t2]
```

(Exercise for the student: why don't we need to check `group[t2] != 0`?)

The total time a server takes is the sum of all tests assigned to it.

```
total_time(s: 1..S) =
  sum({
    test_times[t]
    for t in 1..T:
      assignment[t] == s
  })
```

Our goal, then, is to minimize the maximum `total_time`.

```
minimize max({total_time(s) for s in 1..S})
```

11.2 In Practice: Solvers

Now that we have the problem logically represented, how do we solve it?

By using a solver, of course!

A *solver* is a special tool that finds answers to problems like ours. There are all sorts of solvers, some for special problems, some more general purpose.

We will use [Minizinc](https://www.minizinc.org/)³⁹, purely because [you can try it free online](https://play.minizinc.dev)⁴⁰. You can also download it and run it locally, which will be faster on most machines. Here is the solution in MiniZinc:

Listing 11.1: (Minizinc)

```
int: T = 10; % number tests
int: S = 2; % number servers
set of int: Servers = 1..S;
set of int: Tests = 1..T;

array[Tests] of int: test_times = [5, 6, 3, 7, 4, 3, 4, 4, 6, 9];
array[Tests] of int: group = [0, 0, 1, 1, 2, 0, 1, 0, 0, 2];
array[Tests] of var Servers: assignment;

function var int: total_time(var int: s) =
  sum([test_times[t] | t in Tests where assignment[t] = s]);

constraint forall (t1, t2 in Tests)
  (group[t1] = group[t2] /\ group[t1] != 0
   -> assignment[t1] = assignment[t2]);

function var int: num_assigned(var int: s) =
  count(assignment, s);

constraint forall (s1, s2 in 1..S)
  (abs(count(assignment, s1) - count(assignment, s2)) <= 1);

solve minimize max([total_time(s) | s in 1..S]);

output [
  "Server \s): \([t | t in Tests where assignment[t] = s])"
  ++ " (\(sum([test_times[t] | t in Tests
               where assignment[t] = s])))\n"
```

(continues on next page)

³⁹ <https://www.minizinc.org/>

⁴⁰ <https://play.minizinc.dev>

(continued from previous page)

```

    | s in 1..S
];

```

As with the other tools we have seen, MiniZinc uses its own syntax for logical expressions that is different from ours. I picked arbitrary values for `test_times` and `group`; MiniZinc can also read parameters in from a data file. We have to mark assignment as a `var` so MiniZinc knows it can control that.

Running this will output progressively better results, until it finds a minimum time of 26. If MiniZinc can't find any valid solution it will output UNSATISFIABLE.

Solvers make it easy to add more interesting constraints. For example, we can add a constraint that each server must have about the same number of tests:

```

constraint forall (s1, s2 in 1..S)
  (abs(count(assignment, s1) - count(assignment, s2)) <= 1);

```

The above is known as a “bin-packing problem”, which is one of the most popular use-cases for these solvers. Finding the optimum solution doesn't matter too much with ten tests, but a more realistic workload might be 10,000 tests! That's when a solver can really save a business money.

11.2.1 Speed vs Expressiveness

Our use of MiniZinc explains why we had to store our parameters in such an inexpressive format. MiniZinc does not support strings, structures, arrays of arrays, or any of the affordances we're used to in programming languages. Even with its restrictions, MiniZinc is still more expressive than many other solvers. This because solvers need to be *fast*. The more restrictions we place on the variables and constraints in our problems, the more specialized our solver can be, and in turn the faster we can solve expressible problems.

For example, a *linear programming solver* has only numbers for values, and can only compute expressions of the form $a \cdot x_1 + b \cdot x_2 + \dots$. But dedicated linear solvers can run much faster than a general MiniZinc problem.

[[So why not convert a MiniZinc problem into a linear programming one, if possible? In fact, that's exactly what it does. MiniZinc is a high-level, tool-agnostic language for expressing constraint problems, which it tries to “compile” into simpler forms.]]

There's a dizzying array of subclasses of problems and solvers, but two are of particular interest, showing what can exist on the very ends of the speed/expressiveness spectrum.

Note

Some classes of problems are valuable enough for specialist tools. [Google OR-tools](https://developers.google.com/optimization)⁴¹ is one of the most popular solvers available, and has specialized solvers for problems like scheduling, vehicle routing, and bin packing.

11.2.2 SAT Solvers

If *all* we cared about was speed, what is the most stripped down, barebones constraint language we can make?

We get it by removing *everything*. No strings, arrays, no functions, no numbers. Every variable is “true” or “false”. If we want to see whether test 7 is to server 3, we make a boolean variable tracking that. `a73` is true when test 7 is assigned to server 3, `!a73` when it isn’t. But then we’ll also need `a72`, `a71`, `a63`...

Then we’ll need a constraint saying “each test is assigned to at least one server.” What’s the *simplest possible* way to write that constraint? Probably something like this:

```
(a11 || a12 || a13 ...) &&
(a21 || a22 || a23 ...) &&
(a31 || a32 || a33 ...) &&
```

(This “AND of ORs” is conventionally called *Conjunctive Normal Form* (CNF).)

Looks good. But there’s nothing stopping `a11` and `a12` from both being true at the same time: test 1 is assigned to two different servers! So we also need to say “if test 1 is assigned to 1, it cannot also be assigned to 2 or 3”:

```
(a11 && !a12 && !a13 ...) ||
(!a11 && a12 && !a13 ...) ||
```

But now our solver has to understand *both* “AND of ORs” *and* “OR of ANDs”. That’s too much expressivity! Better find a way to rewrite that in CNF:

```
(!a11 || !a12) &&
(!a11 || !a13) &&
(!a12 || !a13) &&
...
```

⁴¹ <https://developers.google.com/optimization>

Exercise 47

Explain why this means test 1 can't be assigned to two different servers.

[Solution](#) (page 142)

Solvers for problems of this form are called *SAT solvers*, after “Boolean SATisfiability”. The syntax may seem constrained, but a surprisingly large number of problems can be transformed into SAT problems. And in return, SAT solvers are some of the fastest solvers in the world, routinely handling problems gigabytes in size.

Note

Maybe an exercise on converting the tag rule to SAT. We don't need to actually make the tags parameters, just replace them with clauses.

Note

The set of all problems solvable by a SAT solver is the “NP complexity class”, of P vs NP fame. NP is considered “intractable”, meaning we do not have efficient algorithms to solve all of the problems in the class. *Most* SAT problems seen in the wild are “well-behaved” and can be solved quickly. But you can also find small SAT problems that can't be solved in a human lifetime.

For this reason, many people use tools that take expressive forms and convert them into SAT. You can think of SAT as a low level “assembly” language that other tools “compile” to. *Alloy* (page 87) uses a SAT solver internally.

11.2.3 SMT

On the other end of expressivity, we have SMT, or *Satisfiability Modulo Theories* solver. While other solvers target a restricted category of math problem, SMT solvers are flexible and handle a wide range of different problems. As just one example, we can use it to reverse engineer a random number generator (RNG).

One old type of random number generator is the *Linear Congruential Generator*, or LCG. Starting with a seed value x_0 , each next value is determined by $x_{n+1} = (a \cdot x_n + c) \% m$, where (a, c, m) are all fixed values. Given a sequence and m , can we recover (a, c) ? The most popular SMT in use is *Z3*⁴².

⁴² <https://microsoft.github.io/z3guide/>

Listing 11.2: (Python)

```

# requires `pip install z3-solver`
from z3 import *
solver = Solver()

modulus = eval(input("Enter modulus: "))
sequence = eval(input("Enter sequence: ")) # Separate with commas

a = Int('a')
c = Int('c')

solver.add(a >= 0, a < modulus)
solver.add(c >= 0, c < modulus)

for i in range(len(sequence) - 1):
    solver.add(sequence[i+1] == c + (a * sequence[i]) % modulus)

if solver.check() == sat:
    model = solver.model()
    print(f"a = {model[a].as_long()}")
    print(f"c = {model[c].as_long()}")
else:
    print("Could not find parameters")

```

Here is what it looks like to run the code:

```

Enter modulus: 2**31
Enter sequence: 4096, 618876929, 113892918, 1048278319
a = 22695477
c = 1

```

SMT are more expressive than even “generic” constraint solvers, but that expressiveness comes at a price of *completeness*. All of the prior constraint problems we looked at were *decidable*, meaning the solver will either definitely return a value or definitely tell us there is no solution. SMT solvers can also return “UNKNOWN”, meaning the solver couldn’t figure if the problem is even solvable or not. Over time, SMT solvers are getting better and better at finding solutions, but they will never be able to solve all problems. Such is the price of expressiveness.

Nonetheless, SMT solvers are incredibly popular for their flexibility and see all sorts of different use cases. They can [crack cryptographic primitives](https://github.com/kste/cryptosmt)⁴³, [reverse engineer compiled binaries](https://docs.angr.io/en/latest/core-concepts/solver.html)⁴⁴, [find differences in firewall rulesets](https://github.com/Z3Prover/FirewallChecker)⁴⁵, and synthesize code

⁴³ <https://github.com/kste/cryptosmt>

⁴⁴ <https://docs.angr.io/en/latest/core-concepts/solver.html>

⁴⁵ <https://github.com/Z3Prover/FirewallChecker>

from specifications⁴⁶. They are also the engine that powers most work in theorem proving and *formal verification* (page 58). *Dafny* (page 58) uses an SMT solver to verify code, and one research model checker for *TLA+* (page 100) does too.

Note

How can a solver (which finds if solutions exist) power a verifier (which checks that a property always holds)? Easy, just exploit *quantifier duality* (page 19). If the property is all x : $P(x)$, then ask the solver to satisfy $\neg P(x)$. If the solver can't find any solutions, then $\neg(\text{some } x: \neg P(x))$ holds, which is equivalent to our property.

11.3 Which to use?

So given all of the options, which solver is the right one to pick? This depends on a lot of factors, but I can provide some very general heuristics.

First of all, most programmers are unlikely to directly use SAT. SAT is fast and powerful but it takes a lot of skill to represent problems in an optimal way, and to interpret a SAT solution to a problem. For these reasons, the main users of SAT solvers are people who 1) absolutely need the maximum possible performance on their problem, or 2) are building higher-level tools. We are more likely to use a tool that uses a SAT solver as part of its implementation.

After that, the right solver depends on the nature of our goal: is it *satisfaction* or *optimization*? SMT solvers are the right tool when the problem has very few solutions and any one will do. Other solvers are the right tool when the problem has many valid solutions, but some are more optimal than others. As a rough rule, most “satisfaction” problems are technical/“software engineering”-oriented, while most optimization problems are business/“operations research”-oriented: shift scheduling, vehicle routing, manufacturing optimization, resource allocation, etc.

If SMT is the right choice, then we default to Z3. It is most widely supported, has integrations with the most languages, and has the most thorough documentation of any SMT solver.

If an optimization solver is the right choice, the general approach is to find the *least expressive* class of tools that totally expresses the problem, as that will be the fastest. [[ILP, LP, Simplex, MIP, etc]]. There's a dizzying plethora of different tools and classes. A decent enough starting point is MiniZinc and Google's *OR-Tools*⁴⁷.

⁴⁶ <https://cseweb.ucsd.edu/~npolikarpova/publications/pop120.pdf>

⁴⁷ <https://developers.google.com/optimization>

11.4 Summary

- There are classes of problems that are difficult to express programmatically, but are solved efficiently with solvers.
- There is a tradeoff between how easily a solver can express constraint problems and how quickly it can solve them.
- SAT solvers are fast but very inexpressive. They are a core low-level component in a lot of higher-level software.
- SMT solvers are expressive but incomplete. They are also widely used, both directly and as low-level components.
- SAT and SMT solvers are primarily used for “satisfaction” problems, while other solvers are primarily used for “optimization” problems.

The next chapter is about the most literal kind of “logic programming”.

11.4.1 Further Reading

- Hakank’s [common constraint programming problems](http://www.hakank.org/common_cp_models/)⁴⁸ and [MiniZinc page](http://www.hakank.org/minizinc/)⁴⁹
- SAT/SMT by Example: <https://smt.st/main.html>
- Programming Z3: <https://theory.stanford.edu/~nikolaj/programmingz3.html>

⁴⁸ http://www.hakank.org/common_cp_models/

⁴⁹ <http://www.hakank.org/minizinc/>

Chapter 12

Logic Programming

In a book called “Logic for Programmers” I’ve somehow managed to not bring up “logic programming” for ninety pages. I feel like I deserve a medal.

Logic programming (LP) is a distinct paradigm of programming, like imperative and functional are. The most famous logic programming language is called “Prolog”. Prolog was first created in 1970s and since then has split off into many different variants. We will use the “SWI-Prolog” variant, which you can try online at <https://swish.swi-prolog.org/>. By necessity, this chapter will be even more of a broad overview than the other chapters.

12.1 Prolog

There are three basic building blocks of a Prolog program:

1. *Atoms* are value identifiers that start with a lower-case letter (bread, flour). These are ground symbols: bread is equal to bread and nothing else.
2. *Variables* are identifiers that start with a capital (X, Abc). The ultimate goal of a Prolog program is to “unify” variables to values that would make predicates true.
3. Predicates, which can be true for specific atoms, or for atoms that pass a condition. These predicates are called *facts* and *rules*, respectively.

```
% comments start with %  
ingredient(bread, flour). % don't forget the period!  
ingredient(bread, water).
```

This defines the fact `ingredient()` and determines it’s true if the first parameter is bread and the second is flour or water. Now if I call `ingredient` with a variable, I am *asking* the Prolog engine to find a value that makes my expression true.

```
ingredient(X, flour).  
X = bread % result  
  
ingredient(X, potatoes).  
false % no possible X
```

This maps directly to the some quantifier: `ingredient(X, flour)` is true if some `x`: `ingredient(x, flour)`. If multiple values satisfy the expression, then Prolog will return

possible values one at a time.

Representing recipes are not a common programming task, so let us continue with a more practical example. Few people work with recipes in their job, but almost everyone uses version control, and the relationships between commits is directly expressible in facts:

```
parent(a0, a1).
parent(a1, a2).
parent(a2, a3).
parent(a3, a4).
parent(a4, a5).
parent(a1, b1).
parent(b1, b2).
parent(b2, a4).
parent(b2, b3).
```

This represents the commit graph in Fig. 12.1.

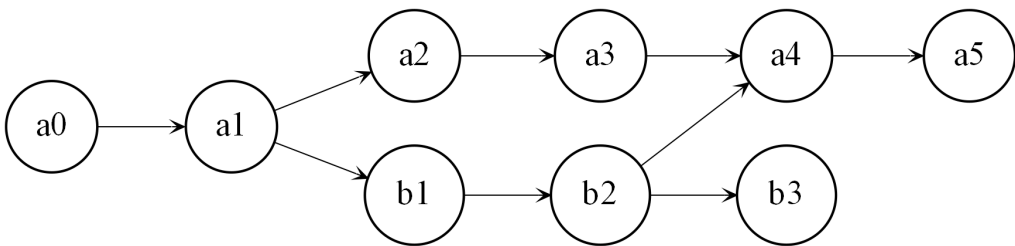


Fig. 12.1: A graph of commits.

Once we have a collection of facts, we can then add “rules”, or predicates with complex bodies. For example, a “merge commit” is one that has two different parents.

```
mergecommit(C) :-
    parent(P1, C),
    parent(P2, C),
    \+ (P1 = P2). % \+ is 'not'

?- mergecommit(C).
C = a4.
```

Rules can have multiple definitions, in which case the predicate is true if *any* rule is true. This makes it easy to express recursive statements, like “A is the ancestor of C if it is a parent of C or the parent of an ancestor of C.”

```

ancestor(A, Commit) :- parent(A, Commit).
ancestor(A, Commit) :-
    parent(A, Y),
    ancestor(Y, Commit).

```

With this, we can express complex queries, like “ancestors of commit A that are not ancestors of commit B”:

```

% \+ is "not"
?- ancestor(a5, X), \+ ancestor(b3, X).
X = a4 ;
X = a3 ;
X = a2

```

Note

Prolog uses a “backtracking” algorithm to find solutions. As such, it does not guarantee that all solutions are unique.

LP languages are general-purpose languages and can do everything that can be done in an imperative or functional language. The question is what can this paradigm do *better* than other programming paradigms? At one time, the answer was “artificial intelligence”, and logic programming was largely seen as the best tool for expert systems and natural language processing. This niche has been largely superseded by statistical methods like machine learning and large language models.

But there are still places where it sees use. Some specific use cases in the wild are listed in the “Further Reading” at the end of this chapter. And there are still some niches where logic programming is broadly the preferred approach.

12.2 Deductive Databases

A *deductive database* is an alternate form of database. Instead of storing data in tables, deductive databases store data as facts and rules. Logic programming then becomes purely a tool for querying, as opposed to general programming. Our previous commit model is arguably a deductive database. Adding new information to the commits is as easy as adding new facts:

```

% commit(id, author, [files_changed])
% written this way to be more compact
commit(a0, alice, [file(f1), file(f2), testfile(f2)]).

```

(continues on next page)

(continued from previous page)

```

commit(a1, bob, [file(f1), file(f3), testfile(f1)]).
commit(a2, eve, [file(f1), file(f2), testfile(f1), testfile(f2)]).

% commit_author_file
caf(C, A, F) :-
    commit(C, A, Files),
    member(F, Files).

```

file(f1) is a prolog *compound term*, equivalent to a struct or product type in other languages. It can be used in [[pattern matching]]: `caf(_, alice, testfile(X))` will retrieve any testing file (but not regular file) that Alice modified.

[[bridge]]

[[In their paper *Evidence Based Failure Prediction*, Nagappan *et al* argue that patterns in how we change files can predict the likelihood of bugs in those files. Files with more commits, “churn”, are more likely to have latent bugs. Let’s implement two rules that suggest a file is more likely to have bugs:]]

1. `high_churn` is true if a file was changed by at least three different commits with different authors. In our model, this applies to just `file(f1)`.
2. `untested_commit` is true if a file was changed in a commit, and its corresponding test was **not** changed. This check should *not* apply to test files. In our model, this applies to `file(f1)` and `file(f3)`.

```

high_churn(File) :-
    caf(_, A1, File), caf(_, A2, File), caf(_, A3, File),
    A1 @< A2, A2 @< A3. % @< = ordering on atoms

untested_commit(file(File)) :-
    commit(_, _, Files),
    member(file(File), Files),
    \+ member(testfile(File), Files).

```

We can write `high_churn` more elegantly (and not hardcode in the number of authors), but that requires more sophisticated Prolog technique. From here, we can map each file to the number of checks it fails.

```

% fails_check just calls the check on the file
% Added for descriptivity
fails_check(File, Check) :- call(Check, File).

checks_failed(_, [], []).
checks_failed(File, [Check|Checks], Failed) :-
    checks_failed(File, Checks, Failed),

```

(continues on next page)

(continued from previous page)

```

\+ fails_check(File, Check).

checks_failed(File, [Check|Checks], [Check|Failed]) :-
    checks_failed(File, Checks, Failed),
    fails_check(File, Check).

checks_failed(File, Failed) :-
    checks_failed(File, [high_churn, untested_commit], Failed).

```

If we just want the count of the checks each file failed, we can write another helper operator.

```

file_suspicion(File, Suspicion) :-
    checks_failed(File, Failed),
    length(Failed, Suspicion).

```

In practice, Prolog is rarely used as the query language for deductive databases for two reasons: Prolog cannot be embedded in other languages, and Prolog queries are not guaranteed to terminate. The main language is instead *Datalog*, a “well-behaved” subset of Prolog without these issues. [Datomic](https://www.datomic.com/)⁵⁰, for example, uses datalog for queries, but embeds it as a DSL in Clojure.

12.3 Constraint Logic Programming

TODO. Connect to answer set programming and package resolution.

12.4 Planning

There is one class of AI problems that (as of 2025) cannot be handled with statistical approaches: *planning*. Given a starting state, a set of valid actions, and a goal state, what sequence of actions should get us to the goal state?

Consider the following situation: we have a set of online servers that need two OS updates. We can only upgrade a server that is offline, and we need to make sure that we always have at least one server online. We can further abstract the servers so that they consist only of a name, a boolean on/off state, and a numerical version.

In this case, the starting state for two servers would be the set $\{(s1, \text{on}, 1), (s2, \text{on}, 1)\}$, the goal state would be $\{(s1, \text{on}, 3), (s2, \text{on}, 3)\}$, and there would be two possible

⁵⁰ <https://www.datomic.com/>

actions:

- Toggle the state of a server, unless doing so would leave all servers off
- Increment the version of an off server.

Prolog does not natively support planning, but my personal favorite logic language, [Picat⁵¹](http://picat-lang.org/), does. Here is the planning problem in Picat:

Listing 12.1: (Picat)

```
import planner, math, util.

final(N) =>
    foreach($server(_, State, Version) in N)
        State = on, Version = 3
    end.

cost(State) = 1.

% At least one server online
valid(State) => member($server(_, on, _), State).

action(From, To, Action, Cost) ?=> % toggle state
    member(X, From),
    (
        Action = $off(X[1]), To = replace(From, X, X.replace(on, off));
        Action = $on(X[1]), To = replace(From, X, X.replace(off, on))
    ),
    valid(To),
    Cost = cost(From).

action(From, To, Action, Cost) => % increment version
    member(X, From),
    X = $server(Name, off, Version),
    To = replace(From, X, X.replace_at(3, Version+1)),
    Action = $upgrade(Name),
    Cost = cost(From).

main =>
    Start = [$server(s1, on, 1), $server(s2, on, 1)],
    best_plan(Start, Plan, Cost),
    writeln(Plan), writeln(Cost).
```

Running this gives me:

⁵¹ <http://picat-lang.org/>


```
[off(s1), upgrade(s1), upgrade(s1), on(s1),
  off(s2), upgrade(s2), upgrade(s2), on(s2)]
8
```

I coded the first output to be the list of steps that solves our problems. The second output is the “cost” of the plan, which Picat will automatically minimize. In this case, each action has cost 1, meaning to total cost is just the number of steps in our plan.

The planner is automatically able to find a sequence of steps that solves our problem. It can also minimize cost. In this case, the “cost” is just the number of steps, leading to an eight-step solution. To showcase we can add an additional penalty for having several online servers with different versions, equal to $(\text{MaxVersion} - \text{MinVersion})$ cubed.

Our program looks very similar:

```
- cost(State) = 1.
+ cost(State) = Out =>
+   member($server(_, on, Vmin), State).minof(Vmin),
+   member($server(_, on, Vmax), State).maxof(Vmax),
+   Out = 1 + max(0, Vmax - Vmin)**3.
```

With these changes, the eight-step solution would have a total cost of 16. Picat, instead, finds a longer solution with a smaller cost:

```
[off(s1), upgrade(s1), on(s1),
  off(s2), upgrade(s2), upgrade(s2), on(s2),
  off(s1), upgrade(s1), on(s1)]
12
```

Planning is mostly used in AI research and *especially* in video game AIs, where it is called *Goal-Oriented Action Planning*.

12.5 Summary

- Logic programming express programs as predicates and allow users to find values that match those predicates. The most famous LP language is *Prolog*.
- LP can also be used for querying data in so-called “deductive databases”. The most famous LP query language is *Datalog*.
- Planner programming find sequences of actions that change a starting state into a goal state. One such planning language is *Picat*.

12.5.1 Further Reading

General Topics:

- Association for Logic Programming: <https://logicprogramming.org/>
- The Power of Prolog: <https://www.metalevel.at/prolog>
- Logic Programming Courseware: <https://athena.ecs.csus.edu/~mei/logicp>

Other Logic Programming Languages:

- miniKanren: <https://minikanren.org>
- Datalog: <https://www.learnatalogtoday.org/>
- Picat: <http://picat-lang.org/> and <http://picat-lang.org/picatbook2015.html>
- Answer set programming: <https://potassco.org/>

LP Case Studies:

- IBM Watson used Prolog for natural language processing: https://www.cs.miami.edu/home/odelia/teaching/csc419_spring19/syllabus/IBM_Watson_Prolog.pdf
- The JVM uses Prolog in the typechecker: <https://docs.oracle.com/javase/specs/jvms/se10/jvms10.pdf>
- The Pubgrub package resolution algorithm uses Answer set programming: <https://github.com/pubgrub-rs/pubgrub>

Appendix A

Math Notation

I used programmer symbols and my own syntax through this book; mathematicians use different symbols. I did this because I wanted everything to be easily greppable and inferable from context. If you’re seeing $\cup$ for the first time, it’s really hard to look up what it means!

A.1 Basic Logic Symbols

Table 1.1: Symbols

English	Book	Math
And	<code>&&</code>	$\wedge$
Or	<code> </code>	$\vee$
Not	<code>!</code>	$\neg$
Implies	<code>=></code>	$\Rightarrow$ (or $\rightarrow$)
If-and-only-if	<code><=></code>	$\Leftrightarrow$
Forall	<code>all x</code>	$\forall x$
Exists	<code>some x</code>	$\exists x$
in	<code>in</code>	$\in$
Union	<code> </code>	$\cup$
Intersection	<code>&</code>	$\cap$
Subset	<code>subset</code>	$\subset$
Cardinality	<code>#S</code>	$ S $

Table 1.2: Sets

English	Book	Math
Integers	<code>Int</code>	$\mathbb{Z}$
Naturals	<code>Nat</code>	$\mathbb{N}$
Power set of S	<code>power_set(S)</code>	2^S

Table 1.3: Temporal Logic

English	Book	Math
Next value of x	x'	x'
Always	$[]$	$\square$
Eventually	$\langle \rangle$	$\diamond$

A.2 Quantified Expressions

Take the expression all x in set: $P(x)$. Here are three different ways mathematicians write it:

$$\forall x \in \text{set}: P(x)$$

$$\forall x. \text{set}(x) \rightarrow P(x)$$

$$\forall x: \text{set} | P(x)$$

Some mathematicians write $\exists!x$ to mean “there exists exactly one x ”, but it’s not by any means a universal convention.

A.3 Tautologies

I wrote the double negative rewrite rule as $!!P = P$. To be more mathematically precise, given $!!P$, we can prove P . Three ways you could write this:

$$\frac{\neg\neg P}{\therefore P}$$

$$\neg\neg P \vdash P$$

$$\neg\neg P \rightarrow P$$

Which one a mathematician uses can depend on the particular field they publish in. In the last case, they will reserve $\rightarrow$ to mean “we can prove” and exclusively use $\Rightarrow$ to mean “implies”.

Appendix B

Useful Rewrite Rules

B.1 Table of Tautologies

Some of these are $=$, to mean the two formulas are identical- you can substitute one for the other. Some are $\Rightarrow$, meaning **they only go one way**.

Propositional Logic:

$$P = \neg\neg P$$

$$P \ \&\& \ \neg P = \text{False}$$

$$P \ || \ \neg P = \text{True}$$

De Morgan's law:

$$\neg P \ \&\& \ \neg Q = \neg(P \ || \ Q)$$

$$\neg P \ || \ \neg Q = \neg(P \ \&\& \ Q)$$

$$\neg P \ \&\& \ Q = \neg(P \ || \ \neg Q)$$

B.1.1 Implication

Definition:

$$P \Rightarrow Q = \neg P \ || \ Q$$

$$\neg(P \Rightarrow Q) = P \ \&\& \ \neg Q$$

Contrapositive:

$$P \Rightarrow Q = \neg Q \Rightarrow \neg P$$

$$(P \Rightarrow Q) \ \&\& \ (Q \Rightarrow P) = (P = Q)$$

Transitivity: $(P \Rightarrow Q \ \&\& \ Q \Rightarrow R) \Rightarrow (P \Rightarrow R)$ Note it's *not* an $=$! It doesn't go both ways!

B.1.2 Quantifiers

Extraction:

$$\text{all } x: P \ \&\&/\ || \ Q(x) = P \ \&\&/\ || \ \text{all } x: Q(x)$$

Duals:

$$\text{all } x: P(x) = \neg(\text{some } x: \neg P(x))$$

$$\text{some } x: P(x) = \neg(\text{all } x: \neg P(x))$$

$$\text{all } x: \neg P(x) = \neg(\text{some } x: P(x))$$

$$\text{some } x: \neg P(x) = \neg(\text{all } x: P(x))$$

Commutativity:

$$\text{all } x \text{ in } S: (\text{all } y \text{ in } T: \dots) =$$

$$\text{all } y \text{ in } T: (\text{all } x \text{ in } S: \dots) =$$

$$\text{all } x \text{ in } S, y \text{ in } T: \dots$$

$$\text{all } x \text{ in } S, y \text{ in } S: \dots =$$

$$\text{all } x, y \text{ in } S: \dots$$

$$\text{some } x \text{ in } S: (\text{some } y \text{ in } T: \dots) =$$

$$\text{some } y \text{ in } T: (\text{some } x \text{ in } S: \dots) =$$

$$\text{some } x \text{ in } S, y \text{ in } T: \dots$$

$$\text{some } x \text{ in } S, y \text{ in } S: \dots =$$

$$\text{some } x, y \text{ in } S: \dots$$

some x: all y can be replaced with all y: some x, which is *stronger*. You cannot go the other way!

With other stuff

Distributivity:

$$\text{some } x: P(x) \ || \ Q(x)$$

$$\text{some } x: P(x) \ || \ \text{some } x: Q(x)$$

$$\text{all } x: P(x) \ \&\& \ Q(x)$$

$$\text{all } x: P(x) \ \&\& \ \text{all } x: Q(x)$$

Appendix C

Beyond Logic

Note

EVERYTHING in the section needs to be thoroughly checked against a mathematician. Also it might be thrown out entirely if it's too long and not helpful.

Under construction.

This entire book is about classical first order logic. That's the logic that most mathematicians use to do math. But *mathematics is flexible* and mathematicians hate taking a system for granted. So many mathematicians have asked "what happens if we make logic different?"

This is some of about those ways of making logic different.

C.1 The Limit: Russell's Paradox

For every set s , we can create a predicate $S(x) = x \in s$. This means every set defines a predicate. Is the opposite true: for every predicate, can we find a set of all things that pass that predicate?

```
CanRunProgram(c) = RAM(c) && (CPU(c) || GPU(c))
```

```
->
```

```
CanRunProgram = RAM & (CPU | GPU)
```

In *naive set theory*, this is true for all sets. Naive set theory has a problem, though, that lead to mathematicians abandoning it over a century ago. Consider the predicate $\text{Evil}(x) = x \text{ not in } x$, aka x is not a set that contains itself. Most sets would pass this predicate: $\{\}$, $\{1\}$, $\{1, 3\}$, $\{abc\}$, etc. Some sets would fail this predicate, like "the set of all non-empty sets". If all predicates formed sets and vice versa, we'd have a set evil , the set of all sets that don't contain themselves.

Now is $\text{Evil}(\text{evil})$ true? If so, evil doesn't contain itself, meaning it's not in evil , so $\text{Evil}(\text{evil})$ is false by definition, but then it's not in evil , meaning it doesn't contain itself, meaning $\text{Evil}(\text{evil})$ is true...

This is called "Russell's paradox" and is considered a reason that not all predicates form sets. The paradox drove much of the development of formal logic in the early

20th century in order to find ways to avoid the paradox. The most mainstream solution to this is called “ZF” Set Theory, but another is modern type theory, which has found a home in modern functional pro

C.2 Higher Order Logic

Early in the book I wrote

So as to prevent eldritch math horrors, predicates *cannot* be in the domain of discourse: there are no predicates that take other predicates.

This makes our logic a *first-order logic*. In a *higher-order logic*, predicates can be both passed as values and used as quantifiers:

$$\text{Symmetric}(P) = \text{all } x, y: P(x, y) == P(y, x)$$

Most mathematicians prefer to stick with first-order logic because higher order logic is too “powerful”. As one logician I interviewed put it, “you don’t want your logic suddenly building a rocket ship.” A rough analogy would be to Turing completeness in computer science. It’s harder to analyze a Turing complete language than a more limited one.

[[Applications: type theory, theoretical computer science I think]]

C.3 Constructive Logic

At the very very beginning of the book, I said “all predicates return true or false”. This is the *Law of Excluded Middle*: any statement is true or false. There is no “third thing” a statement can be.

This leads to something unusual: if you want to prove something true, you have the option of proving it “non-false”. And if you want to prove a set is nonempty, you can instead prove that it’s impossible for the set to be empty. This is called a “non-constructive proof”, since you aren’t actually constructing a value inside that set.

My favorite example of this is Chess. Imagine we are playing a slight variation where, instead of White always moving first, they can choose whether to move first or second. With this extra rule, we can trivially prove that White has a foolproof strategy to *always* win or tie:

Assume White doesn’t have a foolproof strategy. Then, assuming perfect play, Black always wins. But then White can pass on their first turn, making them effectively the

second player, and then follow the winning strategy for Black. This means that Black doesn't have a winning strategy, which means White must have a foolproof one.

It's an elegant and watertight proof. It also gives us zero information on *what the foolproof strategy actually is*, and so chess remains stubbornly intractable.

Non-constructive proofs bother some mathematicians, who proposed an alternate form of logic called *Constructive Logic*. Constructive logic doesn't have the law of excluded middle, nor does it have double-negation: you can't replace $\neg\neg P$ with P . These two removals mean that writing proofs and manipulating statements is harder. But in return, it guarantees the only way to prove the existence of something is to actually find an example.

Another consequence of constructive logic is that without excluded middle, implication can work a little more like it does in normal language. The statement "if I was named Greg, then I'd be king of England" is mathematical true in classical logic (I'm not named Greg), but it's not a true statement in day to day life. And it doesn't have to be in constructive logic, either.

Functional programming-style type systems are constructive by nature. A function of type $\text{Int} \rightarrow \text{Bool}$ is a "proof" that if integers exist, booleans do too, because there's a way to turn an example of an integer into an example of a boolean.

C.4 Modal Logic

Predicate logic augments booleans with statements over *quantity*: is statement P true for all elements of a set, or true for at least one? *Modal logic* instead augments booleans with statements of *quality*: is statement P true "necessarily", or "possibly"?

For example, say we weigh something and find it is 100 grams, but our scale has an uncertainty error of 0.5 grams. The true weight is necessarily less than 101 grams, and possibly less than 99.9 grams. Necessarily and possibly are duals, so "possibly P " is the same as "not necessarily not P ".

Beyond that, what "necessarily" and "possibly" mean are vague, which in turn means that there's many different modal logics. Philosophers use modal logic to explore the nature of knowledge, morality, uncertainty, and many other things.

But the most important modal logic is one you've already seen: the mode of *time*. *TLA+'s* (page 100)'s *always* and *eventually* is just "necessarily" and "possibly"!

Modes are independent of quantifiers (all and some): you can have modes, quantifiers, or both.

Appendix D

Answers to Exercises

Answer to Exercise 1 Implication

1. $\neg(\text{Native}(p) \ \&\& \ (\text{Q}(p) \ || \ \text{R}(p))) \ || \ ((\text{RAM}(c) \ \&\& \ \text{CPU}(c)) \ || \ \text{GPU}(c))$
2. $\text{Native}(p) \ \&\& \ (\text{Q}(p) \ || \ \text{R}(p)) \Rightarrow (\text{RAM}(c) \ \&\& \ \text{CPU}(c)) \ || \ \text{GPU}(c)$

I personally find (2) much easier to read, since we don't have as many nested expressions.

Answer to Exercise 2

```
CanRunProgram(c, p) =  
  Native(p) => (RAM(c, p) && CPU(c, p)) || GPU(c, p)
```

Answer to Exercise 3

1. $(\text{Native}(p) \Rightarrow \neg \text{Web}(p)) \ \&\& \ (\text{Web}(p) \Rightarrow \neg \text{Native}(p))$
2. $\neg(\text{Web}(p) \ \&\& \ \text{Native}(p))$
3. $\neg \text{Native}(p) \ || \ \neg \text{Web}(p)$

Answer to Exercise 4 Implication as conditional

1. $(c \Rightarrow x) \ \&\& \ (\neg c \Rightarrow y)$ is equivalent to $(\neg c \ || \ x) \ \&\& \ (c \ || \ y)$. If you work through the cases, you should find that `IfThenElse` is true when c is true and x is true, or when c is false and y is true.
2. As hinted by the name `IfThenElse` is simulating a conditional. We can also write it like this:

```
IfElse(c: Bool, x: Bool, y: Bool) =  
  if c then x else y
```

Answer to Exercise 5 Sets vs Predicates

$\text{CanRunProgram} = (\text{RAM} \ \& \ \text{CPU}) \ | \ \text{GPU}$

Answer to Exercise 6 Disjoint Sets

$\text{Child} \ \& \ \text{Adult} == \{\}$. Another way would be $\text{Child} - \text{Adult} == \text{Child} \ \&\& \ \text{Adult} - \text{Child} == \text{Adult}$.

Answer to Exercise 7 Symmetric Difference

One way is $(S - T) \mid (T - S)$; another is $(S \mid T) - (S \ \& \ T)$.

Answer to Exercise 8

If not a single developer has reviewed the pr, then `EveryoneApproved` is true (all zero reviewers approved!) while `SomeoneReviewed` is false (nobody reviewed it).

In general, all x in $\{\}$: $P(x)$ is always true (regardless of what P is) and some x in $\{\}$: $P(x)$ is always false.

Answer to Exercise 9

1. all x in Nat : $x < x + 1$
2. all x in Nat : $0 \leq x$

Answer to Exercise 10 Nested Quantifiers

1. all pr in PR : some d in Developer : `ApprovedBy(pr, d)`
2. some d in Developer : all pr in PR : `ReviewedBy(pr, d)`

Answer to Exercise 11

all a, b in Int : $a > b \Rightarrow a..<b = a..=b = \{\}$.

Answer to Exercise 12

```
{x in Int: 1 <= x && x <= 100}
```

Answer to Exercise 13 Divides

```
IsDivisibleBy(num, divisor) =
  some x in 1..=num:
    x*divisor = num
```

Answer to Exercise 14

some $x, y: x \neq y \ \&\& \ P$

Answer to Exercise 15

```
all x, y, z:
  (1. x != y
   2. y != z
   3. z != x
  ) => P(x, y, z)
```

A pretty good argument for adding disj!

Answer to Exercise 16

all $x: P(x)$

Answer to Exercise 17

Here are two I came up with:

1. “All days this week is warm and sunny” is the same as “all days [this week] are warm and all days are sunny”.
2. “Someone has blue eyes or green eyes” is the same as “someone has blue eyes or someone has green eyes.”

Answer to Exercise 18

There are many answers, here are just two:

1. “All people are (alive or dead)” is true, “(all people are alive) or (all people are dead)” is false.
2. “Someone is alive and someone is dead” is true, “Someone is alive and dead” is false.

Notice that each of them *can* go one way. You *can* rewrite “all rocks are blue or all rocks are brown” into “all rocks are blue or brown”, but *not the other way around*.

Answer to Exercise 19 Contrapositives

First rewrite it was $!P \parallel Q$. Then replace Q with $!(Q)$ to get $!(Q) \parallel !P$. Then rewrite that as $!Q \Rightarrow !P$.

Answer to Exercise 20 Rewriting ifs

Starting with our conditional:

1. if P then Q else R (initial condition)
2. $P \Rightarrow Q \ \&\& \ !P \Rightarrow R$ (definition of if)
3. $\!P \Rightarrow R \ \&\& \ P \Rightarrow Q$ ($\&\&$ is commutative)
4. $\!P \Rightarrow R \ \&\& \ !(P \Rightarrow R)$ (double negation)
5. if !P then R else Q (definition of if)

Answer to Exercise 21 Your language's quantifiers

Python and Haskell use `all()` and `any()`. Javascript uses `Array.every()` and `Array.some()`. C++ has `std::all_of()` and `std::any_of()`, and also has `std::none_of()`.

Answer to Exercise 22

In $\text{all } x \text{ in set: } P(x) \ \&\& \ \text{some } x \text{ in set: } P(x)$, the only thing that the `some` is doing is checking that the set is nonempty, as that's the only case where all x can be true and some x can be false. So we can rewrite the code to not have that:

```
return l != [] and all(P(x) for x in l)
```

Answer to Exercise 23

```
x <= 1 || x > 10
```

Answer to Exercise 24 Implication via filtering

In $\text{all } x \text{ in set: } P(x) \Rightarrow Q(x)$ we only check $Q(x)$ on the elements that also satisfy $P(x)$. In $\text{all } x \text{ in } \{x \mid \text{in set: } P(x)\}: Q(x)$ we filter out all of the elements that don't satisfy $P(x)$ and then check $Q(x)$ on the rest. Equivalence follows.

Answer to Exercise 25 Partial Ordering

Recall that P tests that the max of $[1,2,3]$ is 3, while R tests that max values of $[1, 2, 3]$ and $[0, 1, -1]$ are ≥ 0 .

1. To pass R and not P , write a `max(l) = 1`. To pass P and not R , write a `max` that just returns the last value of the input.
- 2.

```
T =
  1. max([1, 2, 3]) == 3
  2. max([0, 1, -1]) == 1
```

This fails both buggy max implementations given above.

3. “T is as strong as P and R” is $T \Rightarrow P \ \&\& \ R$. Since $P \Rightarrow Q$, $T \Rightarrow Q$ too, meaning T is as strong as Q.

Answer to Exercise 26 The Flaw with False

test false will reject any buggy implementation of max... but it will also reject a correct implementation! What makes a given test “a test of max” is that it will pass for a correct implementation, meaning false isn’t a test of max at all (and cannot be stronger than them).

By contrast, test true *is* a valid test of max, and in fact the weakest possible test.

Answer to Exercise 27 Uniqueness

```
IsUnique(l):
    all x, y in 0..

```

Or, using disj, we could write all disj x, y instead and skip the condition.

Answer to Exercise 28 Property Testing Find

In python:

```
@given(s.lists(s.integers()), s.integers())
def test_myfind(l, x):
    out = myfind(l, x)
    if out == -1:
        assert x not in l
    else:
        assert l[out] == x
        assert x not in l[0:out]
```

Note that this will statistically overtest the case where x is not in l. Part of learning to use PBT well is getting a sense of how to best generate inputs. The techniques here are beyond the scope of this book.

Answer to Exercise 29 [[Defensive Programming]]

In the new version of `max_avail_price` it no longer requires “there is at least one available item”. If there are no available items, we never call `max` anyway, so don’t violate its requirements.

On the other hand, `max_avail_price`’s *postconditions* get more complicated. *If* the return value is a number, it still is the highest price of an available item. If the return value is `None`, then there were no available items. So the new contract is this:

```
max_avail_price(items) returns o
helpers:
  available = 'list of available items in items'
requires:
  NOTHING AT ALL
ensures:
  o == None => all i in Item: !i.available
  o != None =>
    'output is priciest available item':
      some i in available:
        1. i.price = out
        2. all i2 in available: i2.price <= i.price
```

Answer to Exercise 30 Fun with square roots

```
1.  function: sqrt(x) -> o
    requires: x >= 0
    ensures: o*o = x
```

2. I’ll do this in something like Python.

```
# requires: a != 0
# requires: b^2 >= 4ac
def quadratic(a, b, c):
    lhs = -b / (2*a)

    # requires: b^2 >= 4ac
    # requires: a != 0
    rhs = sqrt(b**2 - 4*a*c) / (2*a)

    return (lhs + rhs, lhs - rhs)
```

3. There’s one thing I left out of the functional specification in part (1): *sqrt guarantees* the output is nonnegative, too.

```

function: sqrt(x) -> o
requires: x >= 0
ensures: o >= 0
ensures: o*o = x

```

Then the code becomes

```

x = 5
# requires (a): x >= 0
y = sqrt(x)
# ensures (b): y >= 0
# requires (b): y >= 0
z = sqrt(y)

```

So the requirement is satisfied.

Answer to Exercise 31 A Square is not a Rectangle

The issue is that we left out the ensurances of `Rectangle.setWidth`:

```

# ensures: width == x
# ensures: length == old(length)
Rectangle.setWidth(x)

```

In other words, `setWidth` ensures it *only* changes the width; it does not change the length. `Square.setWidth` doesn't have stronger postconditions, they do not imply `Rectangle.setWidthPost`. This problem goes away if we make the classes immutable. In general, implementing a mutable abstraction is harder than implementing a mutable one.

Answer to Exercise 32 A Missing Ensurance

It follows from the preconditions: $x \geq 0$ and $y > 0$ means that $x / y > 0$, and `floor` can't make a positive number into a negative one. Since $q == \text{floor}(x / y)$, $q \geq 0$.

Answer to Exercise 33 Exactness is not Validity

One simple example:

Table 7.3: Invalid

P?	out
T	T
T	F

The table has two rows, but is unsound (two contradictory inputs) and incomplete (missing an input). This means it is invalid.

Answer to Exercise 34 Fizzbuzz

$x \% 3 == 0?$	$x \% 5 == 0?$	out
T	T	"fizzbuzz"
T	F	"fizz"
F	T	"buzz"
F	F	x

Answer to Exercise 35

Z	W	H	M
T	-	-	T
F	webcam	-	F
F	desk	T	T
F	desk	F	F

(I fixed it by disabling the webcam mic in the OS settings)

Answer to Exercise 36 Cartesian Cardinalities

First of all, if S has s elements and T has t elements, $\#S * \#T == s*t$. Since the cardinality of the set does not depend on *what* elements it has, only the number of elements, I can safely assume that $S = 1..=s$ and $T = 1..=t$. Now, I just need to show that $(1..=s) \times (1..=t)$ has $s*t$ elements. To do this, I will put all of the elements in a grid:

```

(1, 1) (1, 2) ... (1, t)
(2, 1) (2, 2) ... (2, t)
.
.
(s, 1) (s, 2) ... (s, t)

```

This grid has s rows and t columns, so it has $s*t$ elements.

Answer to Exercise 37 Compound keys

There's two ways we can do this. The first is to write it all as one predicate:

```

constraint
  all disj ug1, ug2 in user_groups:
    ug1.user_id != ug2.user_id ||
    ug1.group_id != ug2.group_id

```

But the way I'd prefer to do it instead would be to first write a helper operator, and then use that in the actual constraint.

```

SameUserAndGroup(ug1, ug2: user_groups) =
  1. ug1.title = ug2.title
  2. ug1.author = ug2.author

constraint
  all disj ug1, ug2 in user_groups:
    !SameUserAndGroup(ug1, ug2)

```

Answer to Exercise 38

```

constraint UsersInGroupsHaveEmail =
  all ug in user_groups, u in users:
    ug.user_id = u.id => u.email != NULL

```

Answer to Exercise 39

```

MemberOf(u, g) =
  some ug in user_groups:
    1. ug.user_id = u.id
    2. ug.group_id = g.id

constraint MaxFiveMembers =
  all g in groups:
    #{u in User: MemberOf(u, g)} <= 5

```

Answer to Exercise 40 Transition Helper

```

ValidTransitions(task, from, to) =
  t.status = from => t.status' in (to | {from})

```

(We have to wrap from in braces because you can't union a set and a string, only a set and another set.)

Answer to Exercise 41

```
constraint all u, ref in Users:
  u.referrer = ref => u.created_at > ref.created_at
```

Answer to Exercise 42 No valid behaviors

At every step, Alice *must* transfer at least one dollar to Bob. Eventually there is some t where $\text{alice}[t] == 0 \ \&\& \ \text{bob}[t] == 20$. Then Alice can't make a transfer, $\text{Transfer}(t)$ is false, and so Spec is false.

Answer to Exercise 43 Extending to Bob

We can rename $\text{Transfer}(t)$ to $\text{TransferAliceToBob}(t)$, write the converse as a new predicate, and then add it to next.

```
TransferBobToAlice(t: Time) =
  some value in 1..=bob[t]:
    1. alice[t+1] == alice[t] - value
    2. bob[t+1] == bob[t] + value

Next(t) =
  || TransferAliceToBob(t)
  || TransferBobToAlice(t)
```

Now, can Alice and Bob transfer to each other in the same step? No. Let's say they both start with 10 dollars and each try to transfer five dollars to each other. By $\text{TransferAliceToBob}$ we have:

1. $\text{alice}[1] == \text{alice}[0] - 5 == 5$
2. $\text{bob}[1] == \text{bob}[0] + 5 == 15$

And by $\text{TransferBobToAlice}$, we have:

1. $\text{bob}[1] == \text{bob}[0] - 5 == 5$
2. $\text{alice}[1] == \text{alice}[0] + 5 == 15$

So now we have $\text{alice}[1] == 5 \ \&\& \ \text{alice}[1] == 15$, which is always false.

Answer to Exercise 44 Stuttering with Primes

```
Stutter =
  1. alice' == alice
  2. bob' == bob
```

Answer to Exercise 45 Always rules

For (1), we have:

1. $\text{``}[](\text{all } x: P(x))\text{''}$
2. ``all } t \text{ in Time: all } x: P(t, x)\text{''}$  (definition of  $\text{``always'')
3. ``all } x: \text{all } t \text{ in Time: } P(t, x)\text{''}$  (commutivity of  $\text{``all'')
4. ``all } x: []P(x)\text{''}$  (definition of  $\text{``always'')

(2) is solved similarly, except instead using distributivity instead of commutivity.

Answer to Exercise 46 Eventually rules

For (1), we have

1. $\langle \rangle \text{some } x: P(x)$
2. $\text{some } t \text{ in Time: some } x: P(t, x)$ (definition of $\langle \rangle$)
3. $\text{some } x: \text{some } t \text{ in Time: } P(t, x)$ (commutivity of some)
4. $\text{some } x: \langle \rangle P(x)$ (definition of $\langle \rangle$)

(2) is solved similarly, except instead using distributivity instead of commutivity.

(3) is solved with duality:

1. $\langle \rangle P$
2. $\text{some } t \text{ in Time: } P(t)$
3. $\text{!(all } t \text{ in Time: } !P(t))$ (duals)
4. $![]P$

Answer to Exercise 47

There's a couple of ways to show this. The first is to say that if test 1 is assigned to server N, for any other server M we have the clause $(!a1N \parallel !a1M)$. Since we already have $a1N$, the only way for the clause to be true is if $!a1M$, ie test 1 isn't assigned to M.

Another way to see this is to rewrite $(!a11 \parallel !a12) \&\& (!a11 \parallel !a13)$ as $a11 \Rightarrow !a12 \&\& a11 \Rightarrow !a13 \dots$, which is equivalent to $a11 \Rightarrow !a12 \&\& !a13 \&\& \dots$.

Index

Non-alphabetical

' (*prime*), 69
- (*set*), 11
- (*table*), *see any*
& (*set*), 11
&& (*and*), 6
=> (*implies*), 8
`backticks` (*in predicates*), 5
| (*set*), 11
|| (*or*), 6

A

action, 99
all, 14
Alloy, 87
any (*table*), 64
assert, 41
assertion, 41

B

behavior, 95

C

cartesian product, 76
complete, 65
Conjunctive Normal Form, 112
contract, 43

D

Dafny, 58
datalog, 119
Decision Table, 63
deductive database, 119
disj, 18
Domain of Discourse, 10

E

ensures, 42

F

Formal Specification, 86
Formal verification, 58
fuzzing, 38

I

in (*set*), 10
incomplete, *see complete*

L

liveness, 104
Logic programming, 117
loop invariant, 55

M

Metamorphic properties, 39
Minizinc, 110
MISU, 47

P

planning, 121
PlusCal, 101
postcondition, 42
power set, 11
precondition, 42
predicate, 5
proof, 53
Property-Based Testing, 36

Q

quantifier, 12, 13
 scoped quantifier, 13

R

refinement, 90
Relational Model, 73
requires, 42
rewrite rules, 18

S

safety, 104
set, 10
set filter, 12
set map, 12
SMT, 113
some, 13
sound, 65
specification

total, 34
subset, 11

T

Temporal Logic, 98
Theorem, 20
TLA+, 100
truth table, 7

U

unsound, *see* sound

X

$\times$ (*sets*), *see* cartesian product